

# Must-Know Patterns for Placement & Competitive Programming — Volume 2

---

**Target Audience:** SDE placements, online assessments, competitive programming (CP) **Language:**

C++ 17 (with Python equivalents)

**Goal:** Complete revision from one file — intuition, algorithm, dry-run, code, patterns, edge cases, interview prep

---

## 1. TRIE

---

### 1. Overview

A **Trie** (prefix tree) is a tree-like data structure used to store a dynamic set of strings. Each node represents a single character of a string, and the path from the root to a node spells out a prefix. Unlike a binary search tree, nodes in a trie share common prefixes — this makes it extremely efficient for prefix-based operations.

### 2. Intuition

**Simple explanation:** Think of a dictionary where words are organized by their first letter, then second letter, and so on — like how phone contacts autocomplete names as you type. A trie does exactly this: every node has children (one per possible next character), and we walk character-by-character.

**Analogy:** Imagine a filing cabinet. The root is the entrance. The first drawer is labeled 'a', inside it a sub-drawer 'b', then 'c' — the path **a** → **b** → **c** spells "abc". If another word starts with "ab" ("abacus"), it shares the first two drawers.

**Why it works:** By sharing common prefixes, a trie reduces redundant storage and allows  $O(L)$  lookup for any string, where  $L$  is the string length — independent of how many strings are stored. No hash collisions, no rehashing.

### 3. When to Use It

- **Autocomplete / prefix search** — "find all words starting with prefix P"
- **Dictionary word validation** — "is this word valid?" (spell checker)
- **Lexicographical sorting** — DFS on a trie yields sorted order
- **Longest common prefix** of a set of strings
- **Word break problems** — "can the string be segmented into dictionary words?"
- **IP routing (longest prefix match)** — network routers use binary tries (Patricia tries)
- **Phone directory / contact search** — prefix-based suggestions
- **Counting distinct substrings** (using suffix trie/compressed trie)

**Trigger phrases:** "prefix", "autocomplete", "dictionary", "word break", "search suggest", "phone directory", "prefix matching"

### 4. When Not to Use It

- **Single string search** — just use a hash set ( $O(1)$  vs  $O(L)$ , simpler)
- **Very small dictionary** — array or hash set is simpler and uses less memory
- **Memory-constrained environments** — each character per node allocates a full array of 26 pointers (or map overhead)
- **Exact match only** — a hash set is  $O(1)$  average, simpler to implement
- **Suffix queries** — use suffix tree/array instead
- **Need sorted order of all strings** — can do DFS but a simple sort is often faster for small sets

**Common wrong assumption:** "Trie is always faster than hash set for lookup." False — hash sets have  $O(1)$  average, trie is  $O(L)$ . For short strings, hash sets often beat tries. Trie wins at prefix operations and memory sharing.

## 5. Core Concepts

### 5.1 Trie Node

- Contains an array/vector of children (size = alphabet size, often 26 for lowercase)
- A boolean flag `isEndOfWord` marking if a word ends here
- (Optional) frequency counter, word ID, etc.

### 5.2 Root Node

- Empty node representing the start of all prefixes
- Has no character value of its own

### 5.3 Insertion

- Walk character by character from root
- If child doesn't exist, create it
- At the final node, mark `isEndOfWord = true`

### 5.4 Search

- Walk character by character from root
- If any child is missing → word not found
- At the end, return `isEndOfWord` flag

### 5.5 Prefix Search (StartsWith)

- Same as search but doesn't check `isEndOfWord`
- If we traverse all characters successfully → prefix exists

### 5.6 Deletion (optional)

- Recursively delete nodes only when they have no children and are not end-of-word
- Careful: don't delete nodes that are part of other words

### 5.7 Alphabet Size

- 26 for lowercase English letters

- 52 for case-sensitive
- 256 for ASCII
- Dynamic (map/unordered\_map) for large alphabets

## 6. Step-by-Step Algorithm

### Insertion

1. Start at `root`
2. For each character `c` in the word: a. Compute index: `idx = c - 'a'` (or use a map) b. If `children[idx]` is null, create a new node `c`. Move to `children[idx]`
3. At the final node, set `isEndOfWord = true`

### Search

1. Start at `root`
2. For each character `c` in the word: a. Compute index b. If `children[idx]` is null → return `false` c. Move to `children[idx]`
3. Return `isEndOfWord` of the final node

### Prefix Exists (StartsWith)

1. Same as search steps 1–2
2. Return `true` (we reached the end of prefix)

## 7. Dry Run

Insert words: ["cat", "car", "bat", "can"]

```
Insert "cat":  
  root → c (new) → a (new) → t (new, end)  
  
Insert "car":  
  root → c (exists) → a (exists) → r (new, end)  
  
Insert "bat":  
  root → b (new) → a (new) → t (new, end)  
  
Insert "can":  
  root → c (exists) → a (exists) → n (new, end)
```

Search:

| Query | Path               | Result |
|-------|--------------------|--------|
| "cat" | root→c→a→t(end)    | true   |
| "car" | root→c→a→r(end)    | true   |
| "ca"  | root→c→a (not end) | false  |

| Query | Path             | Result |
|-------|------------------|--------|
| "cab" | root→c→a→b(miss) | false  |

Prefix: "ca" → root→c→a → true (exists, even though no word ends at "ca")

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct TrieNode {
    TrieNode* children[26];
    bool isEndOfWord;

    TrieNode() {
        isEndOfWord = false;
        for (int i = 0; i < 26; ++i) children[i] = nullptr;
    }
};

class Trie {
private:
    TrieNode* root;

public:
    Trie() { root = new TrieNode(); }

    // Insert a word
    void insert(const string& word) {
        TrieNode* node = root;
        for (char ch : word) {
            int idx = ch - 'a';
            if (!node->children[idx])
                node->children[idx] = new TrieNode();
            node = node->children[idx];
        }
        node->isEndOfWord = true;
    }

    // Search for exact word
    bool search(const string& word) {
        TrieNode* node = root;
        for (char ch : word) {
            int idx = ch - 'a';
            if (!node->children[idx]) return false;
            node = node->children[idx];
        }
        return node->isEndOfWord;
    }

    // Check if prefix exists
```

```

    bool startsWith(const string& prefix) {
        TrieNode* node = root;
        for (char ch : prefix) {
            int idx = ch - 'a';
            if (!node->children[idx]) return false;
            node = node->children[idx];
        }
        return true;
    }

    // Get all words with given prefix (autocomplete)
    vector<string> getSuggestions(const string& prefix) {
        vector<string> result;
        TrieNode* node = root;
        for (char ch : prefix) {
            int idx = ch - 'a';
            if (!node->children[idx]) return result;
            node = node->children[idx];
        }
        // DFS from this node to collect all words
        string current = prefix;
        dfs(node, current, result);
        return result;
    }

private:
    void dfs(TrieNode* node, string& current, vector<string>& result) {
        if (node->isEndOfWord) result.push_back(current);
        for (int i = 0; i < 26; ++i) {
            if (node->children[i]) {
                current.push_back('a' + i);
                dfs(node->children[i], current, result);
                current.pop_back();
            }
        }
    }
};

// ---- Example Usage ----
int main() {
    Trie trie;
    vector<string> words = {"cat", "car", "bat", "can", "card", "care"};
    for (auto& w : words) trie.insert(w);

    cout << boolalpha;
    cout << "search(cat): " << trie.search("cat") << endl;    // true
    cout << "search(cab): " << trie.search("cab") << endl;    // false
    cout << "startsWith(ca): " << trie.startsWith("ca") << endl; // true

    auto sug = trie.getSuggestions("ca");
    cout << "Suggestions for 'ca': ";
    for (auto& s : sug) cout << s << " ";    // cat can car card care
    cout << endl;
}

```

```
    return 0;
}
```

## 9. Python Implementation

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def search(self, word: str) -> bool:
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.is_end

    def startsWith(self, prefix: str) -> bool:
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return True

    def get_suggestions(self, prefix: str) -> list:
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return []
            node = node.children[ch]
        result = []
        self._dfs(node, list(prefix), result)
        return result

    def _dfs(self, node, curr, result):
        if node.is_end:
            result.append("".join(curr))
```

```

        for ch in sorted(node.children):
            curr.append(ch)
            self._dfs(node.children[ch], curr, result)
            curr.pop()

# Example usage
trie = Trie()
for w in ["cat", "car", "bat", "can", "card", "care"]:
    trie.insert(w)
print(trie.search("cat"))      # True
print(trie.search("cab"))     # False
print(trie.startsWith("ca"))  # True
print(trie.get_suggestions("ca")) # ['can', 'car', 'card', 'care', 'cat']

```

## 10. Code Explanation

### TrieNode

- `children[26]`: fixed array for lowercase letters. Index 0 = 'a', 25 = 'z'. Using `nullptr` means "no such child."
- `isEndOfWord`: marks whether a complete word ends at this node. Critical: without it, we can't distinguish "car" from "ca" (which is just a prefix of "car").

### Insert

- Walk the word character by character. For each char, compute index, create node if missing, move down.
- After the loop, mark the terminal node. This is the only place `isEndOfWord` is set to `true`.

### Search

- Walk character by character. If any child is missing → word cannot exist.
- After traversing all chars, check `isEndOfWord`. This distinguishes prefix from exact match.

### StartsWith (prefix search)

- Same walking logic but ignore `isEndOfWord`. If we survive the walk, prefix exists.

### GetSuggestions (autocomplete)

- First navigate to the end of the prefix (like `startsWith`).
- Then DFS from that node: at every node where `isEndOfWord` is true, add current string to results.
- Backtrack by popping the last character after exploring a branch.

### Why array vs map?

- Array of 26: O(1) child lookup, but wastes memory if alphabet is sparse.
- Map (hash or ordered): saves memory for sparse tries, but has O(log k) or O(1) avg overhead.
- For lowercase English letters, array is faster and standard in CP.

## 11. Complexity Analysis

| Operation  | Time       | Space                         |
|------------|------------|-------------------------------|
| Insert     | $O(L)$     | $O(L \times \text{alphabet})$ |
| Search     | $O(L)$     | $O(1)$                        |
| StartsWith | $O(L)$     | $O(1)$                        |
| Suggest    | $O(L + R)$ | $O(R)$                        |

Where  $L$  = length of word/prefix,  $R$  = total characters in all results.

| Aspect                           | Complexity                                                                                                         |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Build ( $N$ words, avg len $L$ ) | $O(N \times L)$ time, $O(N \times L \times \text{alphabet})$ space                                                 |
| Lookup                           | $O(L)$ time, $O(1)$ extra space                                                                                    |
| Memory                           | Each node has 26 pointers (~208 bytes with overhead). Total $\approx O(\text{total chars} \times \text{alphabet})$ |

**Worst-case memory:** When no strings share prefixes (e.g., "a", "b", "c", ...) — each character creates its own chain with no sharing.

## 12. Common Patterns

### Pattern 1: Word Break / String Segmentation

**Identify:** "Can a string be formed by concatenating dictionary words?"

**Approach:** Use trie for dictionary + DP. At each position  $i$ , check if substring  $S[j..i]$  is in trie.

**Problems:** LeetCode 139 (Word Break), LeetCode 140 (Word Break II)

### Pattern 2: Prefix Matching / Autocomplete

**Identify:** "Find all words starting with given prefix", "Search suggestions"

**Approach:** Navigate to prefix node, DFS from there.

**Problems:** LeetCode 208 (Implement Trie), LeetCode 648 (Replace Words), LeetCode 1268 (Search Suggestions)

### Pattern 3: Maximum XOR Pair (Binary Trie)

**Identify:** "Find two numbers with maximum XOR in an array"

**Approach:** Insert numbers as 32-bit binary strings into a binary trie. For each number, traverse opposite bits to maximize XOR.

**Problems:** LeetCode 421 (Maximum XOR of Two Numbers)

### Pattern 4: Longest Common Prefix



**Identify:** "Find longest common prefix among N strings"

**Approach:** Insert all strings, find the deepest node with exactly one child and no `isEndOfWord` before branching.

**Problems:** LeetCode 14 (Longest Common Prefix)

## Pattern 5: Word Search II (Trie + Backtracking)

**Identify:** "Find all dictionary words in a 2D grid"

**Approach:** Build trie of dictionary, DFS on grid, prune when prefix not in trie.

**Problems:** LeetCode 212 (Word Search II)

## 13. Common Mistakes

- **Forgetting `isEndOfWord` flag** — search returns true for prefixes that aren't complete words
- **Not handling uppercase / special characters** — array size of 26 only works for lowercase
- **Memory leak** — not deleting nodes in C++ (in placement coding, usually acceptable; in production, use smart pointers)
- **Incorrect index calculation** — `ch - 'a'` fails for uppercase or non-letter chars
- **Assuming all words are lowercase** — always check constraints
- **Not resetting `isEndOfWord` on deletion** — partial deletion breaks later searches
- **Deleting shared nodes** — when deleting a word, don't delete nodes that are part of other words
- **Trie vs hash set confusion** — using trie when only exact match is needed (hash set is simpler)

## 14. Edge Cases

- **Empty string** — make sure root's `isEndOfWord` is handled properly. Empty string should be "inserted" by setting root's flag.
- **Single character words** — "a", "b" — ensure first-level nodes are handled correctly
- **Words that are prefixes of each other** — "a" and "ab" — ensure both can be found
- **Duplicate insertions** — shouldn't break anything, just set flag again
- **Very long words ( $L = 10^5$ )** — recursion depth for DFS may cause stack overflow; use iterative approach
- **Unicode / multi-byte characters** — array of 26 won't work; use map
- **Large alphabet** — numbers (0-9), hex, etc. Adjust array size accordingly

## 15. Variations

### 1. Ternary Search Tree (TST)

- Each node has 3 children: left (smaller), middle (equal), right (larger)
- Uses less memory than trie array, simpler than map-based trie
- Used in autocomplete systems (e.g., Java's `TernarySearchTree`)

### 2. Compressed Trie (Radix Tree / Patricia Trie)

- Merges chains of single-child nodes into a single node with a string
- Drastically reduces memory for long strings with few branches
- Used in IP routing (radix trie)

### 3. Suffix Trie / Suffix Tree

- Trie of all suffixes of a string
- Enables  $O(L)$  substring search, longest repeated substring, etc.
- Suffix tree is the compressed version — fundamental in stringology
- Important for CP but complex to implement

### 4. Binary Trie (Bitwise Trie)

- Stores integers as binary strings (usually 32 bits)
- Each node has 2 children: 0 and 1
- Used for maximum XOR pair, range queries on binary representation

### 5. Persistent Trie

- Maintains version history of inserts
- Used in problems requiring queries on past states
- Appears in advanced CP (e.g., Codeforces problems)

## 16. Related Algorithms/Data Structures

| DS                       | Trie vs ...                                                                                                               |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Hash Set</b>          | Hash set: $O(1)$ avg exact match, no prefix search. Trie: $O(L)$ exact & prefix. Use set for exact-only, trie for prefix. |
| <b>BST<br/>(set/map)</b> | BST: $O(\log N)$ per operation, no prefix sharing. Trie: $O(L)$ independent of $N$ .                                      |
| <b>Suffix Array</b>      | For substring queries, suffix array + LCP is more space-efficient and often faster than suffix trie.                      |
| <b>Aho-Corasick</b>      | Trie + failure links for multi-pattern string matching. Builds on trie.                                                   |

## 17. Practice Problems

### Easy

1. **Implement Trie (Prefix Tree)** — LeetCode 208  
*Core trie implementation with insert, search, startsWith.*
2. **Longest Common Prefix** — LeetCode 14  
*Trie-based solution vs horizontal scanning.*

### Medium

1. **Replace Words** — LeetCode 648  
*Replace words with their shortest root (prefix) using trie.*
2. **Search Suggestions System** — LeetCode 1268  
*Autocomplete with product suggestions. Trie + DFS or prefix search.*
3. **Design Add and Search Words Data Structure** — LeetCode 211  
*Wildcard '.' matching. Add DFS with backtracking for wildcards.*

## Hard

1. **Word Search II** — LeetCode 212  
*Trie + DFS on 2D grid. Classic hard problem.*
2. **Palindrome Pairs** — LeetCode 336  
*Trie + reverse string logic. Tricky edge cases.*
3. **Maximum XOR of Two Numbers in an Array** — LeetCode 421  
*Binary trie (bitwise).*

## 18. Interview Explanation

"A trie is a tree where each node represents one character of a string, and the path from the root spells out the word. The key insight is that common prefixes are shared — so looking up any word is  $O(L)$  regardless of how many words are stored. Insertion is also  $O(L)$ .  
I'd use it when I need prefix-based queries, like autocomplete, spell check, or word break. The trade-off is memory: each node can have up to 26 child pointers, so for sparse dictionaries a hash set may be simpler.  
For implementation, each node has an array of child pointers and a boolean flag. Insert walks char-by-char, creating nodes as needed. Search walks the same path and checks the end flag. Prefix search is the same but ignores the flag.  
A common optimization for large alphabets is to use a map or a ternary search tree instead of a fixed array."

## 19. Revision Notes

- **Structure:** Root → node per character, **isEndOfWord** flag
- **Insert:**  $O(L)$ , create missing nodes
- **Search:**  $O(L)$ , walk + check flag
- **Prefix:**  $O(L)$ , walk only
- **Autocomplete:** Navigate to prefix, DFS from there
- **Memory:** 26 pointers per node — can be heavy
- **Key trap:** Forgetting **isEndOfWord** → prefix returns true for words
- **Binary trie:** 2 children (0/1), used for max XOR
- **TST:** 3 children (</=>), memory-efficient

## 20. Final Cheat Sheet

## TRIE – Prefix Tree

USE WHEN: prefix queries, autocomplete, word break, dictionary with prefixes

OPS: insert  $O(L)$ , search  $O(L)$ , startsWith  $O(L)$

CODE:

```
insert: for ch in word: node = node->children[ch - 'a'] ?? createNew()
```

```
search: same walk → return node->isEndOfWord
```

```
prefix: same walk → return true (no flag check)
```

EDGE CASES: empty string, word=prefix of another, no common prefix

TRAPS: isEndOfWord flag, array size=26 only for lowercase, memory for sparse tries

---

## 2. INTERVAL MERGE

---

### 1. Overview

**Interval merge** is the problem of combining overlapping or adjacent intervals into the minimum set of non-overlapping intervals. Given intervals [start, end], if any two intervals overlap (or touch, depending on the definition), they can be merged into a single interval covering their union.

### 2. Intuition

**Simple explanation:** Imagine booking meetings on a calendar. If meeting A is 10–11 and meeting B is 10:30–12, you can describe them as one block: 10–12. Interval merge does exactly that — merge overlapping time slots into contiguous blocks.

**Analogy:** Picture the intervals as colored segments on a number line. Wherever segments overlap, paint over the whole region. The merged intervals are the contiguous painted segments.

**Why it works:** By sorting by start time, we guarantee that overlapping intervals are processed in order. As we scan, if the current interval's start  $\leq$  the last merged interval's end, they overlap — extend the end. Otherwise, start a new interval.

### 3. When to Use It

- **Meeting room scheduling** — merge meeting times
- **Free/busy time slots** — find free time from busy intervals
- **Range coverage** — "does the set of intervals fully cover [A, B]?"
- **Union of intervals** — combine overlapping ranges
- **Interval intersection problems** — find overlap between two sets of intervals
- **Calendar apps, resource allocation, scheduling**
- **Data range consolidation** — merging IP ranges, date ranges, memory segments

**Trigger phrases:** "merge intervals", "overlapping intervals", "non-overlapping", "union of intervals", "cover range", "scheduling"

## 4. When Not to Use It

- **Non-overlapping intervals already** — no merging needed, just sort (if needed)
- **Single interval** — trivial, no algorithm needed
- **Dynamic insertions of intervals** — use interval tree or segment tree for  $O(\log N)$  insert/merge
- **Counting overlaps (non-merge)** — use sweep line instead (see Sweep Line section)
- **Point queries** — "how many intervals cover point X?" — use difference array or segment tree
- **Weighted interval scheduling** — DP, not merge

**Common wrong assumption:** "Merge always means combine if they touch." Some problems define overlap as  $[1,2]$  and  $[2,3]$  as touching (merge) or not. Always check whether adjacency counts.

## 5. Core Concepts

### 5.1 Interval Representation

- Usually  $[start, end]$  inclusive or  $[start, end)$  exclusive
- Clarify inclusivity from problem statement
- In CP, often `pair<int,int>` or a struct

### 5.2 Overlap Condition

- Two intervals  $[a, b]$  and  $[c, d]$  overlap if  $a \leq d \ \&\& \ c \leq b$
- For adjacency merge:  $a \leq d + 1 \ \&\& \ c \leq b + 1$

### 5.3 Merging

- If overlapping:  $merged = [\min(a,c), \max(b,d)]$
- Key: the merged interval's start is the earlier start, end is the later end

### 5.4 Sorting by Start

- Almost all interval merge algorithms start by sorting by start time
- $O(N \log N)$  sorting step dominates

## 6. Step-by-Step Algorithm

1. If intervals array is empty, return empty
2. Sort intervals by start time (ascending)
3. Initialize `result` with the first interval
4. For each remaining interval  $[curr\_start, curr\_end]$ : a. If  $curr\_start \leq last\_end$ : they overlap → update  $last\_end = \max(last\_end, curr\_end)$  b. Else: no overlap → push current interval as new entry to result
5. Return `result`

## 7. Dry Run

Input: intervals =  $[[1,3], [2,6], [8,10], [15,18], [16,17]]$

**Step 1:** Sort by start → same order (already sorted)

| Step | curr    | last in result | Overlap?       | Action                             | Result                   |
|------|---------|----------------|----------------|------------------------------------|--------------------------|
| Init |         |                |                | result = [[1,3]]                   | [[1,3]]                  |
| 1    | [2,6]   | [1,3]          | $2 \leq 3$ Y   | merge $\rightarrow \max(3,6)=6$    | [[1,6]]                  |
| 2    | [8,10]  | [1,6]          | $8 \leq 6$ N   | new interval                       | [[1,6], [8,10]]          |
| 3    | [15,18] | [8,10]         | $15 \leq 10$ N | new interval                       | [[1,6], [8,10], [15,18]] |
| 4    | [16,17] | [15,18]        | $16 \leq 18$ Y | merge $\rightarrow \max(18,17)=18$ | [[1,6], [8,10], [15,18]] |

**Final:** [[1,6], [8,10], [15,18]]

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> mergeIntervals(vector<vector<int>>& intervals) {
    if (intervals.empty()) return {};

    // Sort by start time
    sort(intervals.begin(), intervals.end());

    vector<vector<int>> result;
    result.push_back(intervals[0]);

    for (int i = 1; i < (int)intervals.size(); ++i) {
        // If current start <= last end → overlap
        if (intervals[i][0] <= result.back()[1]) {
            // Merge: extend the end
            result.back()[1] = max(result.back()[1], intervals[i][1]);
        } else {
            // No overlap → new interval
            result.push_back(intervals[i]);
        }
    }
    return result;
}

// ---- Example Usage ----
int main() {
    vector<vector<int>> intervals = {{1,3}, {2,6}, {8,10}, {15,18}};
    auto merged = mergeIntervals(intervals);
    for (auto& v : merged)
        cout << "[" << v[0] << "," << v[1] << "]" << " ";
    // Output: [1,6] [8,10] [15,18]
    return 0;
}
```

## 9. Python Implementation

```
def merge_intervals(intervals):
    if not intervals:
        return []

    # Sort by start time
    intervals.sort(key=lambda x: x[0])
    result = [intervals[0][:]]

    for start, end in intervals[1:]:
        # If current start ≤ last end → overlap
        if start <= result[-1][1]:
            result[-1][1] = max(result[-1][1], end)
        else:
            result.append([start, end])

    return result

# Example
print(merge_intervals([[1,3], [2,6], [8,10], [15,18]]))
# [[1, 6], [8, 10], [15, 18]]
```

## 10. Code Explanation

### Sorting

- `sort(intervals.begin(), intervals.end())` sorts by first element (start) ascending.
- This is critical: without sorting, we can't guarantee correct merging in one pass.

### Merge loop

- `intervals[i][0] <= result.back()[1]` — overlap check.
- We compare current start with the **end of the last interval in result** (not with the raw previous interval).
- `result.back()[1] = max(result.back()[1], intervals[i][1])` — extend end if current ends later.

### Why push first interval before loop?

- This avoids a special case inside the loop. We always compare against the tail of result.

### Edge case in result

- If we need adjacent merge too (if intervals like [1,2] and [3,4] should merge to [1,4]), use `<=` instead of `<` and check `intervals[i][0] <= result.back()[1] + 1`.

## 11. Complexity Analysis

| Operation | Time          | Space                    |
|-----------|---------------|--------------------------|
| Sort      | $O(N \log N)$ | $O(\log N)$ (sort stack) |

| Operation      | Time                            | Space                                             |
|----------------|---------------------------------|---------------------------------------------------|
| Merge pass     | $O(N)$                          | $O(1)$                                            |
| <b>Overall</b> | <b><math>O(N \log N)</math></b> | <b><math>O(N)</math></b> (result) or $O(1)$ extra |

- **N** = number of intervals
- **Space  $O(N)$**  for result in worst case (no intervals overlap)

## 12. Common Patterns

### Pattern 1: Basic Merge

**Identify:** "Given a list of intervals, merge all overlapping intervals."

**Approach:** Sort + scan + merge.

**Problems:** LeetCode 56 (Merge Intervals)

### Pattern 2: Insert Interval (into sorted non-overlapping list)

**Identify:** "Insert a new interval into a list of already sorted non-overlapping intervals."

**Approach:** Find insertion point, merge overlapping ones.

**Problems:** LeetCode 57 (Insert Interval)

### Pattern 3: Interval Intersection (find common overlap of two sets)

**Identify:** "Given two lists of intervals, find their intersection."

**Approach:** Two-pointer scan,  $\text{overlap} = [\max(\text{start1}, \text{start2}), \min(\text{end1}, \text{end2})]$  when valid.

**Problems:** LeetCode 986 (Interval List Intersections)

### Pattern 4: Count overlaps (not merge)

**Identify:** "Find the point with maximum overlapping intervals."

**Approach:** Sweep line — use difference array or event points.

**Problems:** LeetCode 253 (Meeting Rooms II), CSES Restaurant Customers

### Pattern 5: Minimum number of arrows to burst balloons

**Identify:** "Minimum arrows to burst balloons" given interval  $[x_{\text{start}}, x_{\text{end}}]$ .

**Approach:** Sort by end, greedily shoot at end of first, skip overlapping.

**Problems:** LeetCode 452 (Minimum Number of Arrows to Burst Balloons)

## 13. Common Mistakes

- **Not sorting by start** — merging fails catastrophically
- **Incorrect overlap condition** — using  $<$  when  $\leq$  is needed (or vice versa)
- **Not handling empty input** — return `{}` or `[]`
- **Modifying input** — okay in CP, but make a copy if needed
- **Using `result.back()[0]` instead of `[1]`** — comparing start with wrong value
- **Not updating `max(end, curr_end)`** — just setting `last_end = curr_end` loses the old end

## 14. Edge Cases



- **Empty input** → return `[]`
- **Single interval** → return the interval as-is
- **All overlapping** → `[[1,10]]`, result is one interval
- **None overlapping** → return same intervals (but sorted)
- **Adjacent** → `[1,2]`, `[2,3]` — does problem define this as overlapping? Check.
- **Negative values** → `[-5, -1]`, `[-3, 2]` — works fine
- **Unsorted input** → must sort first
- **Same start, different ends** → merge correctly with `max(end)`
- **Intervals fully contained** → `[1,10]`, `[2,5]` → result is `[1,10]`

## 15. Variations

### 1. Interval Insertion

- Insert a new interval into existing sorted non-overlapping list
- $O(N)$  time, handles merge on the fly

### 2. Meeting Rooms II (Minimum Platforms)

- Not merge, but count maximum simultaneous intervals
- Uses sweep line (see next section)

### 3. Non-overlapping Intervals (Remove minimum to make non-overlapping)

- Greedy: sort by end, pick intervals that don't overlap
- Classic "activity selection" problem

### 4. Interval Tree

- Augmented BST for  $O(\log N)$  interval operations
- Overkill for static lists, useful for dynamic inserts

### 5. Interval Map (C++ `std::map` trick)

- Use `map<int, int>` keyed by start. When inserting, merge overlapping neighbors.
- Used in Codeforces problems for dynamic interval ranges.

## 16. Related Algorithms/Data Structures

| Algorithm/DS                       | Connection                                                    |
|------------------------------------|---------------------------------------------------------------|
| <b>Sweep Line</b>                  | For counting overlaps, not merging. Event-based approach.     |
| <b>Segment Tree</b>                | For interval queries (range sum, min, max) over an array.     |
| <b>Difference Array</b>            | $O(1)$ range update, $O(N)$ prefix sum. For overlap counting. |
| <b>Greedy (Activity Selection)</b> | Picking max non-overlapping intervals. Sort by end.           |

## 17. Practice Problems

## Easy

1. **Merge Intervals** — LeetCode 56  
*Core merge algorithm.*
2. **Non-overlapping Intervals** — LeetCode 435  
*Remove minimum intervals to make rest non-overlapping. Greedy.*

## Medium

1. **Insert Interval** — LeetCode 57  
*Insert into sorted non-overlapping list, merge if needed.*
2. **Interval List Intersections** — LeetCode 986  
*Two-pointer intersection of two interval lists.*
3. **Minimum Number of Arrows to Burst Balloons** — LeetCode 452  
*Sort by end, greedily shoot.*

## Hard

1. **Data Stream as Disjoint Intervals** — LeetCode 352  
*Dynamic interval merge on stream. Use ordered set/map.*
2. **Employee Free Time** — LeetCode 759  
*Merged intervals from multiple sorted lists → find gaps.*

## 18. Interview Explanation

"Interval merging is a classic greedy problem. The key insight is to sort by start time. Once sorted, if the current interval's start is within the last merged interval, we extend the end. Otherwise we start a new merged interval.

The algorithm runs in  $O(N \log N)$  due to sorting, and  $O(N)$  for the merge pass. It uses  $O(N)$  space in the worst case for the result.

The critical edge case is handling adjacency — does  $[1,2]$  and  $[2,3]$  overlap? I always clarify this with the interviewer.

For related problems, I can modify the same approach for interval insertion, finding intersections of two interval lists, or the minimum arrows to burst balloons."

## 19. Revision Notes

- **Core:** Sort by start, then greedy merge
- **Overlap:** `curr_start <= last_end`
- **Merge:** `last_end = max(last_end, curr_end)`
- **Complexity:**  $O(N \log N)$  time,  $O(N)$  space
- **Edge cases:** empty, single, all overlapping, adjacent (check definition)
- **Variations:** insert interval, intersection, non-overlapping (activity selection)
- **Trap:** Forgetting to sort; incorrect overlap comparison

## 20. Final Cheat Sheet

## INTERVAL MERGE – Sort + Greedy

USE WHEN: merging overlapping ranges, scheduling, coverage

STEPS: sort by start → init result[0] → scan: overlap? merge else push

CODE:

```

    sort(intervals.begin(), intervals.end())
    for each [s,e]: if s <= result.back()[1]: result.back()[1] = max(result.back()
[1], e)
                        else: result.push_back([s,e])

```

TIME:  $O(N \log N)$       SPACE:  $O(N)$

TRAPS: sorting, adjacency definition, empty input

# 3. SWEEP LINE

## 1. Overview

**Sweep line** (also called plane sweep) is a technique where an imaginary vertical line sweeps across the plane from left to right, and we process events at specific x-coordinates. Instead of handling all coordinates, we only care about **event points** — where something starts or ends.

## 2. Intuition

**Simple explanation:** Imagine a timeline of meetings. Instead of checking every minute, mark each start time as "+1" (a meeting starts) and each end time as "-1" (a meeting ends). As the sweep line moves from left to right, keep a running count. The peaks tell you the maximum number of simultaneous meetings.

**Analogy:** Like walking through a day with a counter. Every time a meeting starts, increment. Every time one ends, decrement. The maximum value of your counter during the day is the maximum overlap.

**Why it works:** By converting intervals into events (start +1, end -1), we reduce a continuous problem to discrete points. Sorting the events by x gives us the correct order to process, and the running sum at any point gives the active count. This is the difference array concept extended to arbitrary coordinates.

## 3. When to Use It

- **Maximum overlap** — "find the point where most intervals overlap"
- **Minimum meeting rooms / platforms** — count simultaneous intervals
- **Interval intersection (union length)** — total covered length by intervals
- **Rectangle union area/perimeter** — classic sweep line application
- **Skyline problem** — building silhouettes (LeetCode 218)
- **Count of active intervals at query points**
- **Nested intervals counting**
- **Event scheduling problems**

**Trigger phrases:** "maximum overlapping", "minimum meeting rooms", "simultaneous", "union of intervals", "number of platforms needed", "sweep line", "event processing"

## 4. When Not to Use It

- **Merging intervals** — interval merge is simpler (just sort + merge)
- **Static overlap count at a single point** — just count intervals covering it ( $O(N)$ )
- **Small fixed range** — use difference array ( $O(N + \text{MAX})$ ) instead
- **Point queries only after all intervals processed** — difference array + prefix sum is simpler
- **Non-overlapping intervals** — sweep line is overkill

**Common wrong assumption:** "Sweep line always requires a balanced BST." Not true — most problems only need sorting + counter.

## 5. Core Concepts

### 5.1 Event

- A point on the line where something changes
- Each interval produces two events:  $(\text{start}, +1)$  and  $(\text{end}, -1)$
- For union length:  $(\text{start}, +1)$  and  $(\text{end}, -1)$  where end is exclusive

### 5.2 Event Sorting

- Sort events by x-coordinate
- Tie-breaking is critical: when  $\text{start} == \text{end}$  of another interval, which goes first?
  - For overlap counting: start (+1) before end (-1) to get correct count at that point
  - For union length: process all start before end, then compute  $\text{prev\_x\_to\_curr\_x} * (\text{count} > 0)$

### 5.3 Active Count

- Running sum of event deltas as we sweep
- At any point,  $\text{active\_count}$  = number of intervals currently covering

### 5.4 Sweep State

- Variables maintained during sweep: active count, last x, max count, union length, etc.

## 6. Step-by-Step Algorithm

For Maximum Overlap (Meeting Rooms II)

1. Create list of events: for each interval  $[s, e]$ :
  - $(s, +1)$  — a meeting starts
  - $(e, -1)$  — a meeting ends
2. Sort events: by x ascending. If x tie, start (+1) before end (-1)
3. Initialize  $\text{active} = 0$ ,  $\text{max\_active} = 0$
4. For each event  $(x, \text{delta})$ :
  - $\text{active} += \text{delta}$
  - $\text{max\_active} = \max(\text{max\_active}, \text{active})$
5. Return  $\text{max\_active}$

## For Union Length of Intervals

1. Create events:  $(s, +1)$  and  $(e, -1)$  (e is exclusive)
2. Sort events by x. Tie: process all +1 before -1
3. Initialize `active = 0`, `prev_x = first_event.x`, `total = 0`
4. For each event  $(x, \text{delta})$ :
  - If `active > 0`: `total += (x - prev_x)`
  - `active += delta`
  - `prev_x = x`
5. Return `total`

## 7. Dry Run

Input: intervals = [[1,3], [2,6], [5,7], [8,10]]

Events:

| x  | delta |
|----|-------|
| 1  | +1    |
| 2  | +1    |
| 3  | -1    |
| 5  | +1    |
| 6  | -1    |
| 7  | -1    |
| 8  | +1    |
| 10 | -1    |

Sweep:

| x  | delta | active | max_active |
|----|-------|--------|------------|
| 1  | +1    | 1      | 1          |
| 2  | +1    | 2      | 2          |
| 3  | -1    | 1      | 2          |
| 5  | +1    | 2      | 2          |
| 6  | -1    | 1      | 2          |
| 7  | -1    | 0      | 2          |
| 8  | +1    | 1      | 2          |
| 10 | -1    | 0      | 2          |

**Maximum overlap = 2** (at x=2 to x=3, and x=5 to x=6)

## 8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// Maximum simultaneous intervals (Meeting Rooms II)
int minMeetingRooms(vector<vector<int>>& intervals) {
    vector<pair<int, int>> events; // (x, delta)

    for (auto& iv : intervals) {
        events.emplace_back(iv[0], +1); // start
        events.emplace_back(iv[1], -1); // end
    }

    // Sort by x. If tie, +1 before -1 (starts before ends)
    sort(events.begin(), events.end(), [](auto& a, auto& b) {
        if (a.first != b.first) return a.first < b.first;
        return a.second > b.second; // +1 > -1, so starts come first
    });

    int active = 0, maxActive = 0;
    for (auto& [x, delta] : events) {
        active += delta;
        maxActive = max(maxActive, active);
    }
    return maxActive;
}

// Union length of intervals (total covered length)
int unionLength(vector<vector<int>>& intervals) {
    vector<pair<int, int>> events;
    for (auto& iv : intervals) {
        events.emplace_back(iv[0], +1);
        events.emplace_back(iv[1], -1); // exclusive end
    }

    sort(events.begin(), events.end(), [](auto& a, auto& b) {
        if (a.first != b.first) return a.first < b.first;
        return a.second > b.second;
    });

    int active = 0, prevX = events[0].first;
    int total = 0;

    for (auto& [x, delta] : events) {
        if (active > 0) total += (x - prevX);
        active += delta;
        prevX = x;
    }
    return total;
}

```

```
// ---- Example Usage ----
int main() {
    vector<vector<int>> intervals = {{1,3}, {2,6}, {5,7}, {8,10}};
    cout << "Max overlap: " << minMeetingRooms(intervals) << endl; // 2
    cout << "Union length: " << unionLength(intervals) << endl; // 1-7 + 8-10 =
6+2 = 8
    return 0;
}
```

## 9. Python Implementation

```
def min_meeting_rooms(intervals):
    events = []
    for s, e in intervals:
        events.append((s, +1))
        events.append((e, -1))

    # Sort by x. +1 (start) before -1 (end) for same x
    events.sort(key=lambda x: (x[0], -x[1]))

    active = max_active = 0
    for x, delta in events:
        active += delta
        max_active = max(max_active, active)
    return max_active

def union_length(intervals):
    events = []
    for s, e in intervals:
        events.append((s, +1))
        events.append((e, -1)) # exclusive end

    events.sort(key=lambda x: (x[0], -x[1]))

    active = 0
    prev_x = events[0][0]
    total = 0

    for x, delta in events:
        if active > 0:
            total += x - prev_x
            active += delta
            prev_x = x
    return total

# Example
print(min_meeting_rooms([[1,3],[2,6],[5,7],[8,10]])) # 2
print(union_length([[1,3],[2,6],[5,7],[8,10]])) # 8
```

## 10. Code Explanation

## Event creation

- Each interval  $[l, r]$  creates  $(l, +1)$  and  $(r, -1)$ .
- For inclusive intervals  $[l, r]$ , the end event should happen after the start of another interval that starts at  $r$ . Tie-breaking ensures starts  $(+1)$  come before ends  $(-1)$ .

## Sort with tie-breaking

- `sort by (x, -delta)` ensures that for same  $x$ ,  $+1$  (start) comes before  $-1$  (end).
- Why? If one meeting ends at 3 and another starts at 3, we should count them as overlapping if we want maximum simultaneous. Start-before-end gives active count of 2 at  $x=3$ .

## Sweep

- `active += delta` updates the running count.
- For max overlap: track `max_active`.
- For union length: when `active > 0`, add  $(curr\_x - prev\_x)$  to total.

## Union length detail

- We only add length when `active > 0` (at least one interval covering).
- The term  $(x - prev\_x)$  gives length of the segment where the active count was constant.
- This works because we only process at event points, and between events the count doesn't change.

## 11. Complexity Analysis

| Variant        | Time                            | Space                    |
|----------------|---------------------------------|--------------------------|
| Event creation | $O(N)$                          | $O(N)$                   |
| Sort           | $O(N \log N)$                   | $O(\log N)$              |
| Sweep          | $O(N)$                          | $O(1)$                   |
| <b>Total</b>   | <b><math>O(N \log N)</math></b> | <b><math>O(N)</math></b> |

| Aspect                 | Complexity    |
|------------------------|---------------|
| $N = \text{intervals}$ |               |
| Sorting dominates      | $O(N \log N)$ |
| Events stored          | $2N$          |

## 12. Common Patterns

### Pattern 1: Maximum Overlap (Meeting Rooms)

**Identify:** "Minimum number of meeting rooms", "minimum platforms", "maximum overlapping"

**Approach:** Events (start  $+1$ , end  $-1$ ), track max of running sum.

**Problems:** LeetCode 253 (Meeting Rooms II), CSES Restaurant Customers, GFG Minimum Platforms

### Pattern 2: Union Length of Intervals



**Identify:** "Total covered length", "union of intervals"

**Approach:** Events, sum distances where active > 0.

**Problems:** LeetCode 56 merge → compute length from result

### Pattern 3: Skyline Problem

**Identify:** "Building skyline", "silhouette of buildings"

**Approach:** Events (x, height, start/end), maintain multiset of active heights.

**Problems:** LeetCode 218 (The Skyline Problem) — Hard

### Pattern 4: Rectangle Area Union

**Identify:** "Area of union of rectangles", "total area covered by rectangles"

**Approach:** Sweep y-coordinates, maintain active x-intervals via segment tree.

**Problems:** LeetCode 850 (Rectangle Area II) — Hard

### Pattern 5: Count of Active Intervals at Points

**Identify:** "For each query point, how many intervals cover it?"

**Approach:** Events (start +1, end -1), sweep and answer queries at points.

**Problems:** Can be offline-processed with sorting.

## 13. Common Mistakes

- **Wrong tie-breaking** — ends before starts gives wrong max overlap count at shared points
- **Using = in active > 0 for union** — active > 0 (not >= 0) for coverage length
- **Not handling exclusive vs inclusive ends** — confusion between [l, r] and [l, r)
- **Not storing events as 2N** — missing end events
- **Sorting incorrectly** — only by x, forgetting the ±1 tie-break
- **Using the wrong sweep direction** — usually left-to-right; some problems need right-to-left or top-to-bottom

## 14. Edge Cases

- **Empty input** → return 0
- **Single interval** → max overlap = 1, union = length
- **All non-overlapping** → max overlap = 1, union = sum of lengths
- **All overlapping** → max overlap = N, union = max\_end - min\_start
- **Same start and end** → [3,3] — zero-length interval. Should it count? Needs clarification.
- **Adjacent intervals** → [1,2] and [2,3] — treat as overlapping or not? Tie-breaking matters.
- **Large coordinates** → coordinate compression + sweep line
- **Negative coordinates** → works fine with events

## 15. Variations

### 1. 2D Sweep Line (Rectangle Union)

- Sweep along one axis, maintain active intervals on the other axis
- Requires segment tree with lazy propagation for optimal  $O(N \log N)$

- Classic geometry problem

## 2. Sweep Line with Balanced BST (Skyline)

- Need **multiset** of active heights during sweep
- Events: building start ( $x, +h$ ), building end ( $x, -h$ )
- Active heights stored in multiset; current max height changes define skyline

## 3. Sweep Line + Coordinate Compression

- When coordinates are large (e.g.,  $10^9$ ), compress them to  $[0, 2N]$
- Use difference array on compressed coordinates instead of events

## 4. Offline Queries with Sweep Line

- If we have interval data and point queries, sweep through points in order
- Process interval starts/ends as we go, answer queries when we reach them

# 16. Related Algorithms/Data Structures

| Variant                       | Connection                                                                         |
|-------------------------------|------------------------------------------------------------------------------------|
| <b>Difference Array</b>       | Sweep line for <b>integer coordinates</b> . $O(N + \text{MAX})$ vs $O(N \log N)$ . |
| <b>Interval Merge</b>         | For merging, not counting. Simpler when you don't need counts.                     |
| <b>Segment Tree</b>           | For complex sweep (rectangle area), segment tree tracks active intervals.          |
| <b>Coordinate Compression</b> | Enables sweep line on large coordinate ranges.                                     |

# 17. Practice Problems

## Easy

1. **Meeting Rooms II** — LeetCode 253 (premium) / LintCode 919  
*Maximum simultaneous meetings. Core sweep line.*
2. **Minimum Platforms** — GFG  
*Same as meeting rooms. Classic CP problem.*

## Medium

1. **Insert Interval** — LeetCode 57  
*Can use sweep line approach: add new interval, find max overlap.*
2. **Number of Flowers in Full Bloom** — LeetCode 2251  
*Sweep line + binary search on prefix of events.*

## Hard

1. **The Skyline Problem** — LeetCode 218  
*Classic sweep line with multiset of heights.*
2. **Rectangle Area II** — LeetCode 850  
*2D sweep line + segment tree. Very hard.*

## 18. Interview Explanation

"The sweep line algorithm processes events at discrete points rather than every coordinate. For problems like 'minimum meeting rooms', each interval generates two events: a start (+1) and an end (-1). After sorting events by x (with start events processed before end events at the same x), we sweep left to right maintaining a running count. The maximum count is the answer.

The complexity is  $O(N \log N)$  due to sorting. The key nuance is tie-breaking: when an interval ends and another starts at the same time, processing the start first gives the correct simultaneous count.

For union length, instead of tracking the max, we add  $(x - \text{prevX})$  to total whenever the active count is positive. This gives the total covered length."

## 19. Revision Notes

- **Core:** Convert intervals to events (start +1, end -1). Sort by x. Sweep.
- **Tie-breaking:** Starts (+1) before ends (-1) for same x → sorted by  $(x, -\text{delta})$
- **Max overlap:** `active += delta`, track `max_active`
- **Union length:** `if active > 0: total += x - prev_x`
- **Complexity:**  $O(N \log N)$  time,  $O(N)$  space
- **Variations:** Skyline (multiset of heights), rectangle union (2D sweep + segtree)
- **Difference array alternative:** If coordinates  $\leq 10^6$ , use difference array  $O(N + \text{MAX})$

## 20. Final Cheat Sheet

SWEEP LINE – Event-based interval processing

USE WHEN: maximum overlap, union length, simultaneous events, skyline

STEPS: events(s,+1)(e,-1) → sort(x, -delta) → sweep: active+=delta, track max/sum

CODE:

```
for [s,e]: events += {(s,1),(e,-1)}
```

```
sort by (x, -delta) // starts before ends
```

```
for (x,d): active+=d; maxActive = max(maxActive, active)
```

TIME:  $O(N \log N)$  SPACE:  $O(N)$

TRAPS: tie-breaking, exclusive vs inclusive ends

ALTERNATIVE: diff array if coordinates  $\leq 10^6$

# 4. TOPOLOGICAL SORT

## 1. Overview

**Topological sort** is a linear ordering of the vertices of a **Directed Acyclic Graph (DAG)** such that for every directed edge  $u \rightarrow v$ , vertex  $u$  comes before  $v$  in the ordering. It answers the question: "In what order should I do these tasks given their dependencies?"

## 2. Intuition

**Simple explanation:** Imagine you have a list of courses with prerequisites. You can't take "Advanced ML" before "Linear Algebra." Topological sort gives you a valid order to take all courses while respecting prerequisites.

**Analogy:** Think of a recipe. You need to chop vegetables before cooking them. You need to preheat the oven before baking. These are dependencies. Topological sort gives you the sequence of steps that respects all "before/after" constraints.

**Why it works:** In a DAG, there's always at least one node with no incoming edges (in-degree = 0), because if every node had an incoming edge, you could follow edges backward forever and find a cycle. You repeatedly pick such nodes, remove them (conceptually), and their outgoing edges stop pointing to their neighbors. This process must finish if the graph has no cycles.

### 3. When to Use It

- **Course schedule / prerequisite chains** — ordering tasks with dependencies
- **Build systems** — which files to compile first (make, gradle)
- **Task scheduling with dependencies** — project planning
- **Deadlock detection** — if topological sort fails, there's a cycle → deadlock
- **Dependency resolution** — package managers, dependency graphs
- **Instruction scheduling** — compilers
- **Detect cycles in directed graph** — if topological sort uses fewer than N nodes, there's a cycle
- **Longest path in DAG** — use topological order + DP

**Trigger phrases:** "prerequisite", "dependency", "course schedule", "order of tasks", "build order", "DAG", "cycle detection", "valid ordering"

### 4. When Not to Use It

- **Undirected graphs** — topological sort only makes sense for directed graphs
- **Cyclic graphs** — topological sort is impossible (no valid linear order)
- **Just need to detect a cycle** — DFS with visited/recursion states is simpler
- **Shortest path in a graph with cycles** — use Bellman-Ford or Dijkstra
- **Single-source reachability** — use DFS/BFS, not topo sort

**Common wrong assumption:** "Topological sort only works for trees." False — it works for any DAG. Trees are a special case.

## 5. Core Concepts

### 5.1 DAG (Directed Acyclic Graph)

- A directed graph with no cycles
- Necessary and sufficient condition for topological sort to exist

### 5.2 In-degree

- Number of incoming edges to a node
- Nodes with in-degree = 0 have no prerequisites — they can go first

### 5.3 Kahn's Algorithm (BFS-based)

- Uses in-degree array + queue
- Start with all in-degree-0 nodes
- Process node, decrement in-degree of neighbors
- When a neighbor's in-degree becomes 0, add to queue

### 5.4 DFS-based Topological Sort

- Run DFS on the graph
- After processing all neighbors of a node, add it to a stack
- Reverse the stack to get topological order
- Use visited array with 3 states: 0=unvisited, 1=in-progress, 2=done (for cycle detection)

### 5.5 Cycle Detection

- Kahn's: if `result.size() != N`, there's a cycle
- DFS: if we encounter a node with state=1 (in-progress), there's a cycle

## 6. Step-by-Step Algorithm

### Kahn's Algorithm

1. Build adjacency list from edges
2. Compute in-degree for each node
3. Push all nodes with in-degree = 0 into a queue
4. While queue is not empty: a. Pop node `u` b. Add `u` to result c. For each neighbor `v` of `u`:
  - Decrement `in_degree[v]`
  - If `in_degree[v]` becomes 0, push `v` into queue
5. If `result.size() != N` → graph has a cycle (invalid)
6. Return result

### DFS-based Topological Sort

1. Build adjacency list
2. Initialize `visited` array with 0 (unvisited)
3. For each node from 0 to N-1:
  - If `visited[u] == 0`, call `dfs(u)`
4. In `dfs(u)`: a. Set `visited[u] = 1` (in progress) b. For each neighbor `v`:
  - If `visited[v] == 1` → cycle detected
  - If `visited[v] == 0` → call `dfs(v)` c. Set `visited[u] = 2` (done) d. Push `u` to stack
5. Reverse stack to get topological order

## 7. Dry Run

Input:

Vertices: 0, 1, 2, 3, 4

Edges: 0→1, 0→2, 1→3, 2→3, 3→4

Adjacency list:

0 → [1, 2]  
1 → [3]  
2 → [3]  
3 → [4]  
4 → []

In-degree:

node 0: 0  
node 1: 1 (from 0)  
node 2: 1 (from 0)  
node 3: 2 (from 1, 2)  
node 4: 1 (from 3)

Kahn's Execution:

| Step | Queue | Pop | Result | Updates (in-degree changes)               |
|------|-------|-----|--------|-------------------------------------------|
| Init | [0]   |     | []     |                                           |
| 1    | []    | 0   | [0]    | in[1]=0→0? No, 1→0.<br>in[2]=1→0 ✓ push 2 |

Wait: actually in[1] becomes 0 after decrement from 0's edge. Let me redo carefully

Let me redo with careful tracking:

Init: in-degree = [0, 1, 1, 2, 1] Queue = [0]

| Step | Queue | Pop | Result      | Action                                                 |
|------|-------|-----|-------------|--------------------------------------------------------|
| 0    | [0]   | -   | []          | start                                                  |
| 1    | [1,2] | 0   | [0]         | from 0→1: in[1]-- (0). from 0→2: in[2]-- (0). Push 1,2 |
| 2    | [2]   | 1   | [0,1]       | from 1→3: in[3]-- (1). Not 0, no push                  |
| 3    | [3]   | 2   | [0,1,2]     | from 2→3: in[3]-- (0). Push 3                          |
| 4    | [4]   | 3   | [0,1,2,3]   | from 3→4: in[4]-- (0). Push 4                          |
| 5    | []    | 4   | [0,1,2,3,4] | no neighbors.                                          |

**Result:** [0,1,2,3,4] or any valid order. Another valid order: [0,2,1,3,4]

DFS Execution:

Start: visited = [0,0,0,0,0], stack = []

- dfs(0):
  - visited[0]=1
  - dfs(1): visited[1]=1 → dfs(3): visited[3]=1 → dfs(4): visited[4]=1 → visited[4]=2, stack=[4]
    - visited[3]=2, stack=[4,3]
  - visited[1]=2, stack=[4,3,1]
  - dfs(2): visited[2]=1 → neighbor 3 is 2 (done), skip. visited[2]=2, stack=[4,3,1,2]
  - visited[0]=2, stack=[4,3,1,2,0]

Reverse stack → [0,2,1,3,4]

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Kahn's Algorithm (BFS)
vector<int> topologicalSortKahn(int N, vector<vector<int>>& adj) {
    vector<int> inDegree(N, 0);
    for (int u = 0; u < N; ++u) {
        for (int v : adj[u]) {
            inDegree[v]++;
        }
    }

    queue<int> q;
    for (int i = 0; i < N; ++i) {
        if (inDegree[i] == 0) q.push(i);
    }

    vector<int> result;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        result.push_back(u);

        for (int v : adj[u]) {
            if (--inDegree[v] == 0) {
                q.push(v);
            }
        }
    }

    if ((int)result.size() != N) {
        // Cycle detected → no valid topological order
        return {};
    }
    return result;
}
```

```

}

// DFS-based Topological Sort
bool dfs(int u, vector<vector<int>>& adj, vector<int>& visited, vector<int>&
result) {
    visited[u] = 1; // in progress

    for (int v : adj[u]) {
        if (visited[v] == 1) return false; // cycle
        if (visited[v] == 0) {
            if (!dfs(v, adj, visited, result)) return false;
        }
    }

    visited[u] = 2; // done
    result.push_back(u);
    return true;
}

vector<int> topologicalSortDFS(int N, vector<vector<int>>& adj) {
    vector<int> visited(N, 0);
    vector<int> result;

    for (int i = 0; i < N; ++i) {
        if (visited[i] == 0) {
            if (!dfs(i, adj, visited, result)) return {}; // cycle
        }
    }

    reverse(result.begin(), result.end());
    return result;
}

// ---- Example Usage ----
int main() {
    int N = 5;
    vector<vector<int>> adj(N);
    adj[0] = {1, 2};
    adj[1] = {3};
    adj[2] = {3};
    adj[3] = {4};

    auto order = topologicalSortKahn(N, adj);
    cout << "Kahn's order: ";
    for (int v : order) cout << v << " ";
    cout << endl;

    auto order2 = topologicalSortDFS(N, adj);
    cout << "DFS order: ";
    for (int v : order2) cout << v << " ";
    cout << endl;
}

```



```
    return 0;
}
```

## 9. Python Implementation

```
from collections import deque

def topological_sort_kahn(N, adj):
    in_degree = [0] * N
    for u in range(N):
        for v in adj[u]:
            in_degree[v] += 1

    q = deque([i for i in range(N) if in_degree[i] == 0])
    result = []

    while q:
        u = q.popleft()
        result.append(u)
        for v in adj[u]:
            in_degree[v] -= 1
            if in_degree[v] == 0:
                q.append(v)

    if len(result) != N:
        return [] # cycle
    return result

def topological_sort_dfs(N, adj):
    visited = [0] * N # 0:unvisited, 1:in-progress, 2:done
    result = []

    def dfs(u):
        visited[u] = 1
        for v in adj[u]:
            if visited[v] == 1:
                return False # cycle
            if visited[v] == 0:
                if not dfs(v):
                    return False
        visited[u] = 2
        result.append(u)
        return True

    for i in range(N):
        if visited[i] == 0:
            if not dfs(i):
                return []

    return result[::-1]
```

```
# Example
N = 5
adj = [[] for _ in range(N)]
adj[0] = [1, 2]
adj[1] = [3]
adj[2] = [3]
adj[3] = [4]

print(topological_sort_kahn(N, adj))    # [0, 1, 2, 3, 4] or similar
print(topological_sort_dfs(N, adj))    # [0, 2, 1, 3, 4] or similar
```

## 10. Code Explanation

### Kahn's Algorithm

#### In-degree computation:

- For each edge  $u \rightarrow v$ , increment `inDegree[v]`.
- Nodes with `inDegree == 0` have no dependencies.

#### Queue processing:

- Pop node, add to result. This node has all its dependencies satisfied.
- For each neighbor `v`: `--inDegree[v]`. If it becomes 0, all its prerequisites are done  $\rightarrow$  push to queue.
- When a node's in-degree reaches 0 during processing, it means all nodes pointing to it have already been added to the result.

#### Cycle detection:

- If the graph has a cycle, nodes in the cycle never reach in-degree 0.
- `result.size() != N` means some nodes were never processed  $\rightarrow$  cycle exists.

### DFS-based Approach

#### Three-state visited array:

- `0`: unvisited — haven't explored this node yet
- `1`: in progress — currently exploring this node's DFS subtree
- `2`: done — all descendants processed

#### Cycle detection:

- If we encounter a node with `visited[v] == 1`, we've found a back edge  $\rightarrow$  cycle.
- This works because if `v` is currently on the recursion stack, going to `v` again means there's a cycle.

#### Adding to result:

- After processing all neighbors (post-order), add node to result.
- We're essentially recording the order in reverse: a node comes after all its descendants.
- Final `reverse()` gives the correct topological order.

### Kahn's vs DFS

| Aspect          | Kahn's (BFS)          | DFS                  |
|-----------------|-----------------------|----------------------|
| Implementation  | Simple                | Slightly trickier    |
| Cycle detection | Built-in (size check) | Built-in (back edge) |
| Order type      | "Earliest possible"   | Any valid order      |
| Stack overflow  | No                    | Possible for large N |

## 11. Complexity Analysis

| Algorithm       | Time       | Space      |
|-----------------|------------|------------|
| Build adjacency | $O(N + E)$ | $O(N + E)$ |
| Kahn's (BFS)    | $O(N + E)$ | $O(N)$     |
| DFS-based       | $O(N + E)$ | $O(N)$     |

| Aspect         | Complexity                                    |
|----------------|-----------------------------------------------|
| N = vertices   |                                               |
| E = edges      |                                               |
| <b>Overall</b> | <b><math>O(N + E)</math></b>                  |
| <b>Space</b>   | <b><math>O(N + E)</math></b> (adjacency list) |

## 12. Common Patterns

### Pattern 1: Course Schedule (Prerequisite order)

**Identify:** "Can you finish all courses given prerequisites?"

**Approach:** Topo sort; if result size = N → yes.

**Problems:** LeetCode 207 (Course Schedule), LeetCode 210 (Course Schedule II)

### Pattern 2: Dependency Resolution / Build Order

**Identify:** "Valid order to compile/process given dependencies"

**Approach:** Standard topo sort.

**Problems:** LeetCode 269 (Alien Dictionary) — build graph from word ordering, then topo sort

### Pattern 3: Longest Path in DAG

**Identify:** "Maximum time to complete all tasks given dependencies and durations"

**Approach:** Topo sort + DP:  $\text{dist}[v] = \max(\text{dist}[v], \text{dist}[u] + \text{weight}[u \rightarrow v])$  processed in topological order.

**Problems:** LeetCode 1136 (Parallel Courses), "Longest path in DAG"

### Pattern 4: Detect Cycle in Directed Graph

**Identify:** "Does this directed graph have a cycle?"

**Approach:** Kahn's → if result.size != N, cycle exists. Or DFS with 3-state visited.

**Problems:** LeetCode 207, many graph problems

## Pattern 5: Minimum Vertices to Reach All Nodes

**Identify:** "Minimum number of nodes to start from to reach all nodes in a DAG"

**Approach:** Any node with in-degree > 0 can be reached from another. Answer = nodes with in-degree = 0.

**Problems:** LeetCode 1557 (Minimum Number of Vertices to Reach All Nodes)

## 13. Common Mistakes

- **Forgetting the graph must be a DAG** — topo sort is impossible with cycles
- **Not handling disconnected graph** — Kahn's works fine; DFS needs a loop over all nodes
- **Using `vector<bool>` visited in DFS for topo sort** — need 3 states, not 2
- **Incorrect adjacency list size** — N nodes, edges may be 1-indexed; adjust indexing
- **Queue vs stack in Kahn's** — queue gives BFS order (lexicographically smallest if using min-heap)
- **Not reversing DFS result** — DFS produces reverse topological order
- **Memory for large N** — recursion in DFS may cause stack overflow; use Kahn's for large graphs
- **In-degree overflow** — in-degree ≤ N, safe with int

## 14. Edge Cases

- **Empty graph (N=0)** → return `[]`
- **Single node (N=1, no edges)** → return `[0]`
- **Disconnected DAG** — topo sort works (no edges between components)
- **Self-loop** → cycle → return `[]`
- **Multiple parallel edges** → fine, just increment in-degree each time
- **Graph already sorted** → algorithm still works, produces valid order
- **All nodes isolated** → any order is valid; both algorithms return all nodes

## 15. Variations

### 1. Lexicographically Smallest Topological Order

- Use `priority_queue<int, vector<int>, greater<int>>` (min-heap) instead of queue
- Important for problems like "smallest sequence in dictionary order"
- Time:  $O(N \log N + E)$

### 2. All Topological Sorts (Backtracking)

- Generate all valid orderings
- Use recursion + backtracking with in-degree tracking
- Exponential — only feasible for small N

### 3. Parallel Course Scheduling (Minimum Semesters)

- Process all nodes with in-degree 0 at the same "level" (semester)
- BFS level-by-level (like Kahn's but track levels)

## 4. Topological Sort with Weights (Longest Path)

- Edge weights represent duration
- After topo sort, DP:  $\text{dist}[v] = \max(\text{dist}[v], \text{dist}[u] + w)$

## 16. Related Algorithms/Data Structures

| Algorithm/DS                               | Connection                                                             |
|--------------------------------------------|------------------------------------------------------------------------|
| <b>DFS/BFS</b>                             | Fundamentals. Topo sort is special case of DFS (post-order).           |
| <b>Kahn's</b>                              | BFS-based. In-degree tracking.                                         |
| <b>Cycle Detection (directed)</b>          | Same DFS with 3-state visited.                                         |
| <b>Longest Path in DAG</b>                 | Uses topo sort as prerequisite.                                        |
| <b>Strongly Connected Components (SCC)</b> | For cyclic graphs, SCC condensation gives a DAG, then topo sort on it. |

## 17. Practice Problems

### Easy

1. **Course Schedule** — LeetCode 207  
*Detect if cycle in directed graph. Kahn's or DFS.*
2. **Course Schedule II** — LeetCode 210  
*Return a valid order. Core Kahn's implementation.*

### Medium

1. **Alien Dictionary** — LeetCode 269  
*Build graph from sorted words, then topo sort.*
2. **Parallel Courses** — LeetCode 1136  
*Minimum semesters. Kahn's with level tracking.*
3. **Sequence Reconstruction** — LeetCode 444  
*Check if a sequence is the unique topological order of a graph.*

### Hard

1. **Minimum Height Trees** — LeetCode 310  
*Kahn-like approach (topological removal of leaves) on an undirected graph.*
2. **Sort Items by Groups Respecting Dependencies** — LeetCode 1203  
*Two-level topological sort: items within groups, then groups.*

## 18. Interview Explanation

"Topological sort gives a linear ordering of a DAG such that every edge goes from earlier to later. I use either Kahn's algorithm (BFS with in-degree tracking) or DFS with post-order recording. Kahn's works by tracking in-degrees: start with nodes that have no incoming edges, process them, and decrement neighbors' in-degrees. When a neighbor's in-degree hits zero, add it to the queue. If the

result doesn't include all nodes, there's a cycle.

DFS-based topo sort runs DFS and adds a node to a stack after processing all its neighbors. Then I reverse the stack. I use a 3-state visited array to detect cycles: unvisited, in-progress, done. If I encounter a node that's in-progress, there's a cycle.

Both run in  $O(N + E)$  time. I prefer Kahn's for simplicity and because it naturally gives a BFS-like order, but DFS is more useful when I already need DFS for other reasons.

Common applications are course scheduling, build systems, and longest path problems in DAGs."

## 19. Revision Notes

- **Kahn's:** Compute in-degree → queue of 0s → process → decrement neighbors → add when 0
- **DFS:** 3-state visited (0,1,2) → post-order push to stack → reverse
- **Cycle detection:** Kahn's → `result.size != N`. DFS → encounter visited[] == 1
- **Complexity:**  $O(N + E)$  time,  $O(N + E)$  space
- **Must be DAG** — topological sort impossible with cycles
- **Lexicographically smallest** → use min-heap instead of queue
- **Trap:** DFS stack overflow for large N; DFS result needs reverse

## 20. Final Cheat Sheet

TOPOLOGICAL SORT — Linear order respecting edges in a DAG

USE WHEN: dependencies, prerequisites, build order, valid sequence

KAHN'S: in-degree array → queue of zeros → process → check `result.size == N`

DFS: 3-state visited → post-order → reverse

TIME:  $O(N + E)$       SPACE:  $O(N + E)$

CYCLE: if DFS hits visited=1, or Kahn's result too small

TRAPS: graph must be DAG; 3-state not boolean; reverse DFS result

# 5. SHORTEST PATH

## 1. Overview

**Shortest path algorithms** find the minimum-cost path between nodes in a weighted graph. Depending on the graph's properties (directed/undirected, positive/negative weights, cycles), different algorithms apply — Dijkstra, Bellman-Ford, Floyd-Warshall, BFS (for unweighted).

## 2. Intuition

**Simple explanation:** You want the cheapest (shortest, fastest) route from point A to B on a map where roads have different lengths or travel times.

**Analogy:** Dijkstra is like pouring water from a source — it spreads out evenly, reaching closer nodes before farther ones. Bellman-Ford is like making iterative improvements: "Given what I know so far, can I find a shorter path to that node by going through an extra neighbor?"

## Why they work:

- **Dijkstra:** Uses the fact that the shortest path to the closest unvisited node is finalized once visited (greedy). Requires non-negative weights.
- **Bellman-Ford:** Relaxes all edges  $N-1$  times. After  $k$  iterations, we know shortest paths using at most  $k$  edges. Since any simple path has at most  $N-1$  edges,  $N-1$  iterations suffice.
- **BFS:** In unweighted graphs, the first time we reach a node via BFS is the shortest path (queue processes level by level).
- **Floyd-Warshall:** Dynamic programming —  $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$  for all  $k$ .

## 3. When to Use It

- **Navigation / GPS** — shortest road distance
- **Network routing** — OSPF uses Dijkstra, BGP uses path-vector
- **Social networks** — shortest friend chain
- **Word ladder** — shortest transformation sequence
- **Game maps** — AI pathfinding ( $A^*$  is Dijkstra with heuristic)
- **Cheapest flight connections**
- **Detect negative cycles** — Bellman-Ford for currency arbitrage

**Trigger phrases:** "shortest path", "minimum distance", "cheapest flight", "least cost", "minimum time", "network delay", "path finding", "Dijkstra"

## 4. When Not to Use It

- **Unweighted graph** — BFS is simpler and faster ( $O(N + E)$  vs  $O(E \log N)$ )
- **All-pairs shortest paths, small  $N$  ( $\leq 500$ )** — Floyd-Warshall is simpler than running Dijkstra  $N$  times
- **Single source on a tree** — DFS/BFS is enough (trees have unique paths)
- **Graph with negative edges but no negative cycles** — Dijkstra fails, need Bellman-Ford or SPFA
- **Graph with negative cycles** — shortest path is undefined (can loop forever to reduce cost)
- **Need shortest paths in terms of edges, not weights** — BFS
- **DAG with topological order known** — DP in topological order is  $O(N + E)$

**Common wrong assumption:** "Dijkstra works with negative weights." False — once Dijkstra marks a node as visited, it assumes no future path can be shorter, which breaks with negative edges.

## 5. Core Concepts

### 5.1 Relaxation

- $\text{if } (\text{dist}[v] > \text{dist}[u] + w) \text{ dist}[v] = \text{dist}[u] + w$
- The fundamental operation of all shortest path algorithms
- Means: "I found a better path to  $v$  by going through  $u$ "

### 5.2 Priority Queue (Dijkstra)

- Min-heap storing  $(\text{distance}, \text{node})$
- Always extracts the node with smallest known distance

- Ensures we process nearest unvisited nodes first

### 5.3 Edge Relaxation Count (Bellman-Ford)

- Must relax all edges exactly  $N-1$  times
- On the  $k$ -th iteration, we find shortest paths using at most  $k$  edges
- One extra iteration detects negative cycles

### 5.4 DP Table (Floyd-Warshall)

- `dist[i][j]` = shortest distance from  $i$  to  $j$
- Updated using intermediate node  $k$
- $k$  is the outermost loop

### 5.5 Negative Cycle

- A cycle whose total weight is negative
- Shortest path is undefined — you can loop forever to get arbitrarily small distance
- Bellman-Ford detects it: if any distance improves on the  $N$ -th iteration, a negative cycle exists

## 6. Step-by-Step Algorithm

### Dijkstra (Single Source, Non-negative Weights)

1. Initialize `dist[src] = 0, dist[all others] = INF`
2. Push `(0, src)` into min-heap
3. While heap is not empty: a. Pop `(d, u)`. If `d > dist[u]`, skip (stale entry). b. For each neighbor `(v, w)`:
  - If `dist[v] > dist[u] + w`:
    - `dist[v] = dist[u] + w`
    - Push `(dist[v], v)` into heap
4. Return `dist` array

### Bellman-Ford (Single Source, Negative Weights Allowed)

1. Initialize `dist[src] = 0, dist[all others] = INF`
2. Repeat  $N-1$  times: a. For each edge `(u, v, w)`:
  - If `dist[u] != INF` and `dist[v] > dist[u] + w`:
    - `dist[v] = dist[u] + w`
3. (Optional) Detect negative cycles: for each edge, if `dist[v] > dist[u] + w`, cycle exists
4. Return `dist`

### Floyd-Warshall (All-Pairs)

1. Initialize `dist[i][j] = INF, dist[i][i] = 0`
2. For each edge `(u, v, w)`: `dist[u][v] = w`
3. For  $k = 0$  to  $N-1$ : For  $i = 0$  to  $N-1$ : For  $j = 0$  to  $N-1$ : If `dist[i][k] != INF` and `dist[k][j] != INF`: `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])`
4. Return `dist`



## Dijkstra — Graph:

Nodes: 0, 1, 2, 3, 4  
Edges ( $u \rightarrow v, w$ ):  
0 $\rightarrow$ 1: 4, 0 $\rightarrow$ 2: 1  
1 $\rightarrow$ 3: 1  
2 $\rightarrow$ 1: 2, 2 $\rightarrow$ 3: 5  
3 $\rightarrow$ 4: 3

| Step | Heap<br>(d,u) | Pop   | dist[]                  | Relaxation                                               |
|------|---------------|-------|-------------------------|----------------------------------------------------------|
| Init | (0,0)         |       | [0, INF, INF, INF, INF] |                                                          |
| 1    | (4,1)(1,2)    | (0,0) | [0, INF, INF, INF, INF] | 0→1: 0+4 < INF → dist[1]=4. 0→2: 0+1 < INF → dist[2]=1   |
| 2    | (4,1)(6,3)    | (1,2) | [0, 4, 1, INF, INF]     | 2→1: 1+2=3 < 4 → dist[1]=3. 2→3: 1+5=6 < INF → dist[3]=6 |
| 3    | (3,1)(6,3)    | (3,1) | [0, 3, 1, 6, INF]       | 1→3: 3+1=4 < 6 → dist[3]=4.                              |
| 4    | (4,3)         | (4,3) | [0, 3, 1, 4, INF]       | 3→4: 4+3=7 < INF → dist[4]=7                             |
| 5    | (7,4)         | (7,4) | [0, 3, 1, 4, 7]         | no neighbors                                             |

## 8. C++ Implementation

## Dijkstra

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;
typedef pair<int, int> pii;

vector<int> dijkstra(int src, int N, vector<vector<pii>>& adj) {
    vector<int> dist(N, INF);
    dist[src] = 0;

    priority_queue<pii, vector<pii>, greater<pii>> pq; // min-heap
    pq.push({0, src});

    while (!pq.empty()) {
```

```

    auto [d, u] = pq.top(); pq.pop();

    if (d > dist[u]) continue; // stale entry

    for (auto& [v, w] : adj[u]) {
        if (dist[v] > dist[u] + w) {
            dist[v] = dist[u] + w;
            pq.push({dist[v], v});
        }
    }
}
return dist;
}

```

## Bellman-Ford

```

vector<int> bellmanFord(int src, int N, vector<tuple<int,int,int>>& edges) {
    vector<int> dist(N, INF);
    dist[src] = 0;

    // Relax N-1 times
    for (int i = 0; i < N - 1; ++i) {
        bool updated = false;
        for (auto& [u, v, w] : edges) {
            if (dist[u] != INF && dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                updated = true;
            }
        }
        if (!updated) break; // early exit
    }

    // Check for negative cycles
    for (auto& [u, v, w] : edges) {
        if (dist[u] != INF && dist[v] > dist[u] + w) {
            // Negative cycle detected
            return {}; // or throw
        }
    }
    return dist;
}

```

## Floyd-Warshall

```

vector<vector<int>> floydWarshall(int N, vector<tuple<int,int,int>>& edges) {
    vector<vector<int>> dist(N, vector<int>(N, INF));
    for (int i = 0; i < N; ++i) dist[i][i] = 0;

    for (auto& [u, v, w] : edges) {

```

```

        dist[u][v] = min(dist[u][v], w); // handle parallel edges
    }

    for (int k = 0; k < N; ++k) {
        for (int i = 0; i < N; ++i) {
            if (dist[i][k] == INF) continue;
            for (int j = 0; j < N; ++j) {
                if (dist[k][j] == INF) continue;
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }
    return dist;
}

```

## Full Example with Dijkstra

```

int main() {
    int N = 5;
    vector<vector<pii>> adj(N);
    adj[0] = {{1,4}, {2,1}};
    adj[1] = {{3,1}};
    adj[2] = {{1,2}, {3,5}};
    adj[3] = {{4,3}};

    auto dist = dijkstra(0, N, adj);
    for (int i = 0; i < N; ++i)
        cout << "dist[0->" << i << "] = " << dist[i] << endl;
    // 0:0, 1:3, 2:1, 3:4, 4:7
    return 0;
}

```

## 9. Python Implementation

```

import heapq

INF = 10**9

def dijkstra(src, N, adj):
    dist = [INF] * N
    dist[src] = 0
    pq = [(0, src)] # (distance, node)

    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if dist[v] > dist[u] + w:

```

```

        dist[v] = dist[u] + w
        heapq.heappush(pq, (dist[v], v))

    return dist

def bellman_ford(src, N, edges):
    dist = [INF] * N
    dist[src] = 0

    for _ in range(N - 1):
        updated = False
        for u, v, w in edges:
            if dist[u] != INF and dist[v] > dist[u] + w:
                dist[v] = dist[u] + w
                updated = True
        if not updated:
            break

    # Check negative cycles
    for u, v, w in edges:
        if dist[u] != INF and dist[v] > dist[u] + w:
            return [] # negative cycle
    return dist

def floyd_warshall(N, edges):
    INF = 10**9
    dist = [[INF] * N for _ in range(N)]
    for i in range(N):
        dist[i][i] = 0
    for u, v, w in edges:
        dist[u][v] = min(dist[u][v], w)

    for k in range(N):
        for i in range(N):
            if dist[i][k] == INF:
                continue
            for j in range(N):
                if dist[k][j] == INF:
                    continue
                if dist[i][j] > dist[i][k] + dist[k][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    return dist

```

## 10. Code Explanation

### Dijkstra

#### Priority queue:

- `priority_queue<pii, vector<pii>, greater<pii>>` creates a min-heap.
- Each entry is `(distance, node)`.
- `greater<pii>` makes the smallest distance appear at the top.

**Stale entry check:**

- `if (d > dist[u]) continue` — a node may be pushed multiple times with different distances. When we pop an old (larger) distance, we skip it.
- This is essential for correctness and performance.

**Relaxation:**

- For each neighbor `(v, w)`: if we found a shorter path to v via u, update and push.

## Bellman-Ford

**N-1 iterations:**

- Why N-1? The longest simple path in a graph with N nodes has at most N-1 edges. After k iterations, we know shortest paths using  $\leq k$  edges. After N-1, we have all shortest paths.

**Early exit:**

- If no distance updates in an iteration, we're done early. Real-world graphs often converge much faster than N-1.

**Negative cycle detection:**

- Run one more pass. If any edge still relaxable  $\rightarrow$  negative cycle exists.
- `dist[v] > dist[u] + w` should be impossible after N-1 correct iterations.

## Floyd-Warshall

**k-loop order:**

- k must be the outermost loop. This is because `dist[i][j]` depends on `dist[i][k]` and `dist[k][j]` from the previous k's iteration.
- If k were innermost, we'd be using already-updated values incorrectly.

**INF checks:**

- `if (dist[i][k] == INF) continue` avoids overflow from `INF + something`.

## 11. Complexity Analysis

| Algorithm               | Time              | Space        | Constraints                |
|-------------------------|-------------------|--------------|----------------------------|
| <b>Dijkstra</b>         | $O((N+E) \log N)$ | $O(N + E)$   | Non-negative weights       |
| <b>Bellman-Ford</b>     | $O(N \times E)$   | $O(N + E)$   | Negative weights allowed   |
| <b>Floyd-Warshall</b>   | $O(N^3)$          | $O(N^2)$     | $N \leq 300-500$ typically |
| <b>BFS (unweighted)</b> | $O(N + E)$        | $O(N)$       | Unweighted graph           |
| Aspect                  | Dijkstra          | Bellman-Ford | Floyd-Warshall             |
| Single source           | ✓                 | ✓            | All-pairs                  |

| Aspect                   | Dijkstra | Bellman-Ford              | Floyd-Warshall   |
|--------------------------|----------|---------------------------|------------------|
| Negative weights         | X        | ✓                         | ✓ (no neg cycle) |
| Negative cycle detection | X        | ✓                         | ✓                |
| Large N ( $10^5$ )       | ✓        | X ( $O(N \cdot E)$ heavy) | X ( $N^3$ )      |

## 12. Common Patterns

### Pattern 1: Cheapest Flights / Network Delay

**Identify:** "Minimum cost to reach all nodes", "time for signal to reach all nodes"

**Approach:** Dijkstra from source.

**Problems:** LeetCode 743 (Network Delay Time), LeetCode 787 (Cheapest Flights Within K Stops)

### Pattern 2: Path with Maximum Probability

**Identify:** "Maximum probability path", "most reliable path"

**Approach:** Dijkstra with max-heap (or negate log of probabilities).

**Problems:** LeetCode 1514 (Path with Maximum Probability)

### Pattern 3: Detect Negative Cycles (Currency Arbitrage)

**Identify:** "Can you profit from currency exchange cycles?"

**Approach:** Bellman-Ford on log-converted exchange rates.

**Problems:** LeetCode 1368 (not direct), classic "Currency Arbitrage"

### Pattern 4: K-Shortest Paths / Limited Edges

**Identify:** "Cheapest flight with at most K stops"

**Approach:** Bellman-Ford for exactly K iterations (or DP).

**Problems:** LeetCode 787

### Pattern 5: All-Pairs / City-to-City

**Identify:** "Find the city with smallest reachability to all others", "minimum maximum distance"

**Approach:** Floyd-Warshall for  $N \leq 500$ .

**Problems:** LeetCode 1334 (Find the City With the Smallest Number of Neighbors)

### Pattern 6: 0-1 BFS

**Identify:** "Graph with edge weights 0 or 1"

**Approach:** Deque instead of priority queue. Push front for weight 0, back for weight 1.

**Problems:** Various CF problems, SPOJ

## 13. Common Mistakes

- **Dijkstra with negative weights** — Dijkstra gives wrong answers
- **Not using `long long`** — distances can overflow 32-bit int (use `long long` or `int64_t`)
- **Not skipping stale entries in Dijkstra** — causes  $O(2^N)$  worst-case

- **Bellman-Ford without early exit** — unnecessary  $O(N \times E)$  iterations
- **Floyd-Warshall with k innermost** — wrong results (uses k as intermediate incorrectly)
- **INF set too low** — actual shortest path may be  $> \text{INF}$ , causing wrong relaxations
- **INF + something overflows** — check `dist[u] != INF` before relaxing
- **Not checking for negative cycles** — may return wrong distances
- **1-indexed nodes vs 0-indexed** — subtle off-by-one errors

## 14. Edge Cases

- **Single node** → `dist[src] = 0`, all others unreachable
- **Disconnected graph** → unreachable nodes have INF distance
- **Multiple edges between same nodes** → keep minimum weight
- **Self-loop** → ignored in Dijkstra (`dist[u] > dist[u] + w` is false for  $w \geq 0$ ), but problematic for negative w
- **Zero-weight edges** → Dijkstra works fine
- **All same weight** → BFS would be faster
- **Large weights (up to  $10^9$ )** → use `long long`, `INF = LLONG_MAX/2` or `1e18`
- **Dense graph ( $E \approx N^2$ )** → Dijkstra with adjacency list is fine ( $E \log N \approx N^2 \log N$ ), but for Floyd-Warshall  $N \leq 500$

## 15. Variations

### 1. 0-1 BFS (for weights 0 or 1)

- Deque: push front for 0-weight edges, push back for 1-weight edges
- $O(N + E)$  — faster than Dijkstra for this special case

### 2. A\* Search

- Dijkstra + heuristic function `h(n)` estimating distance to target
- Reduces search space in large graphs with a known target
- Used in games, navigation

### 3. SPFA (Shortest Path Faster Algorithm)

- Queue-based Bellman-Ford optimization
- Average  $O(E)$ , worst-case  $O(N \times E)$
- Can be faster in practice but can be slow on adversarial inputs

### 4. DAG Shortest Path (DP + Topological Sort)

- If graph is a DAG, topo sort + DP gives  $O(N + E)$  shortest path
- Works with negative weights too (no cycles)

### 5. Dial's Algorithm (Dijkstra with bucket queue)

- When weights are small integers, use array of buckets instead of heap
- $O(N + E + \text{maxWeight})$  — faster than log factor

## 16. Related Algorithms/Data Structures

| Algorithm/DS                  | Connection                                                    |
|-------------------------------|---------------------------------------------------------------|
| <b>BFS</b>                    | Shortest path in unweighted graphs. $O(N + E)$ . Level-order. |
| <b>Topological Sort</b>       | Enables $O(N + E)$ shortest path on DAGs.                     |
| <b>Union-Find (DSU)</b>       | For MST (minimum spanning tree), not shortest path.           |
| <b>Segment Tree / Fenwick</b> | For shortest path on grid with range updates (advanced).      |
| <b>Heap (Priority Q)</b>      | Core data structure for Dijkstra.                             |
| <b>DP</b>                     | Bellman-Ford and Floyd-Warshall are DP algorithms.            |

## 17. Practice Problems

### Easy

1. **Network Delay Time** — LeetCode 743  
*Dijkstra from source. Find max distance to any node.*
2. **Find the City With the Smallest Number of Neighbors** — LeetCode 1334  
*Floyd-Warshall for all-pairs, count reachable within threshold.*

### Medium

1. **Cheapest Flights Within K Stops** — LeetCode 787  
*Bellman-Ford limited to K iterations (DP).*
2. **Path with Maximum Probability** — LeetCode 1514  
*Dijkstra with max-heap (or -log weights).*
3. **Number of Ways to Arrive at Destination** — LeetCode 1976  
*Dijkstra + count number of shortest paths.*

### Hard

1. **Minimum Cost to Make at Least One Valid Path in a Grid** — LeetCode 1368  
*0-1 BFS on grid with direction costs.*
2. **Kth Smallest Score / K-th Shortest Path** — LeetCode 1514 variations  
*Advanced shortest path.*

## 18. Interview Explanation

"For shortest path problems, I choose the algorithm based on graph properties.  
 For unweighted graphs, BFS is simplest and fastest —  $O(N + E)$ .  
 For weighted graphs with non-negative weights, I use Dijkstra: maintain a min-heap of (distance, node), relax edges, and skip stale entries. It runs in  $O(E \log N)$ .  
 For graphs with negative weights (but no negative cycles), I use Bellman-Ford: relax all edges  $N-1$  times. It's  $O(N \times E)$  but also detects negative cycles.  
 For all-pairs shortest paths with  $N \leq 500$ , Floyd-Warshall is elegant: three nested loops updating  $\text{dist}[i][j]$  via intermediate  $k$ .  
 One nuance: Dijkstra's correctness depends on non-negative weights. If a negative edge exists, the 'finalized' assumption breaks, and I must use Bellman-Ford instead."



## 19. Revision Notes

- **BFS:** Unweighted  $\rightarrow O(N+E)$ , queue
- **Dijkstra:** Non-negative weights  $\rightarrow$  min-heap,  $O(E \log N)$
- **Bellman-Ford:** Negative weights  $\rightarrow$  relax  $N-1 \rightarrow$  detect neg cycles,  $O(N \cdot E)$
- **Floyd-Warshall:** All-pairs,  $N \leq 500 \rightarrow$  triple loop  $k \rightarrow i \rightarrow j$ ,  $O(N^3)$
- **0-1 BFS:** Deque for weight 0/1,  $O(N+E)$
- **Key code:** `if (dist[v] > dist[u] + w) dist[v] = dist[u] + w`
- **Trap:** Dijkstra with negatives; INF overflow; stale entries
- **Edge cases:** disconnected (INF distances), self-loops, parallel edges

## 20. Final Cheat Sheet

SHORTEST PATH – Dijkstra, Bellman-Ford, Floyd-Warshall, BFS

```

BFS:      unweighted  $\rightarrow$  queue  $\rightarrow O(N+E)$ 
DIJKSTRA: non-negative  $\rightarrow$  min-heap  $\rightarrow O((N+E) \log N)$ 
           if (d > dist[u]) continue; // skip stale
BELLMAN:  any weight  $\rightarrow$  relax  $N-1$  times  $\rightarrow O(N \cdot E)$ 
           N-th pass: if any edge still relaxable  $\rightarrow$  negative cycle
FLOYD:    all-pairs,  $N \leq 500 \rightarrow$  triple loop  $k, i, j \rightarrow O(N^3)$ 

KEY:      dist[v] = min(dist[v], dist[u] + w)
TRAPS:    Dijkstra with negative weights, INF overflow, stale entries

```

## 6. BITMASK

### 1. Overview

**Bitmask** is a technique where we use the bits of an integer to represent a set, with each bit indicating the presence (1) or absence (0) of an element. Combined with DP, this forms **Bitmask DP** (also called DP over subsets) — solving problems where the state is defined by which elements have been used/selected.

### 2. Intuition

**Simple explanation:** Imagine you have  $N$  items and you need to try all possible subsets. Instead of storing a boolean array of size  $N$  (like `[true, false, true, ...]`), you store one integer where bit  $i$  is 1 if item  $i$  is selected. An integer `mask` can represent any subset of up to 64 elements.

**Analogy:** Think of a row of  $N$  light switches. Each switch is either on (1) or off (0). A 32-bit integer can represent 32 switches. Toggling switch  $i$  is `mask ^ (1 << i)`. Checking if switch  $i$  is on is `mask & (1 << i)`.

**Why it works:** Integers are the native data type of CPUs. Bitwise operations (`&`, `|`, `^`, `<<`, `>>`) are single-cycle operations —  $O(1)$  and incredibly fast. Iterating over all subsets of  $N$  elements takes  $O(2^N)$  time, but using bitmasks makes each operation  $O(1)$  instead of  $O(N)$  for array-based subset tracking.

### 3. When to Use It

- **Subset generation** — all subsets of  $N$  elements
- **DP over subsets** — TSP, Hamiltonian path, partition problems
- **Assignment problems** — assign  $N$  items to  $N$  positions/times
- **Matching in small graphs** — maximum bipartite matching with small  $N$
- **State space with small  $N$**  —  $N \leq 20$  ( $2^N \leq \sim 10^6$ )
- **$N$ -Queens and backtracking** — mask for columns/diagonals
- **Bitmask enumeration** — iterate over all subsets, supersets, submasks
- **Meet-in-the-middle** — split  $N$  into two halves of size  $N/2$ , enumerate both

**Trigger phrases:** "subset", "all combinations", " $N \leq 20$ ", "mask", "state compression", "assign", "partition into two sets", "traveling salesman"

### 4. When Not to Use It

- **$N > 20$**  ( $2^N$  becomes too large) —  $2^{20} = 1,048,576$  (feasible),  $2^{25} = 33\text{M}$  (maybe too much),  $2^{30} = 1\text{B}$  (infeasible)
- **Problem can be solved greedily** — don't use bitmask DP if simpler exists
- **Continuous / large state space** — need other DP techniques
- **Unweighted problems that don't need subsets** — simpler solutions exist
- **Problems where order within subset doesn't matter** — maybe combinations suffice
- **Memory constraints** —  $2^N \times N$  DP table can be large ( $2^{20} \times 20 \times 4 \text{ bytes} \approx 80 \text{ MB}$ )

**Common wrong assumption:** "Bitmask always means DP." No — bitmasks are useful even without DP: for representing sets in backtracking, checking subset inclusion, etc.

## 5. Core Concepts

### 5.1 Bit Mask

- An integer where bit  $i$  (0-indexed) represents element  $i$
- $\text{mask} = 0 \rightarrow$  empty set
- $\text{mask} = (1 \ll N) - 1 \rightarrow$  full set (all  $N$  elements)

### 5.2 Basic Bit Operations

| Operation                 | Expression                            |
|---------------------------|---------------------------------------|
| Check if bit $i$ is set   | $\text{mask} \& (1 \ll i)$            |
| Set bit $i$               | $\text{mask} \mid (1 \ll i)$          |
| Clear bit $i$             | $\text{mask} \& \sim(1 \ll i)$        |
| Toggle bit $i$            | $\text{mask} \wedge (1 \ll i)$        |
| Set all bits $0..N-1$     | $(1 \ll N) - 1$                       |
| Count set bits            | <code>__builtin_popcount(mask)</code> |
| Least significant set bit | $\text{mask} \& -\text{mask}$         |

| Operation  | Expression                         |
|------------|------------------------------------|
| Remove LSB | <code>mask &amp; (mask - 1)</code> |

### 5.3 Submask Enumeration

- To iterate over all submasks of a given mask:

```
for (int sub = mask; sub; sub = (sub - 1) & mask) {
    // sub is a non-empty submask
}
```

- Complexity:  $O(3^N)$  if done for all masks (which is  $2^N$  total submasks across all masks)

### 5.4 DP over Subsets

- `dp[mask]` = best value for subset represented by `mask`
- Transition: add one element to a smaller mask
- `dp[mask | (1 << i)] = min(dp[mask | (1 << i)], dp[mask] + cost(i))`

### 5.5 Bitmask DP with Last Element (TSP)

- `dp[mask][last]` = shortest path visiting nodes in `mask`, ending at `last`
- `dp[mask | (1 << v)][v] = min(dp[mask][u] + dist[u][v])`

## 6. Step-by-Step Algorithm

### Subset Generation via Bitmask

- For `mask` from 0 to  $(1 \ll N) - 1$ :
  - For each bit `i` from 0 to  $N-1$ :
    - If `mask & (1 << i)`: element `i` is in the subset `b`. Process the subset

### DP Over Subsets (General)

- Initialize `dp[0] = base_value` (empty set)
- For `mask` from 1 to  $(1 \ll N) - 1$ :
  - For each bit `i` in `mask`:
    - `prev_mask = mask ^ (1 << i)` (remove `i`)
    - `dp[mask] = min(dp[mask], dp[prev_mask] + cost(i, prev_mask))`
- Answer = `dp[(1 << N) - 1]`

### Traveling Salesman Problem (TSP)

- `dp[mask][i]` = min cost to visit cities in `mask`, ending at city `i`
- Initialize: `dp[1 << 0][0] = 0` (start at city 0)
- For `mask` from 1 to  $(1 \ll N) - 1$ :
  - For each `last` in `mask`:
    - For each `next` not in `mask`:
      - `newMask = mask | (1 << next)`
      - `dp[newMask][next] = min(dp[newMask][next], dp[mask][last] + dist[last][next])`
- Answer = min over `i` of `dp[(1<<N)-1][i] + dist[i][0]` (return to start)

## 7. Dry Run

Problem: Partition Array into Two Equal Sum Sets

Given `arr = [1, 5, 11, 5]`, can we partition into two subsets of equal sum?

Total sum = 22, target = 11.

Use bitmask DP: `dp[mask]` = sum of elements in subset.

$N = 4, 2^4 = 16$  masks.

| mask | bits  | dp[mask] | sum = 11? |
|------|-------|----------|-----------|
| 0000 | {}    | 0        |           |
| 0001 | {0}   | 1        |           |
| 0010 | {1}   | 5        |           |
| 0011 | {0,1} | 6        |           |
| 0100 | {2}   | 11       | ✓ found!  |

So subset {2} with value 11 exists. Answer = true.

### TSP Dry Run

$N = 4$  cities, distances:

```
0→1:10, 0→2:15, 0→3:20
1→0:10, 1→2:35, 1→3:25
2→0:15, 2→1:35, 2→3:30
3→0:20, 3→1:25, 3→2:30
```

Initialize: `dp[0001][0] = 0`

Processing (partial):

- mask=0001: last=0 → next=1: `dp[0011][1] = 0+10 = 10`
- mask=0001: last=0 → next=2: `dp[0101][2] = 0+15 = 15`
- mask=0001: last=0 → next=3: `dp[1001][3] = 0+20 = 20`
- mask=0011: last=1 → next=2: `dp[0111][2] = min(INF, 10+35=45) = 45`
- mask=0011: last=1 → next=3: `dp[1011][3] = min(INF, 10+25=35) = 35`
- ... continues
- mask=1111: min over i of `dp[1111][i] + dist[i][0] = min(..., ...)`

Answer would be 80 (route 0→1→3→2→0 = 10+25+30+15 = 80)

## 8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// ---- Utility functions ----
bool isSet(int mask, int i) { return mask & (1 << i); }
int setBit(int mask, int i) { return mask | (1 << i); }
int clearBit(int mask, int i) { return mask & ~(1 << i); }
int toggleBit(int mask, int i) { return mask ^ (1 << i); }
int countBits(int mask) { return __builtin_popcount(mask); }
int lsb(int mask) { return mask & -mask; }
int removeLSB(int mask) { return mask & (mask - 1); }

// ---- Partition Equal Subset Sum ----
bool canPartition(vector<int>& arr) {
    int sum = accumulate(arr.begin(), arr.end(), 0);
    if (sum & 1) return false; // odd sum can't be partitioned
    int target = sum / 2;
    int N = arr.size();

    // dp[mask] = sum of selected elements
    vector<int> dp(1 << N, 0);

    for (int mask = 0; mask < (1 << N); ++mask) {
        if (dp[mask] == target) return true;
        if (dp[mask] > target) continue; // prune
        for (int i = 0; i < N; ++i) {
            if (!(mask & (1 << i))) {
                int newMask = mask | (1 << i);
                dp[newMask] = dp[mask] + arr[i];
                // optimization: break early if found
            }
        }
    }
    return false;
}

// ---- Traveling Salesman Problem (TSP) ----
int tsp(vector<vector<int>>& dist) {
    int N = dist.size();
    vector<vector<int>> dp(1 << N, vector<int>(N, INT_MAX / 2));
    dp[1 << 0][0] = 0; // start at city 0

    for (int mask = 1; mask < (1 << N); ++mask) {
        for (int last = 0; last < N; ++last) {
            if (!(mask & (1 << last))) continue;
            if (dp[mask][last] == INT_MAX / 2) continue;

            for (int nxt = 0; nxt < N; ++nxt) {
                if (mask & (1 << nxt)) continue; // already visited
                int newMask = mask | (1 << nxt);
                dp[newMask][nxt] = min(dp[newMask][nxt],
                                         dp[mask][last] + dist[last][nxt]);
            }
        }
    }
}

```

```

    }
}

int fullMask = (1 << N) - 1;
int ans = INT_MAX;
for (int i = 1; i < N; ++i) {
    ans = min(ans, dp[fullMask][i] + dist[i][0]);
}
return ans;
}

// ---- Submask Enumeration ----
void enumerateSubmasks(int mask) {
    for (int sub = mask; sub; sub = (sub - 1) & mask) {
        // process submask
    }
    // also process sub = 0 separately if needed
}

// ---- Example ----
int main() {
    // Partition
    vector<int> arr = {1, 5, 11, 5};
    cout << "Can partition: " << canPartition(arr) << endl; // 1

    // TSP
    vector<vector<int>> dist = {
        {0, 10, 15, 20},
        {10, 0, 35, 25},
        {15, 35, 0, 30},
        {20, 25, 30, 0}
    };
    cout << "TSP min cost: " << tsp(dist) << endl; // 80

    return 0;
}

```

## 9. Python Implementation

```

def can_partition(arr):
    total = sum(arr)
    if total & 1:
        return False
    target = total // 2
    N = len(arr)

    dp = [0] * (1 << N)
    for mask in range(1 << N):
        if dp[mask] == target:
            return True

```

```

        if dp[mask] > target:
            continue
        for i in range(N):
            if not (mask & (1 << i)):
                new_mask = mask | (1 << i)
                dp[new_mask] = dp[mask] + arr[i]
        return False

def tsp(dist):
    N = len(dist)
    INF = 10**9
    dp = [[INF] * N for _ in range(1 << N)]
    dp[1 << 0][0] = 0

    for mask in range(1 << N):
        for last in range(N):
            if not (mask & (1 << last)):
                continue
            if dp[mask][last] == INF:
                continue
            for nxt in range(N):
                if mask & (1 << nxt):
                    continue
                new_mask = mask | (1 << nxt)
                dp[new_mask][nxt] = min(dp[new_mask][nxt],
                                         dp[mask][last] + dist[last][nxt])

    full_mask = (1 << N) - 1
    ans = INF
    for i in range(1, N):
        ans = min(ans, dp[full_mask][i] + dist[i][0])
    return ans

# Example
print(can_partition([1, 5, 11, 5])) # True

d = [[0,10,15,20],[10,0,35,25],[15,35,0,30],[20,25,30,0]]
print(tsp(d)) # 80

```

## 10. Code Explanation

### Partition Equal Subset Sum

**DP array:** `dp[mask]` holds the sum of elements selected in `mask`.

- `dp[0] = 0`: empty set has sum 0.
- For each mask, we try adding each unselected element. We add its value to the sum.
- If any mask's sum equals target, return true.
- Pruning: if `dp[mask] > target`, skip (can't reach target by adding more positive numbers).

**Optimization note:** This is  $O(N \times 2^N)$ . For  $N \leq 20$ , it's fine. For larger  $N$ , use meet-in-the-middle or subset-sum DP.

## TSP

**DP table:**  $dp[mask][last]$  — minimum cost to visit all cities in  $mask$ , ending at city  $last$ .

**Initialization:** Start at city 0.  $dp[1 \ll 0][0] = 0$ .

**Transition:** For each state  $(mask, last)$ , try going to an unvisited city  $nxt$ :

- $newMask = mask | (1 \ll nxt)$
- $dp[newMask][nxt] = \min(dp[newMask][nxt], dp[mask][last] + dist[last][nxt])$

**Answer:** After visiting all cities ( $fullMask$ ), compute min over  $dp[fullMask][i] + dist[i][0]$  (return to start).

**Complexity:**  $O(N^2 \times 2^N)$ . For  $N=20$ :  $\sim 400 \times 1M \approx 400M$  operations — borderline but often acceptable with optimization.

## Submask Enumeration

```
for (int sub = mask; sub; sub = (sub - 1) & mask)
```

This iterates over all non-zero submasks. The trick:  $(sub - 1) \& mask$  clears the LSB of  $sub$  and resets lower bits, giving the next smaller submask. Each iteration runs in  $O(1)$  amortized.

## 11. Complexity Analysis

| Problem                   | Time                | Space             | Constraints    |
|---------------------------|---------------------|-------------------|----------------|
| Subset enumeration        | $O(N \times 2^N)$   | $O(1)$            | $N \leq 20-25$ |
| Subset sum DP             | $O(N \times 2^N)$   | $O(2^N)$          | $N \leq 20-25$ |
| TSP                       | $O(N^2 \times 2^N)$ | $O(N \times 2^N)$ | $N \leq 20$    |
| All submasks of all masks | $O(3^N)$            | $O(1)$            | $N \leq 15-17$ |

| Aspect                      | Complexity                 |
|-----------------------------|----------------------------|
| Total masks                 | $2^N$                      |
| DP transitions              | $O(N \times 2^N)$ (simple) |
| TSP transitions             | $O(N^2 \times 2^N)$        |
| Memory (TSP)                | $N \times 2^N$ ints        |
| Submask enumeration (total) | $3^N$                      |

## 12. Common Patterns

Pattern 1: Minimum Cost to Assign Tasks (Assignment Problem)



**Identify:** "Assign N workers to N jobs, each worker has a cost for each job"

**Approach:**  $dp[mask] = \text{min cost to complete tasks in } mask$ . For each task, try each worker not yet assigned.

**Problems:** LeetCode / CP "Assignment Problem", Hungarian algorithm alternative for  $N \leq 20$

## Pattern 2: Hamiltonian Path / Cycle

**Identify:** "Path visiting every node exactly once"

**Approach:** TSP DP without returning to start.  $dp[mask][last] = \text{does path exist ending at last?}$

**Problems:** LeetCode 847 (Shortest Path Visiting All Nodes)

## Pattern 3: Partition into K Equal Sum Subsets

**Identify:** "Can we split array into K equal sum subsets?"

**Approach:** Bitmask DP.  $dp[mask] = \text{sum of elements in mask}$ . Check if mask can be completed to target.

**Problems:** LeetCode 698 (Partition to K Equal Sum Subsets)

## Pattern 4: Maximum XOR Subset

**Identify:** "Maximum XOR of a subset"

**Approach:** Linear basis (Gaussian elimination), NOT bitmask DP. But the subset enumeration is similar.

## Pattern 5: Bitmask + BFS (Shortest Path Visiting All Nodes)

**Identify:** "Shortest path that visits all nodes in a graph"

**Approach:** BFS with state =  $(\text{node}, \text{mask})$  — which nodes visited so far.

**Problems:** LeetCode 847

# 13. Common Mistakes

- **Integer overflow for  $1 \ll N$  when  $N \geq 31$**  — use  $1LL \ll N$  or  $(1 \ll N)$  only for  $N < 31$
- **Not checking if  $N > 20$**  —  $2^N$  becomes too large
- **Using `int` for mask when  $N > 31$**  — use `long long` or `unsigned int`
- **Forgetting to skip  $dp[mask][last] == INF$**  — unnecessary computations
- **Incorrect submask iteration** —  $sub = (sub - 1) \& mask$  is correct;  $sub--$  is not
- **Not handling  $mask = 0$**  — base case for empty set
- **Off-by-one in city 0 for TSP** — remembering that city 0 is starting point
- **Zero-indexed vs one-indexed nodes** — consistent indexing is critical

# 14. Edge Cases

- **$N = 0$**  → trivial, only empty set
- **$N = 1$**  → single mask (bit 0 set), DP is simple
- **All elements equal** → many subsets have same sum; DP still works
- **Large weights** → use `long long` for sums to avoid overflow
- **Zero-weight elements** → multiple subsets with same sum, DP must handle
- **$N = 20$**  →  $2^{20} = 1,048,576$  masks, DP array ~1M entries — fine for memory
- **$N = 21-25$**  →  $2^{25} = 33M$  — borderline, may need memory optimization (map or iterative)

# 15. Variations

## 1. Meet-in-the-Middle

- Split  $N$  into two halves of  $N/2$
- Enumerate subsets of each half
- Combine results (e.g., find subset closest to target sum)
- Allows  $N$  up to 30-40 for subset sum problems

## 2. SOS DP (Sum Over Subsets / Superset DP)

- For each mask, compute sum of values of all its submasks
- Not bitmask DP per se, but uses bitmask representation
- Used in competitive programming for advanced combinatorics
- Important for Codeforces problems

## 3. DP over Permutations (Factorial DP)

- `dp[mask]` where order matters — like TSP
- $N$  up to 12-15 typically

## 4. Bitmask with Gaussian Elimination (Linear Basis)

- For maximum XOR subset, not DP but linear algebra
- Uses bit representation of numbers

## 5. $DP[mask] = \text{possible (boolean)} / \text{count}$

- Instead of min/max, track feasibility or count number of ways

## 16. Related Algorithms/Data Structures

| Concept               | Connection                                                     |
|-----------------------|----------------------------------------------------------------|
| <b>Backtracking</b>   | Bitmask used in pruning / state tracking in recursion          |
| <b>Subset Sum</b>     | Classical DP. Bitmask gives all subsets (good for small $N$ ). |
| <b>Meet-in-Middle</b> | Combines with bitmask for larger $N$ .                         |
| <b>SOS DP</b>         | Advanced subset enumeration. $3^N$ to $2^N \times N$ .         |
| <b>Linear Basis</b>   | XOR subset problems. Not DP but uses bit representation.       |

## 17. Practice Problems

### Easy

1. **Power Set (Subsets)** — LeetCode 78  
*Generate all subsets using bitmask iteration.*
2. **Number of 1 Bits** — LeetCode 191  
*Popcount. `__builtin_popcount`.*

### Medium

1. **Partition to K Equal Sum Subsets** — LeetCode 698  
*Bitmask DP, backtracking with pruning.*
2. **Shortest Path Visiting All Nodes** — LeetCode 847  
*BFS + bitmask state: `(node, mask)`.*
3. **Maximum Length of a Concatenated String with Unique Characters** — LeetCode 1239  
*Bitmask for character sets.*

## Hard

1. **Traveling Salesman Problem** — LeetCode (no exact match) / Codeforces / SPOJ  
*Classic TSP DP:  $O(N^2 \times 2^N)$ .*
2. **Minimum Cost to Connect Two Groups of Points** — LeetCode 1595  
*Bitmask DP for assignment/bipartite matching.*

## 18. Interview Explanation

"Bitmask DP uses an integer where each bit represents whether an element is selected. This lets us compactly represent subsets and do dynamic programming over them.

The core idea: `dp[mask]` stores the optimal value for subset `mask`. Transitions add one element:

`dp[mask | (1<<i)] = min(dp[mask | (1<<i)], dp[mask] + cost(i)).`

I use this when  $N \leq 20$  and I need to try all subsets — like partition problems, traveling salesman, or assignment problems. The complexity is  $O(N \times 2^N)$  or  $O(N^2 \times 2^N)$ .

Key bit operations: checking if bit `i` is set (`mask & (1<<i)`), setting it (`mask | (1<<i)`), clearing it (`mask & ~(1<<i)`), and enumerating submasks (`for sub = mask; sub; sub = (sub-1) & mask`).

The main limitation is  $N$ :  $2^{20} \approx 1$  million (feasible), but  $2^{25} \approx 33$  million (borderline). For larger  $N$ , I'd use meet-in-the-middle or other techniques."

## 19. Revision Notes

- **Mask = integer representing subset.** Bit `i` = element `i` selected.
- $N \leq 20$  for DP,  $N \leq 30$  for meet-in-the-middle.
- **Popcount:** `__builtin_popcount(mask)` (C++) or `bin(mask).count('1')` (Python)
- **Submask enumeration:** `for sub = mask; sub; sub = (sub-1) & mask`
- **TSP:** `dp[mask][last]`, transition to `dp[mask | (1<<nxt)][nxt]`
- **Key code:** `mask & (1 << i)` to check; `mask | (1 << i)` to set
- **Overflow:** `1 << N` for  $N \geq 31$  needs `1LL << N`

## 20. Final Cheat Sheet

BITMASK – Subset state compression with integers

USE WHEN:  $N \leq 20$ , need all subsets, TSP, assignment, partition

OPS: `mask & (1<<i)` = check, `mask | (1<<i)` = set, `mask ^ (1<<i)` = toggle

POPCOUNT: `__builtin_popcount(mask)`

SUBMASK: `for(int sub=mask; sub; sub=(sub-1)&mask)`

DP: `dp[mask]` = best for subset `mask`

TSP DP: `dp[mask][last]`,  $O(N^2 \times 2^N)$

TIME:  $O(N \times 2^N)$  or  $O(N^2 \times 2^N)$  SPACE:  $O(2^N)$  or  $O(N \times 2^N)$   
 TRAPS:  $1 < N$  overflow for  $N \geq 31$ , INF handling,  $N > 20$  infeasible

## 7. TREE DP

### 1. Overview

**Tree DP** (Dynamic Programming on Trees) is the application of DP on a tree structure, where we compute values for each node based on its children (bottom-up) or parent (top-down), using the tree's natural recursive structure. Since trees have no cycles, DP transitions are clean and unambiguous.

### 2. Intuition

**Simple explanation:** In a tree, each node's answer often depends on the answers of its children (subtree). You can recursively compute the answer for each node after computing it for all children. This is DFS with memoization built into the tree structure.

**Analogy:** Think of a company hierarchy. To determine the maximum total salary bonus for a department (node), you first compute the optimal bonus for each team (child subtree). Then you decide: do I include the manager (current node) or not? The children's results feed directly into the parent's decision.

**Why it works:** Trees are naturally recursive. Every node is the root of its own subtree. DP on trees uses post-order DFS: compute for all children, then combine results for the current node. Since there are no cycles, each node is visited exactly once, and its value depends only on its children.

### 3. When to Use It

- **Tree diameter, tree height, subtree sizes**
- **Maximum sum path with constraints** — house robber / independent set on tree
- **Tree DP with rerooting** — compute a metric for every node as root
- **Maximum matching in tree** — DP with states (selected/not selected)
- **Tree distance problems** — sum of distances from each node to all others
- **Tree coloring/painting with constraints** — adjacent nodes can't have same color
- **Centroid decomposition problems** — often combined with DP
- **DP on tree with knapsack** — selecting items from tree with capacity

**Trigger phrases:** "tree", "subtree", "rooted tree", "maximum sum path in tree", "tree house robber", "tree diameter", "reroot", "independent set on tree"

### 4. When Not to Use It

- **Non-tree graph** — use graph DP (more complex, may need topological order for DAG)
- **Small N, brute force works** —  $N \leq 15$ , maybe just try all subsets
- **Graph is not connected** — run DP on each component separately, but this is fine
- **Tree with cycles** — a tree is acyclic by definition
- **Problem requires path (not subtree)** — sometimes you need DP on paths within a tree (use centroid decomposition or heavy-light decomposition)

**Common wrong assumption:** "Tree DP always uses bottom-up." True for most cases, but some problems also need top-down (reroot DP).

## 5. Core Concepts

### 5.1 Subtree DP (Bottom-up)

- DFS from root, recursively compute for children
- Combine children's results to compute parent's result
- Post-order traversal (process children before parent)

### 5.2 Rerooting DP (Top-down)

- Compute initial DP with root = 0 (or any)
- Do a second DFS to propagate "parent's contribution" to children
- Used for "sum of distances from each node" type problems

### 5.3 DP States

- Common states:  $dp[u][0]$  = best without taking u,  $dp[u][1]$  = best taking u
- Or:  $dp[u]$  = subtree value,  $up[u]$  = contribution from outside subtree

### 5.4 Tree Diameter

- Can be computed with two DFS (BFS from any node → farthest, BFS from that → diameter)
- Also computable with DP: at each node, track top two heights of children

### 5.5 DP on Rooted vs Unrooted Tree

- Most tree DP assumes a rooted tree
- For unrooted, pick any node as root (usually 0 or 1)

## 6. Step-by-Step Algorithm

### House Robber III (Maximum sum with no adjacent nodes)

1. Root the tree at 0
  2. DFS from root: a. For each node  $u$ :
    - $take[u] = value[u]$  (rob this house)
    - $skip[u] = 0$  (don't rob this house)
    - For each child  $v$ :
      - DFS(v) first
      - $take[u] += skip[v]$  (if we take u, can't take child)
      - $skip[u] += \max(take[v], skip[v])$  (if we skip u, can either take or skip child)
- Return  $\max(take[u], skip[u])$

### Tree Diameter (via DP)

1. Root the tree
2.  $height[u]$  = longest distance from u to any leaf in its subtree

3. For each node  $u$ :
  - Find the longest and second longest height among children
  - $\text{diameter\_passing\_through\_u} = \text{longest} + \text{second\_longest}$  (or just  $\text{longest}$  if one child)
  - Global diameter = max of all such values
4. Also useful:  $\text{height}[u] = 1 + \max(\text{height}[\text{child}])$

## Rerooting (Sum of distances from each node)

1. First DFS:  $\text{subSize}[u]$  = size of subtree at  $u$ .  $\text{sumDown}[u]$  = sum of distances from  $u$  to all nodes in its subtree.
2. Second DFS:  $\text{sumAll}[u]$  = sum of distances from  $u$  to all nodes in the tree.
  - For root:  $\text{sumAll}[\text{root}] = \text{sumDown}[\text{root}]$
  - For child  $v$  of  $u$ :
    - When moving from  $u$  to  $v$ , distances to nodes in  $v$ 's subtree decrease by 1 each, distances to nodes outside  $v$ 's subtree increase by 1 each.
    - $\text{sumAll}[v] = \text{sumAll}[u] - \text{subSize}[v] + (N - \text{subSize}[v])$

## 7. Dry Run

### House Robber III

Tree:



Root at 0. Children:

- 0: children [1, 2]
- 1: child [3]
- 2: child [4]
- 3: leaf
- 4: leaf

Values: [3, 2, 3, 3, 1]

DFS:

- Node 3 (leaf):  $\text{take}=3$ ,  $\text{skip}=0 \rightarrow \text{max}=3$
- Node 4 (leaf):  $\text{take}=1$ ,  $\text{skip}=0 \rightarrow \text{max}=1$
- Node 1:  $\text{take}=2 + \text{skip}[3]=0 = 2$ .  $\text{skip}=0 + \max(3,0)=3. \rightarrow \text{max}=3$
- Node 2:  $\text{take}=3 + \text{skip}[4]=0 = 3$ .  $\text{skip}=0 + \max(1,0)=1. \rightarrow \text{max}=3$
- Node 0:  $\text{take}=3 + \text{skip}[1]+\text{skip}[2] = 3+3+1 = 7$ .  $\text{skip}=0 + \max(3,3)+\max(3,1)=3+3=6. \rightarrow \text{max}=7$

Answer = 7 (rob nodes 1, 2, 3  $\rightarrow 2+3+3 = 8$ ? Wait let me recheck)

Actually:  $\text{take}[0]=3+\text{skip}[1]+\text{skip}[2]$

- $\text{skip}[1] = \max(\text{take}[1], \text{skip}[1])$ ... wait,  $\text{skip}[1]$  is the value when we don't take node 1.
- So  $\text{skip}[1] = \max(\text{take}[1], \text{skip}[1])$ ... hmm let me use different variable names.

Let  $\text{take}[u]$  = value when we rob  $u$ . Then we can't rob children. Let  $\text{skip}[u]$  = value when we don't rob  $u$ . Then we can take or skip children.

- $\text{skip}[1] = \max(\text{take}[1], \text{skip}[1])$  ... This is confusing. Let me use:
  - $\text{dp}[u][0]$  = max value from subtree of  $u$ ,  $u$  is NOT selected
  - $\text{dp}[u][1]$  = max value from subtree of  $u$ ,  $u$  IS selected
- Leaf 3:  $\text{dp}[3][0]=0, \text{dp}[3][1]=3 \rightarrow \max=3$
- Leaf 4:  $\text{dp}[4][0]=0, \text{dp}[4][1]=1 \rightarrow \max=1$
- Node 1:  $\text{dp}[1][0] = \text{sum of } \max(\text{dp}[\text{child}][0], \text{dp}[\text{child}][1]) = \max(3,0)+\max(3,3)=0+3=3$ .  $\text{dp}[1][1] = \text{val}[1]=2 + \text{sum of } \text{dp}[\text{child}][0] = 2+0=2$ .  $\max=3$
- Wait, that gives 3 for node 1. Node 3 is 3, node 0 would be:  $\text{dp}[0][0] = \max(\text{dp}[1][0], \text{dp}[1][1]) + \max(\text{dp}[2][0], \text{dp}[2][1]) = \max(3,2) + \max(3,3) = 3+3 = 6$   $\text{dp}[0][1] = \text{val}[0]=3 + \text{dp}[1][0] + \text{dp}[2][0] = 3 + 3 + 3 = 9$   $\max = 9$

So answer = 9 (rob root 3 and leaf 3 and leaf 1 =  $3+3+1 = 7$ ? No,  $\text{dp}[1][0]=3$  means when node 1 is not robbed, the best from its subtree is 3 (by robbing node 3).  $\text{dp}[2][0]=3$  means when node 2 is not robbed, best from its subtree is 3 (by robbing node 4? No, node 4 has value 1).

Let me re-examine node 2's children: node 4 has value 1.  $\text{dp}[4][0]=0, \text{dp}[4][1]=1$   $\text{dp}[2][0] = \max(\text{dp}[4][0], \text{dp}[4][1]) = \max(0,1) = 1$   $\text{dp}[2][1] = 3 + \text{dp}[4][0] = 3 + 0 = 3$

So  $\text{dp}[0][0] = \max(3,2) + \max(1,3) = 3+3 = 6$   $\text{dp}[0][1] = 3 + \text{dp}[1][0] + \text{dp}[2][0] = 3 + 3 + 1 = 7$

Answer =  $\max(6,7) = 7$  (rob root 3, skip child 1 (rob grandchild 3), skip child 2 (rob root 2)  $\rightarrow 3+3+3 = 9$ ? No,  $\text{dp}[1][0]=3$  means we skip node 1 and take node 3, giving 3.  $\text{dp}[2][0]=1$  means we skip node 2... wait  $\text{dp}[2][0] = \max(\text{dp}[4][0], \text{dp}[4][1]) = \max(0,1) = 1$  (skip node 2, take node 4, value 1).

So: take 0 (3), skip 1 (take 3, value 3), skip 2 (take 4, value 1) =  $3+3+1 = 7$ . ✓

## 8. C++ Implementation

### House Robber III (Tree Independent Set)

```
#include <bits/stdc++.h>
using namespace std;

// dp[u][0] = max value from subtree u, u NOT taken
// dp[u][1] = max value from subtree u, u IS taken
pair<int,int> dfs(int u, int parent, vector<vector<int>>& adj, vector<int>& val) {
    int take = val[u];    // if we take u
    int skip = 0;         // if we skip u

    for (int v : adj[u]) {
        if (v == parent) continue;
    }
}
```

```

        auto [child_skip, child_take] = dfs(v, u, adj, val);
        // child_skip = dp[v][0], child_take = dp[v][1]
        take += child_skip;           // can't take child if we take u
        skip += max(child_skip, child_take); // can take or skip child
    }
    return {skip, take}; // {dp[u][0], dp[u][1]}
}

int rob(vector<int>& nums, vector<vector<int>>& adj) {
    auto [skip_root, take_root] = dfs(0, -1, adj, nums);
    return max(skip_root, take_root);
}

```

## Tree Diameter

```

pair<int,int> dfsDiameter(int u, int parent, vector<vector<int>>& adj) {
    int maxHeight1 = 0, maxHeight2 = 0;
    int diameter = 0;

    for (int v : adj[u]) {
        if (v == parent) continue;
        auto [childHeight, childDiameter] = dfsDiameter(v, u, adj);
        diameter = max(diameter, childDiameter);

        if (childHeight > maxHeight1) {
            maxHeight2 = maxHeight1;
            maxHeight1 = childHeight;
        } else if (childHeight > maxHeight2) {
            maxHeight2 = childHeight;
        }
    }

    // Diameter passing through this node
    diameter = max(diameter, maxHeight1 + maxHeight2);
    // Height of this node
    int height = 1 + maxHeight1;
    return {height, diameter};
}

int treeDiameter(vector<vector<int>>& adj) {
    return dfsDiameter(0, -1, adj).second;
}

```

## Rerooting: Sum of Distances

```

void dfsSumDown(int u, int parent, vector<vector<int>>& adj,
                vector<long long>& subSize, vector<long long>& sumDown) {
    subSize[u] = 1;
    sumDown[u] = 0;
}

```



```

    for (int v : adj[u]) {
        if (v == parent) continue;
        dfsSumDown(v, u, adj, subSize, sumDown);
        subSize[u] += subSize[v];
        sumDown[u] += sumDown[v] + subSize[v];
    }
}

void dfsReroot(int u, int parent, vector<vector<int>>& adj,
              vector<long long>& subSize, vector<long long>& sumDown,
              vector<long long>& sumAll, int N) {
    for (int v : adj[u]) {
        if (v == parent) continue;
        // When moving root from u to v:
        sumAll[v] = sumAll[u] - subSize[v] + (N - subSize[v]);
        dfsReroot(v, u, adj, subSize, sumDown, sumAll, N);
    }
}

vector<long long> sumOfDistances(int N, vector<vector<int>>& adj) {
    vector<long long> subSize(N), sumDown(N), sumAll(N);
    dfsSumDown(0, -1, adj, subSize, sumDown);
    sumAll[0] = sumDown[0];
    dfsReroot(0, -1, adj, subSize, sumDown, sumAll, N);
    return sumAll;
}

```

## Full Example

```

int main() {
    // Build a tree
    int N = 5;
    vector<vector<int>> adj(N);
    vector<pair<int,int>> edges = {{0,1},{0,2},{1,3},{1,4}};
    for (auto& [u,v] : edges) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    // House robber
    vector<int> robValues = {3, 2, 3, 3, 1};
    cout << "Max robbery: " << rob(robValues, adj) << endl; // depends on tree

    // Diameter
    cout << "Diameter: " << treeDiameter(adj) << endl;

    // Sum of distances
    auto sums = sumOfDistances(N, adj);
    for (int i = 0; i < N; ++i)
        cout << "sumDist from " << i << ": " << sums[i] << endl;
}

```

```
    return 0;
}
```

## 9. Python Implementation

```
def tree_robber(adj, values):
    def dfs(u, parent):
        take = values[u]
        skip = 0
        for v in adj[u]:
            if v == parent:
                continue
            child_skip, child_take = dfs(v, u)
            take += child_skip
            skip += max(child_skip, child_take)
        return skip, take

    skip_root, take_root = dfs(0, -1)
    return max(skip_root, take_root)

def tree_diameter(adj):
    def dfs(u, parent):
        h1 = h2 = 0
        diam = 0
        for v in adj[u]:
            if v == parent:
                continue
            child_h, child_d = dfs(v, u)
            diam = max(diam, child_d)
            if child_h > h1:
                h1, h2 = child_h, h1
            elif child_h > h2:
                h2 = child_h
            diam = max(diam, h1 + h2)
        return 1 + h1, diam
    return dfs(0, -1)[1]

def sum_of_distances(N, adj):
    sub = [0] * N
    down = [0] * N
    all_sum = [0] * N

    def dfs1(u, p):
        sub[u] = 1
        for v in adj[u]:
            if v == p:
                continue
            dfs1(v, u)
            sub[u] += sub[v]
            down[u] += down[v] + sub[v]
```

```

def dfs2(u, p):
    for v in adj[u]:
        if v == p:
            continue
        all_sum[v] = all_sum[u] - sub[v] + (N - sub[v])
        dfs2(v, u)

dfs1(0, -1)
all_sum[0] = down[0]
dfs2(0, -1)
return all_sum

```

## 10. Code Explanation

### House Robber / Tree Independent Set

#### State definition:

- $dp[u][0]$  (skip): best value in subtree of  $u$  when  $u$  is **not** selected
- $dp[u][1]$  (take): best value in subtree of  $u$  when  $u$  **is** selected

#### Transition:

- If we take  $u$ : we cannot take any child. So  $take = val[u] + \sum(dp[child][0])$
- If we skip  $u$ : we can take or skip each child. So  $skip = \sum(\max(dp[child][0], dp[child][1]))$

**Why this works:** The decision at each node only depends on its children's states, and there's no circular dependency (tree property).

### Tree Diameter via DP

**Key insight:** For each node, the longest path in the tree either:

- Passes through this node (sum of two longest child heights)
- Or is entirely within one child's subtree

We track the top two heights from children and take their sum as the candidate diameter through this node.

### Rerooting

**First DFS:** Compute  $sumDown[u]$  — sum of distances from  $u$  to all nodes in its subtree.

- $sumDown[u] = \sum(sumDown[child] + subSize[child])$
- Each child's nodes are all 1 unit farther from  $u$  than from the child.

**Second DFS:** Compute  $sumAll[u]$  — sum of distances from  $u$  to ALL nodes.

- When moving root from  $u$  to  $v$ :
  - For nodes in  $v$ 's subtree, distance decreases by 1: subtract  $subSize[v]$
  - For all other nodes (outside  $v$ 's subtree), distance increases by 1: add  $N - subSize[v]$
- Formula:  $sumAll[v] = sumAll[u] - subSize[v] + (N - subSize[v])$

## 11. Complexity Analysis

| Problem                   | Time   | Space  |
|---------------------------|--------|--------|
| House Robber (tree)       | $O(N)$ | $O(N)$ |
| Tree Diameter             | $O(N)$ | $O(N)$ |
| Sum of Distances (reroot) | $O(N)$ | $O(N)$ |
| Tree DP (general)         | $O(N)$ | $O(N)$ |

| Aspect                 | Complexity                 |
|------------------------|----------------------------|
| $N$ = nodes            |                            |
| Each node visited once | $O(N)$                     |
| DFS recursion stack    | $O(N)$ worst (skewed tree) |
| DP array size          | $O(N)$                     |

## 12. Common Patterns

### Pattern 1: Independent Set / Maximum Sum (No Adjacent)

**Identify:** "Select nodes with maximum sum, no two adjacent"

**Approach:**  $dp[u][0]$ ,  $dp[u][1]$  — take/skip states.

**Problems:** LeetCode 337 (House Robber III)

### Pattern 2: Tree Diameter

**Identify:** "Longest path between any two nodes in a tree"

**Approach:** Two DFS or DP (track top two heights).

**Problems:** LeetCode 543 (Diameter of Binary Tree)

### Pattern 3: Rerooting DP (Sum of Distances)

**Identify:** "For each node, compute something that depends on the whole tree"

**Approach:** Two-pass DFS: compute down, then reroot.

**Problems:** LeetCode 834 (Sum of Distances in Tree), CSES Tree Distances

### Pattern 4: Maximum Path Sum Between Any Two Nodes

**Identify:** "Maximum path sum (any path in tree, not necessarily through root)"

**Approach:** For each node, compute max path through it =  $left\_max + right\_max + val[u]$ .

**Problems:** LeetCode 124 (Binary Tree Maximum Path Sum)

### Pattern 5: Tree Knapsack

**Identify:** "Select items from a tree with capacity constraint"

**Approach:**  $DP[u][c]$  = best value from subtree  $u$  using capacity  $c$ . Complexity depends on knapsack variant.

## 13. Common Mistakes

- **Not passing parent parameter** — DFS goes infinite in undirected tree if you don't track parent
- **Forgetting base case** — leaf nodes have no children; dp values are just  $val[u]$
- **Incorrect reroot formula** — mixing up  $subSize$  and  $N-subSize$
- **Integer overflow for large trees** — sum of distances can be  $O(N^2)$ , use long long
- **Recursion depth for large N** — N up to  $2 \times 10^5$  may cause stack overflow; use iterative stack or increase stack limit
- **Not handling multiple components** — if tree is disconnected, run DP per component
- **DP array initialization** — forgetting to initialize for leaf nodes

## 14. Edge Cases

- **Single node (N=1)** → diameter = 0, distance sum = 0
- **Linear tree (path)** → worst-case recursion depth
- **Star tree (root + N-1 leaves)** → rerooting works fine
- **Complete binary tree** → balanced recursion
- **Negative values in max path sum** → dp must handle negative (max can be negative)
- **Unweighted vs weighted** — adjust distance formulas

## 15. Variations

### 1. DP on Tree with Multiple States

- 3 states:  $dp[u][0/1/2]$  for different conditions (color, type)
- Example: Tree coloring with no two adjacent same color

### 2. Tree DP with Knapsack

- $dp[u][c]$  = best with capacity c in subtree u
- Often  $O(N \times K^2)$  using subtree size optimization (tree DP with knapsack merge)
- Important for CP (e.g., "Tree Shopping" problems)

### 3. Centroid Decomposition

- Repeatedly find centroid, process paths through centroid, delete centroid and recurse
- Used for complex path queries (distance  $\leq K$ , etc.)
- Advanced CP topic

### 4. Heavy-Light Decomposition + DP

- Segment tree on chains of the tree
- For queries/updates on paths

### 5. DP on Tree with XOR / Bitmask

- Each node has a value, find paths with XOR = 0
- Combine with map for prefix XOR counting

## 16. Related Algorithms/Data Structures

| Concept                  | Connection                                           |
|--------------------------|------------------------------------------------------|
| <b>DFS (post-order)</b>  | Foundation of tree DP. Children before parent.       |
| <b>BFS / Level Order</b> | For some problems, level-order works instead of DFS. |
| <b>Rerooting</b>         | Second DFS with parent's contribution.               |
| <b>Tree Diameter</b>     | Can be computed via DP or two BFS.                   |
| <b>Centroid Decomp</b>   | Advanced tree DP. Break tree into balanced subtrees. |
| <b>HLD</b>               | Chain-based tree queries.                            |

## 17. Practice Problems

### Easy

1. **Diameter of Binary Tree** — LeetCode 543  
*Top two heights from each node.*
2. **Balanced Binary Tree** — LeetCode 110  
*Height difference check.*

### Medium

1. **House Robber III** — LeetCode 337  
*Tree independent set with take/skip states.*
2. **Binary Tree Maximum Path Sum** — LeetCode 124  
*Max path through any node. Track max single + max through node.*
3. **Sum of Distances in Tree** — LeetCode 834  
*Classic rerooting problem.*

### Hard

1. **Tree DP (K-grade)** — Codeforces / AtCoder  
*Various problems tagged "dp on trees".*
2. **Minimum Score After Removals on a Tree** — LeetCode 2059  
*Tree DP with bitmask states.*

## 18. Interview Explanation

"Tree DP uses the tree's recursive structure: compute the answer for each node by combining answers from its children. I do a post-order DFS, compute for children first, then combine results for the current node.

For example, House Robber III: at each node I compute two values — the max if I take this node (add children's skip values) and the max if I skip it (add children's best).

For problems that need a value for every node as root, I use rerooting: first compute subtree distances in one pass, then do a second pass where I propagate the parent's contribution to each child using the formula  $\text{sumAll}[\text{child}] = \text{sumAll}[\text{parent}] - \text{subSize}[\text{child}] + (N - \text{subSize}[\text{child}])$ .

Time complexity is always  $O(N)$  since each node is visited once in each DFS pass. The key trick is to define the DP state correctly for the specific problem."

## 19. Revision Notes

- **Bottom-up:** DFS post-order. Compute children, then parent.
- **Reroot:** 2 passes. Down (subtree) + across (parent's contribution).
- **Take/Skip pattern:**  $\text{take}[u] = \text{val}[u] + \text{sum}(\text{skip}[\text{child}])$ ,  $\text{skip}[u] = \text{sum}(\max(\text{take}[\text{child}], \text{skip}[\text{child}]))$
- **Diameter:** Top two heights from each node  $\rightarrow$  sum  $\rightarrow$  global max.
- **Complexity:**  $O(N)$  per pass.
- **Trap:** parent tracking in DFS, recursion depth, long long for distances.

## 20. Final Cheat Sheet

TREE DP – Dynamic programming on tree structure

BOTTOM-UP: DFS post-order: compute children, combine for parent

TAKE/SKIP:  $\text{take}[u] = \text{val}[u] + \text{sum}(\text{skip}[\text{child}])$ ,

$\text{skip}[u] = \text{sum}(\max(\text{take}[\text{child}], \text{skip}[\text{child}]))$

REROOT:  $\text{sumAll}[v] = \text{sumAll}[u] - \text{subSize}[v] + (N - \text{subSize}[v])$

DIAMETER: top 2 heights from each child  $\rightarrow h_1 + h_2$

TIME:  $O(N)$  per DFS pass      SPACE:  $O(N)$

TRAPS: recursion depth (use iterative or increase limit), long long for sums

## 8. LCA (LOWEST COMMON ANCESTOR)

### 1. Overview

**Lowest Common Ancestor (LCA)** of two nodes in a rooted tree is the deepest node that is an ancestor of both. The LCA problem is fundamental: given two nodes, find their deepest common ancestor efficiently — ideally  $O(\log N)$  per query after  $O(N \log N)$  preprocessing.

### 2. Intuition

**Simple explanation:** In a family tree, the LCA of two people is the closest common relative — like their common grandparent, not a distant great-great-grandparent.

**Analogy:** Think of a tree's root at the top and leaves at the bottom. Draw a line from each of the two nodes upward toward the root. The point where the two paths meet is the LCA — the deepest (lowest) node where the paths converge.

**Why it works (Binary Lifting):** Every node can be represented by its ancestors at powers of two: parent, grandparent (2 steps), great-great-grandparent (4 steps), etc. By precomputing  $\text{up}[u][k] = 2^k\text{-th ancestor of } u$ , we can "jump" any distance in  $O(\log N)$  time. To find LCA:

1. Bring both nodes to the same depth using binary lifting.
2. Binary search for the LCA: try largest jumps where ancestors differ; the node just after they all differ is the LCA.

**Alternative (Euler Tour + RMQ):** Do a DFS, record each node's entry time and depth. The LCA of  $u$  and  $v$  is the node with minimum depth in the range between their entry times in the Euler tour. Use a segment tree or sparse table for  $O(1)$  range minimum query.

### 3. When to Use It

- **Distance between two nodes** in a tree:  $\text{dist}(u, v) = \text{depth}[u] + \text{depth}[v] - 2 * \text{depth}[\text{lca}]$
- **Path between two nodes** — describe the path via LCA
- **Tree queries** — "Is node  $u$  an ancestor of node  $v$ ?"
- **Subtree queries** — using entry/exit times ( $\text{tin}/\text{tout}$ )
- **K-th ancestor** — find the  $k$ -th ancestor of a node
- **Tree DP where LCA is needed** — e.g., "sum of values on path between  $u$  and  $v$ "
- **Parity / path problems** — path length parity, path XOR, etc.

**Trigger phrases:** "lowest common ancestor", "LCA", "distance between nodes in tree", "path in tree", "ancestor query", "k-th ancestor"

### 4. When Not to Use It

- **Non-tree graph** — LCA is defined only for trees
- **Single query** — just do a DFS to find the path between two nodes  $O(N)$
- **Small tree ( $N \leq 1000$ )** —  $O(N)$  per query is fine, no need for preprocessing
- **Unrooted tree without parent info** — root it first arbitrarily
- **Dynamic tree (nodes added/removed)** — use HLD or Link-Cut Tree

**Common wrong assumption:** "LCA requires binary lifting." Euler tour + RMQ with sparse table gives  $O(1)$  queries with  $O(N \log N)$  preprocessing. Both are valid.

## 5. Core Concepts

### 5.1 Depth

- $\text{depth}[u]$  = distance from root to  $u$  (root depth = 0)
- Key for leveling: we bring both nodes to same depth before finding LCA

### 5.2 Binary Lifting Table

- $\text{up}[u][k] = 2^k$ -th ancestor of node  $u$
- Base:  $\text{up}[u][0] = \text{parent of } u$  (or -1 for root)
- Recurrence:  $\text{up}[u][k] = \text{up}[\text{up}[u][k-1]][k-1]$
- Table size:  $N \times (\log_2 N + 1)$

### 5.3 Euler Tour + RMQ

- DFS records entry time, depth, and node ID at each entry
- LCA is RMQ on depth between entry times of  $u$  and  $v$
- Use sparse table for  $O(1)$  query

### 5.4 tin/tout (Entry/Exit Times)



- $\text{tin}[u]$  = time when DFS enters  $u$
- $\text{tout}[u]$  = time when DFS exits  $u$
- $u$  is ancestor of  $v$  iff  $\text{tin}[u] \leq \text{tin}[v] \ \&\& \ \text{tout}[v] \leq \text{tout}[u]$

## 5.5 K-th Ancestor

- Express  $k$  in binary; for each set bit  $i$ , jump  $\text{up}[u][i]$
- $u = \text{up}[u][i]$  for each set bit in  $k$

## 6. Step-by-Step Algorithm

### Binary Lifting LCA

#### Preprocessing (DFS):

1. Set  $\text{up}[\text{root}][0] = \text{root}$  (or -1)
2. For each node  $u$ , for  $k = 1$  to  $\text{LOG}-1$ :
  - $\text{up}[u][k] = \text{up}[\text{up}[u][k-1]][k-1]$
3. Record  $\text{depth}[u]$

#### LCA Query:

1. If  $\text{depth}[u] < \text{depth}[v]$ , swap (make  $u$  the deeper one)
2. Lift  $u$  up to depth of  $v$ :
  - $\text{diff} = \text{depth}[u] - \text{depth}[v]$
  - For each bit  $i$  in  $\text{diff}$ : if bit is set,  $u = \text{up}[u][i]$
3. If  $u == v$ , return  $u$
4. For  $k$  from  $\text{LOG}-1$  down to 0:
  - If  $\text{up}[u][k] \neq \text{up}[v][k]$ :
    - $u = \text{up}[u][k]$
    - $v = \text{up}[v][k]$
5. Return  $\text{up}[u][0]$  (parent of  $u/v$ , which is the LCA)

### Euler Tour + RMQ (Sparse Table) LCA

#### Preprocessing (DFS):

1. DFS from root:
  - Record  $\text{tin}[u] = \text{time}$  (first occurrence)
  - Record  $\text{euler}[\text{time}] = u$  (node at this time)
  - Record  $\text{depthAtTime}[\text{time}] = \text{depth}[u]$
  - Increment time, recurse to children, increment time again when backtracking (optional)

#### Sparse Table:

1. Build sparse table  $\text{st}[k][i]$  for RMQ on  $\text{depthAtTime}$
2.  $\text{st}[0][i] = i$
3.  $\text{st}[k][i] = \text{index of minimum depth in range } [i, i+2^k-1]$

#### LCA Query:

1.  $l = \min(\text{tin}[u], \text{tin}[v]), r = \max(\text{tin}[u], \text{tin}[v])$
2. Find min depth index in  $[l, r]$  using sparse table
3. Return `euler[bestIndex]`

## 7. Dry Run

### Binary Lifting

Tree (root=0):



$N=7$ ,  $\text{LOG} \approx 3$  (since  $2^3=8 > 7$ )

**Parent array:** `parent = [-1, 0, 0, 1, 1, 2, 4]` **Depth:** `depth = [0, 1, 1, 2, 2, 2, 3]`

**up table (simplified for LOG=3):**

`up[6][0]=4, up[6][1]=up[4][0]=1, up[6][2]=up[1][1]=up[up[1][0]][0]=up[0][0]=0`

**LCA(6, 5):**

- `depth[6]=3, depth[5]=2. diff=1. Lift 6 by 1: up[6][0]=4 → u=4, v=5`
- Now both at depth 2. `u ≠ v.`
- `k=2 (4): up[4][2]=0, up[5][2]=0 → same, skip`
- `k=1 (2): up[4][1]=1, up[5][1]=0 → different! u=1, v=0`
- `k=0 (1): up[1][0]=0, up[0][0]=0 → same, skip`
- Return `up[1][0]=0`

`LCA(6, 5) = 0 ✓`

## 8. C++ Implementation

### Binary Lifting LCA

```

#include <bits/stdc++.h>
using namespace std;

class LCA {
private:
    int N, LOG;
    vector<vector<int>>> adj;

```

```

vector<int> depth;
vector<vector<int>> up; // up[u][k] = 2^k-th ancestor

void dfs(int u, int parent) {
    up[u][0] = parent;
    for (int k = 1; k < LOG; ++k) {
        up[u][k] = up[up[u][k-1]][k-1];
    }
    for (int v : adj[u]) {
        if (v == parent) continue;
        depth[v] = depth[u] + 1;
        dfs(v, u);
    }
}

public:
LCA(int n, vector<vector<int>>& _adj) : N(n), adj(_adj) {
    LOG = 1;
    while ((1 << LOG) <= N) ++LOG;
    depth.assign(N, 0);
    up.assign(N, vector<int>(LOG));
    dfs(0, 0); // root at 0, parent of root = itself
}

int lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);

    // Lift u to depth of v
    int diff = depth[u] - depth[v];
    for (int k = 0; k < LOG; ++k) {
        if (diff & (1 << k)) {
            u = up[u][k];
        }
    }

    if (u == v) return u;

    // Binary search for LCA
    for (int k = LOG - 1; k >= 0; --k) {
        if (up[u][k] != up[v][k]) {
            u = up[u][k];
            v = up[v][k];
        }
    }

    return up[u][0];
}

int kthAncestor(int u, int k) {
    for (int i = 0; i < LOG; ++i) {
        if (k & (1 << i)) {
            u = up[u][i];
            if (u == -1) break;
        }
    }
}

```

```

    }
    return u;
}

int distance(int u, int v) {
    int w = lca(u, v);
    return depth[u] + depth[v] - 2 * depth[w];
}

bool isAncestor(int u, int v) {
    // Is u ancestor of v?
    return lca(u, v) == u;
}
};

// ---- Example Usage ----
int main() {
    int N = 7;
    vector<vector<int>> adj(N);
    vector<pair<int,int>> edges = {{0,1},{0,2},{1,3},{1,4},{2,5},{4,6}};
    for (auto& [u,v] : edges) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    LCA lca(N, adj);
    cout << "LCA(6,5): " << lca.lca(6, 5) << endl; // 0
    cout << "LCA(3,4): " << lca.lca(3, 4) << endl; // 1
    cout << "LCA(6,4): " << lca.lca(6, 4) << endl; // 4
    cout << "Dist(6,5): " << lca.distance(6, 5) << endl; // 3+2-0 = 5? Let me
compute:
    // 6→4→1→0→2→5 = 5 edges. depth[6]=3, depth[5]=2, depth[0]=0 → 3+2-0=5 ✓
    cout << "Kth(6,2): " << lca.kthAncestor(6, 2) << endl; // 6→4→1 = 1
    cout << "Is 1 ancestor of 6? " << lca.isAncestor(1, 6) << endl; // true

    return 0;
}

```

## Euler Tour + Sparse Table LCA

```

class LCA_RMQ {
private:
    int N;
    vector<vector<int>> adj;
    vector<int> depth, euler, first, lg;
    vector<vector<int>> st; // sparse table (stores indices)

    void dfs(int u, int parent, int d) {
        depth[u] = d;
        first[u] = euler.size();
        euler.push_back(u);
    }
}

```

```

        for (int v : adj[u]) {
            if (v == parent) continue;
            dfs(v, u, d + 1);
            euler.push_back(u);
        }
    }

public:
    LCA_RMQ(int n, vector<vector<int>>& _adj) : N(n), adj(_adj) {
        depth.assign(N, 0);
        first.assign(N, 0);
        euler.reserve(2 * N);
        dfs(0, -1, 0);

        int M = euler.size();
        lg.assign(M + 1, 0);
        for (int i = 2; i <= M; ++i) lg[i] = lg[i/2] + 1;

        int K = lg[M] + 1;
        st.assign(K, vector<int>(M));
        for (int i = 0; i < M; ++i) st[0][i] = i;

        for (int k = 1; k < K; ++k) {
            for (int i = 0; i + (1 << k) <= M; ++i) {
                int left = st[k-1][i];
                int right = st[k-1][i + (1 << (k-1))];
                st[k][i] = (depth[euler[left]] < depth[euler[right]]) ? left :
right;
            }
        }
    }

    int lca(int u, int v) {
        int l = first[u], r = first[v];
        if (l > r) swap(l, r);
        int len = r - l + 1;
        int k = lg[len];
        int left = st[k][l];
        int right = st[k][r - (1 << k) + 1];
        int idx = (depth[euler[left]] < depth[euler[right]]) ? left : right;
        return euler[idx];
    }

    int distance(int u, int v) {
        return depth[u] + depth[v] - 2 * depth[lca(u, v)];
    }
};

```

## 9. Python Implementation

```

class LCA:
    def __init__(self, N, adj):
        self.N = N
        self.adj = adj
        self.LOG = (N.bit_length()) + 1
        self.depth = [0] * N
        self.up = [[0] * self.LOG for _ in range(N)]
        self._dfs(0, 0)

    def _dfs(self, u, parent):
        self.up[u][0] = parent
        for k in range(1, self.LOG):
            self.up[u][k] = self.up[self.up[u][k-1]][k-1]
        for v in self.adj[u]:
            if v == parent:
                continue
            self.depth[v] = self.depth[u] + 1
            self._dfs(v, u)

    def lca(self, u, v):
        if self.depth[u] < self.depth[v]:
            u, v = v, u

        # Lift u to depth of v
        diff = self.depth[u] - self.depth[v]
        for k in range(self.LOG):
            if diff & (1 << k):
                u = self.up[u][k]

        if u == v:
            return u

        for k in range(self.LOG - 1, -1, -1):
            if self.up[u][k] != self.up[v][k]:
                u = self.up[u][k]
                v = self.up[v][k]

        return self.up[u][0]

    def distance(self, u, v):
        w = self.lca(u, v)
        return self.depth[u] + self.depth[v] - 2 * self.depth[w]

    def kth_ancestor(self, u, k):
        for i in range(self.LOG):
            if k & (1 << i):
                u = self.up[u][i]
        return u

# Example
N = 7
adj = [[] for _ in range(N)]

```

```

edges = [(0,1),(0,2),(1,3),(1,4),(2,5),(4,6)]
for u,v in edges:
    adj[u].append(v)
    adj[v].append(u)

lca = LCA(N, adj)
print(lca.lca(6, 5)) # 0
print(lca.lca(3, 4)) # 1
print(lca.distance(6, 5)) # 5

```

## 10. Code Explanation

### Binary Lifting

#### DFS preprocessing:

- $up[u][0]$  = parent: direct parent.
- $up[u][1]$  =  $up[up[u][0]][0]$ : grandparent (2 steps).
- $up[u][k]$  =  $up[up[u][k-1]][k-1]$ :  $2^k$ -th ancestor.
- LOG is chosen such that  $2^{LOG} \geq N$ .

#### Leveling step:

- Compute  $diff = depth[u] - depth[v]$ .
- For each bit set in  $diff$ , jump  $up[u][k]$ .
- Example: if  $diff = 13$  (binary 1101), jump by 1, 4, 8.

#### Binary search for LCA:

- Start from largest jump ( $2^{(LOG-1)}$ ) down to 1.
- If  $up[u][k] \neq up[v][k]$ , both jump. This works because if the  $2^k$ -th ancestors are different, the LCA is above both.
- After the loop,  $u$  and  $v$  are children of the LCA. Return  $up[u][0]$ .

#### Why the binary search works:

- We're trying to find the highest (closest to root) where ancestors differ.
- By jumping when they differ, we get to just below the LCA.
- The parent of that node is the LCA.

### Euler Tour + RMQ

#### Euler tour:

- DFS records each node every time we enter it (and optionally on backtrack).
- $euler[i]$  = node at position  $i$  in the tour.
- $first[u]$  = first occurrence of  $u$  in euler array.
- The LCA of  $u$  and  $v$  is the node with minimum depth between  $first[u]$  and  $first[v]$ .

#### Sparse Table:

- `st[k][i]` stores the index of minimum depth in range  $[i, i+2^k-1]$ .
- Query  $[l, r]$ : combine two overlapping intervals of length  $2^k$  (largest power of 2  $\leq \text{len}$ ).
- Returns index, then we get `euler[index]`.

## 11. Complexity Analysis

| Method                 | Preprocess    | Query       | Space         |
|------------------------|---------------|-------------|---------------|
| Binary Lifting         | $O(N \log N)$ | $O(\log N)$ | $O(N \log N)$ |
| Euler Tour + Sparse T  | $O(N \log N)$ | $O(1)$      | $O(N \log N)$ |
| Euler Tour + Segment T | $O(N)$        | $O(\log N)$ | $O(N)$        |
| Naive (DFS per query)  | $O(1)$        | $O(N)$      | $O(N)$        |

| Aspect               | Complexity        |
|----------------------|-------------------|
| $N = \text{nodes}$   |                   |
| $Q = \text{queries}$ |                   |
| Binary Lifting       | $O((N+Q) \log N)$ |
| Sparse Table (RMQ)   | $O(N \log N + Q)$ |

## 12. Common Patterns

### Pattern 1: Distance Between Two Nodes

**Identify:** "Find the number of edges between  $u$  and  $v$ "

**Approach:** `dist = depth[u] + depth[v] - 2 * depth[lca]`

**Problems:** LeetCode 1740 (Distance Between Two Nodes in BST), general tree distance

### Pattern 2: Path XOR / Sum

**Identify:** "XOR of values on path from  $u$  to  $v$ "

**Approach:** `pathXor = prefixXor[u] ^ prefixXor[v] ^ val[lca]` (or twice if XOR).

For sum: `pathSum = prefixSum[u] + prefixSum[v] - 2 * prefixSum[lca] + val[lca]`

**Problems:** LeetCode 2440, Codeforces path query problems

### Pattern 3: K-th Ancestor

**Identify:** "Find the  $k$ -th ancestor of a node"

**Approach:** Binary lifting on the up table.

**Problems:** LeetCode 1483 (Kth Ancestor of a Tree Node)

### Pattern 4: Ancestor Check

**Identify:** "Is  $u$  an ancestor of  $v$ ?"

**Approach:** `tin[u] <= tin[v] && tout[v] <= tout[u]` (using entry/exit times)

**Problems:** Subtree queries, leaf-to-root checks



## Pattern 5: Path Queries (Heavy-Light Decomposition)

**Identify:** "Update and query values on path"

**Approach:** HLD + segment tree. LCA is used to decompose path into two chains.

**Problems:** CSES Path Queries, Codeforces tree query problems

## 13. Common Mistakes

- **Up table for root:** root's parent should point to itself (or -1 with careful handling)
- **LOG too small:**  $(1 \ll \text{LOG}) \leq N$  is required; use `while ((1 << LOG) <= N) LOG++`
- **Not handling same node:**  $\text{LCA}(u, u) = u$ . The binary lifting loop handles this via `if (u == v) return u`.
- **Incorrect leveling:** For diff bits, use `if (diff & (1 << k))` — the bit position matters.
- **Sparse table index out of bounds:** Check `i + (1 << k) <= M`.
- **Euler tour size:**  $2N-1$  for a tree traversal that records on entry and exit.
- **Recursion depth:** For  $N$  large ( $10^6$ ), iterative DFS may be needed.

## 14. Edge Cases

- **Root as LCA:**  $\text{LCA}(\text{root}, \text{any}) = \text{root}$
- **Same node:**  $\text{LCA}(u, u) = u$
- **u is ancestor of v:**  $\text{LCA}(u, v) = u$
- **Single node tree:**  $\text{LCA}(\text{root}, \text{root}) = \text{root}$
- **Large k for kth ancestor:** if  $k > \text{depth}[u]$ , handle gracefully (return -1 or root)

## 15. Variations

### 1. Binary Lifting (Most common)

- Simple, intuitive,  $O(\log N)$  per query
- Also supports kth ancestor directly

### 2. Euler Tour + Sparse Table (RMQ)

- $O(1)$  per query, same preprocessing
- More complex to implement but faster for many queries

### 3. Tarjan's Offline LCA (DSU-based)

- Process all queries in one DFS, uses DSU for union-find
- $O(N + Q \alpha(N))$  — very fast for offline queries
- Important for CP when  $Q$  is large

### 4. Heavy-Light Decomposition

- For complex path queries (sum, max, updates)
- LCA is computed as part of the decomposition

### 5. LCA in Cartesian Tree

- LCA of two indices in an array → RMQ problem
- Connects arrays to trees

## 16. Related Algorithms/Data Structures

| Concept                 | Connection                                                       |
|-------------------------|------------------------------------------------------------------|
| <b>Binary Lifting</b>   | DP on ancestors. Also used for kth ancestor, LCA.                |
| <b>Sparse Table</b>     | RMQ on Euler tour depth for $O(1)$ LCA.                          |
| <b>Segment Tree</b>     | RMQ variant for Euler tour LCA. $O(\log N)$ query, $O(N)$ space. |
| <b>DSU (Union-Find)</b> | Tarjan's offline LCA.                                            |
| <b>HLD</b>              | Heavy-Light Decomposition for path queries. Requires LCA.        |
| <b>DFS</b>              | Foundation. Entry/exit times, depth, parent.                     |

## 17. Practice Problems

### Easy

1. **Lowest Common Ancestor of a Binary Search Tree** — LeetCode 235  
*BST property simplifies LCA: if both values are on one side, go there.*
2. **Lowest Common Ancestor of a Binary Tree** — LeetCode 236  
*Recursive DFS approach ( $O(N)$  per query).*

### Medium

1. **Kth Ancestor of a Tree Node** — LeetCode 1483  
*Binary lifting implementation.*
2. **Distance Between Two Nodes in a Tree** — LeetCode 1740  
*LCA + depth formula.*
3. **Smallest Common Region** — LeetCode 1257  
*LCA in an N-ary tree (like file system).*

### Hard

1. **Tree Queries (path sum with updates)** — CSES / Codeforces  
*HLD + LCA for path queries.*
2. **Connecting Cities With Minimum Cost** — LeetCode 1135  
*MST, but LCA used for path max queries in MST.*

## 18. Interview Explanation

"LCA finds the deepest node that is ancestor to both query nodes. I use binary lifting: precompute  $up[u][k] = 2^k$ -th ancestor for all nodes and  $k$  up to  $\log N$ .

To answer a query, first bring both nodes to the same depth by lifting the deeper one using the binary representation of the depth difference. Then binary search upward: for  $k$  from  $\log N$  down to 0, if ancestors differ, jump both up. After the loop, their parent is the LCA.

Preprocessing is  $O(N \log N)$  and each query is  $O(\log N)$ . An alternative approach uses Euler tour + RMQ for  $O(1)$  queries with the same preprocessing.

The key applications are tree distance ( $\text{depth}[u] + \text{depth}[v] - 2 * \text{depth}[\text{lca}]$ ), path queries, and ancestry checks. Binary lifting also directly supports k-th ancestor queries."

## 19. Revision Notes

- **Binary Lifting:**  $\text{up}[u][k] = \text{up}[\text{up}[u][k-1]][k-1]$
- **LCA:** Level  $\rightarrow$  equalize depth  $\rightarrow$  binary search for LCA  $\rightarrow$  return parent
- **Distance:**  $\text{depth}[u] + \text{depth}[v] - 2 * \text{depth}[\text{lca}]$
- **K-th ancestor:** Decompose k into bits, jump  $\text{up}[u][i]$  for each set bit
- **Complexity:** Preprocess  $O(N \log N)$ , Query  $O(\log N)$
- **Edge:**  $\text{LCA}(u, u) = u$ ,  $\text{LCA}(\text{root}, \text{any}) = \text{root}$
- **RMQ variant:** Euler tour + sparse table  $\rightarrow O(1)$  query
- **Root's parent:** Point to itself (or -1 with guards)

## 20. Final Cheat Sheet

LCA – Lowest Common Ancestor (Binary Lifting)

PREPROCESS:  $\text{up}[u][k] = 2^k\text{-th ancestor}$ . DFS: depth, parent.

$\text{up}[u][0] = \text{parent}$ ;  $\text{up}[u][k] = \text{up}[\text{up}[u][k-1]][k-1]$

QUERY:

1. level u,v (lift deeper by diff bits)
2. if  $u=v$ : return u
3. for  $k = \text{LOG}-1..0$ : if  $\text{up}[u][k] \neq \text{up}[v][k]$ :  $u=\text{up}[u][k]$ ;  $v=\text{up}[v][k]$
4. return  $\text{up}[u][0]$

DISTANCE:  $\text{depth}[u] + \text{depth}[v] - 2 * \text{depth}[\text{lca}]$

TIME:  $O(N \log N)$  pre,  $O(\log N)$  query      SPACE:  $O(N \log N)$

TRAPS: LOG size ( $\geq \text{ceil}(\log_2(N))$ ), root's parent = root, 1-indexing

## 9. SEGMENT TREE

### 1. Overview

A **Segment Tree** is a binary tree data structure that stores intervals (segments) of an array. It allows range queries (sum, min, max, etc.) and point updates in  $O(\log N)$  time, and range updates with lazy propagation in  $O(\log N)$  as well.

### 2. Intuition

**Simple explanation:** You have an array of N elements. You want to answer "what's the sum of elements from index L to R?" quickly, while also being able to update individual elements. A segment tree splits the array into halves, then halves again, building a tree where each node stores the answer for its segment.

**Analogy:** Think of a company's regional sales report. The CEO looks at total sales (root covers all regions). Regional directors have subtotals for their areas (children nodes cover subranges). Each team lead knows their team's numbers (leaves = individual elements). If one team updates their number, the change propagates up the hierarchy.

**Why it works:** The array is recursively partitioned into two halves until each segment is size 1 (a single element). Each node stores the combined result (sum, min, max, etc.) of its segment. For a query, we combine  $O(\log N)$  nodes whose segments exactly cover the query range. For an update, we update the leaf and all ancestors in  $O(\log N)$  time.

### 3. When to Use It

- **Range queries with point updates** — the classic use case
- **Range updates (add, assign) with range queries** — with lazy propagation
- **Any associative operation** — sum, product, min, max, gcd, xor, bitwise AND/OR
- **Dynamic array** — queries and updates interleaved
- **Array with many queries** — Q up to  $10^5$ , N up to  $10^5$ - $10^6$
- **Range updates (add value to range)**
- **Range assignment (set all to value)**
- **2D segment tree** — for 2D range queries ( $N \leq 1000$  typically)

**Trigger phrases:** "range query", "range update", "segment tree", "range sum", "range minimum", "range maximum", "lazy propagation", "interval query"

### 4. When Not to Use It

- **Only prefix queries** — Fenwick tree is simpler and faster (same  $O(\log N)$ )
- **Only point updates and prefix queries** — Fenwick is more efficient
- **Static array (no updates)** — prefix sum array ( $O(1)$  range sum) or sparse table ( $O(1)$  range min)
- **Small N ( $\leq 1000$ )** — brute force  $O(N)$  per query is fine
- **Single query** — just iterate,  $O(N)$
- **Non-associative operations** — segment tree requires associativity:  $f(a, f(b, c)) = f(f(a, b), c)$
- **Sqrt-decomposition works better** — for certain cases like range add + point query, difference array is  $O(1)$

**Common wrong assumption:** "Segment tree always needs lazy propagation." No — if you only have point updates, you don't need lazy propagation.

## 5. Core Concepts

### 5.1 Tree Representation (Array)

- Root at index 1
- For node at index  $i$ :
  - Left child:  $2*i$
  - Right child:  $2*i + 1$
  - Parent:  $i/2$
- Array size:  $4 * N$  (safe upper bound)

## 5.2 Node

- Each node `tree[node]` represents segment `[l, r]`
- Stores the combined value for that segment

## 5.3 Build

- Recursively build: if `l == r`, store `arr[l]`; else build left and right, then combine
- $O(N)$  time

## 5.4 Point Update

- Go to leaf ( $O(\log N)$ ), update value, then recalculate all ancestors

## 5.5 Range Query

- Three cases:
  - Segment fully inside query range: return stored value
  - Segment fully outside: return identity (0 for sum, INF for min, -INF for max)
  - Partial overlap: query left + query right, combine results

## 5.6 Lazy Propagation (for Range Updates)

- `lazy[node]` stores a pending update that hasn't been pushed to children
- When visiting a node, if lazy value exists, apply it to the node and propagate to children (push)
- Allows  $O(\log N)$  range updates

# 6. Step-by-Step Algorithm

## Build

1. Create array `tree` of size  $4 * N$
2. Call `build(1, 0, N-1)`:
  - If `l == r`: `tree[node] = arr[l]`
  - Else: `mid = (l + r) / 2`
    - Build left child: `build(2*node, l, mid)`
    - Build right child: `build(2*node+1, mid+1, r)`
    - `tree[node] = combine(tree[2*node], tree[2*node+1])`

## Range Query

1. Call `query(1, 0, N-1, ql, qr)`:
  - If `ql > r || qr < l`: return identity
  - If `ql <= l && r <= qr`: return `tree[node]`
  - Else: push lazy (if any), query left + right, combine, return

## Point Update

1. Call `update(1, 0, N-1, pos, newVal)`:
  - If `l == r`: `tree[node] = newVal`

- Else: mid, go to correct child, update, then `tree[node] = combine(children)`

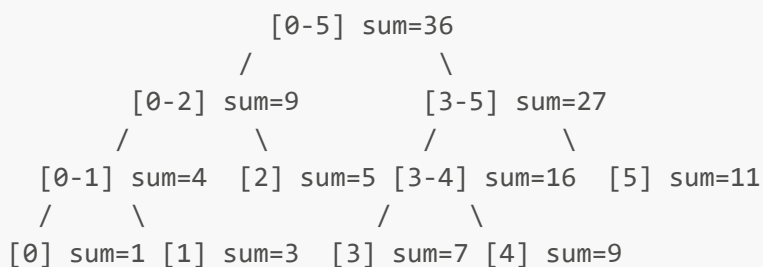
## Range Update (with Lazy)

1. Call `rangeUpdate(1, 0, N-1, ql, qr, addVal)`:
  - If fully inside: `tree[node] += addVal * (r-l+1)` and `lazy[node] += addVal`
  - Else: push lazy, go to children, combine
2. Push function: apply `lazy[node]` to children, clear lazy

## 7. Dry Run

Build: `arr = [1, 3, 5, 7, 9, 11]`

Segment tree (sum):



Query: sum of [1, 4]

- Query(1, 0, 5, 1, 4):
  - node 1 ([0,5]): not fully inside, partial overlap.
  - Go left: node 2 ([0,2]), ql=1, qr=4.
    - [0,2] partially overlaps [1,4] → go deeper.
    - node 4 ([0,1]): partially → node 8 (leaf [0]): outside → 0. node 9 (leaf [1]): inside → return 3.
    - node 5 ([2]): ql=1 ≤ 2 ≤ 4 → return 5.
  - return 3+5=8
  - Go right: node 3 ([3,5]), ql=1, qr=4.
    - node 6 ([3,4]): fully inside → return 16.
    - node 7 ([5]): outside → 0.
  - return 16+0=16
  - Combine: 8+16=24

Answer: `arr[1]+arr[2]+arr[3]+arr[4] = 3+5+7+9 = 24 ✓`

## 8. C++ Implementation

Segment Tree (Range Sum, Point Update)

```
#include <bits/stdc++.h>
using namespace std;

class SegTree {
```

```

private:
    int N;
    vector<int> tree;

    void build(int node, int l, int r, vector<int>& arr) {
        if (l == r) {
            tree[node] = arr[l];
            return;
        }
        int mid = (l + r) / 2;
        build(node * 2, l, mid, arr);
        build(node * 2 + 1, mid + 1, r, arr);
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }

    void pointUpdate(int node, int l, int r, int idx, int val) {
        if (l == r) {
            tree[node] = val;
            return;
        }
        int mid = (l + r) / 2;
        if (idx <= mid)
            pointUpdate(node * 2, l, mid, idx, val);
        else
            pointUpdate(node * 2 + 1, mid + 1, r, idx, val);
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }

    int rangeQuery(int node, int l, int r, int ql, int qr) {
        if (ql > r || qr < l) return 0; // identity for sum
        if (ql <= l && r <= qr) return tree[node];
        int mid = (l + r) / 2;
        return rangeQuery(node * 2, l, mid, ql, qr) +
            rangeQuery(node * 2 + 1, mid + 1, r, ql, qr);
    }

public:
    SegTree(int n, vector<int>& arr) : N(n) {
        tree.assign(4 * N, 0);
        build(1, 0, N - 1, arr);
    }

    void update(int idx, int val) {
        pointUpdate(1, 0, N - 1, idx, val);
    }

    int query(int l, int r) {
        return rangeQuery(1, 0, N - 1, l, r);
    }
};

```

## Segment Tree with Lazy Propagation (Range Add, Range Sum)

```

class SegTreeLazy {
private:
    int N;
    vector<long long> tree, lazy;

    void build(int node, int l, int r, vector<int>& arr) {
        if (l == r) {
            tree[node] = arr[l];
            return;
        }
        int mid = (l + r) / 2;
        build(node * 2, l, mid, arr);
        build(node * 2 + 1, mid + 1, r, arr);
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }

    void push(int node, int l, int r) {
        if (lazy[node] == 0) return;
        tree[node] += lazy[node] * (r - l + 1);
        if (l != r) { // not a leaf
            lazy[node * 2] += lazy[node];
            lazy[node * 2 + 1] += lazy[node];
        }
        lazy[node] = 0;
    }

    void rangeAdd(int node, int l, int r, int ql, int qr, int addVal) {
        push(node, l, r);
        if (ql > r || qr < l) return;
        if (ql <= l && r <= qr) {
            lazy[node] += addVal;
            push(node, l, r);
            return;
        }
        int mid = (l + r) / 2;
        rangeAdd(node * 2, l, mid, ql, qr, addVal);
        rangeAdd(node * 2 + 1, mid + 1, r, ql, qr, addVal);
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }

    long long rangeSum(int node, int l, int r, int ql, int qr) {
        push(node, l, r);
        if (ql > r || qr < l) return 0;
        if (ql <= l && r <= qr) return tree[node];
        int mid = (l + r) / 2;
        return rangeSum(node * 2, l, mid, ql, qr) +
            rangeSum(node * 2 + 1, mid + 1, r, ql, qr);
    }

public:
    SegTreeLazy(int n, vector<int>& arr) : N(n) {
        tree.assign(4 * N, 0);
    }
}

```



```

        lazy.assign(4 * N, 0);
        build(1, 0, N - 1, arr);
    }

    void add(int l, int r, int val) {
        rangeAdd(1, 0, N - 1, l, r, val);
    }

    long long sum(int l, int r) {
        return rangeSum(1, 0, N - 1, l, r);
    }
};

```

## Generic Segment Tree (Template)

```

template<typename T, T (*combine)(T, T), T identity>
class SegTreeGeneric {
private:
    int N;
    vector<T> tree;

    void build(int node, int l, int r, vector<T>& arr) {
        if (l == r) { tree[node] = arr[l]; return; }
        int mid = (l + r) / 2;
        build(node * 2, l, mid, arr);
        build(node * 2 + 1, mid + 1, r, arr);
        tree[node] = combine(tree[node * 2], tree[node * 2 + 1]);
    }

    void pointUpdate(int node, int l, int r, int idx, T val) {
        if (l == r) { tree[node] = val; return; }
        int mid = (l + r) / 2;
        if (idx <= mid) pointUpdate(node * 2, l, mid, idx, val);
        else pointUpdate(node * 2 + 1, mid + 1, r, idx, val);
        tree[node] = combine(tree[node * 2], tree[node * 2 + 1]);
    }

    T rangeQuery(int node, int l, int r, int ql, int qr) {
        if (ql > r || qr < l) return identity;
        if (ql <= l && r <= qr) return tree[node];
        int mid = (l + r) / 2;
        return combine(rangeQuery(node * 2, l, mid, ql, qr),
                       rangeQuery(node * 2 + 1, mid + 1, r, ql, qr));
    }

public:
    SegTreeGeneric(int n, vector<T>& arr) : N(n) {
        tree.assign(4 * N, identity);
        build(1, 0, N - 1, arr);
    }

    void update(int idx, T val) { pointUpdate(1, 0, N - 1, idx, val); }

```

```

T query(int l, int r) { return rangeQuery(1, 0, N - 1, l, r); }
};

// Combine functions
int sum(int a, int b) { return a + b; }
int minn(int a, int b) { return min(a, b); }
int maxx(int a, int b) { return max(a, b); }

// Usage:
// SegTreeGeneric<int, sum, 0> sumTree(N, arr);
// SegTreeGeneric<int, minn, INT_MAX> minTree(N, arr);

```

## Example Usage

```

int main() {
    vector<int> arr = {1, 3, 5, 7, 9, 11};

    SegTree seg(6, arr);
    cout << "Sum [1,4]: " << seg.query(1, 4) << endl; // 24
    seg.update(2, 10); // arr[2] = 10
    cout << "Sum [1,4] after update: " << seg.query(1, 4) << endl; // 3+10+7+9 =
29

    // Lazy segtree
    SegTreeLazy lazySeg(6, arr);
    lazySeg.add(2, 4, 5); // add 5 to indices 2,3,4
    cout << "Sum [1,5] after range add: " << lazySeg.sum(1, 5) << endl; // 3+
(5+5)+(7+5)+(9+5)+11 = 50

    return 0;
}

```

## 9. Python Implementation

```

class SegTree:
    def __init__(self, arr):
        self.N = len(arr)
        self.tree = [0] * (4 * self.N)
        self._build(1, 0, self.N - 1, arr)

    def _build(self, node, l, r, arr):
        if l == r:
            self.tree[node] = arr[l]
            return
        mid = (l + r) // 2
        self._build(node * 2, l, mid, arr)
        self._build(node * 2 + 1, mid + 1, r, arr)
        self.tree[node] = self.tree[node * 2] + self.tree[node * 2 + 1]

```

```

def update(self, idx, val):
    self._point_update(1, 0, self.N - 1, idx, val)

def _point_update(self, node, l, r, idx, val):
    if l == r:
        self.tree[node] = val
        return
    mid = (l + r) // 2
    if idx <= mid:
        self._point_update(node * 2, l, mid, idx, val)
    else:
        self._point_update(node * 2 + 1, mid + 1, r, idx, val)
    self.tree[node] = self.tree[node * 2] + self.tree[node * 2 + 1]

def query(self, ql, qr):
    return self._range_query(1, 0, self.N - 1, ql, qr)

def _range_query(self, node, l, r, ql, qr):
    if ql > r or qr < l:
        return 0
    if ql <= l and r <= qr:
        return self.tree[node]
    mid = (l + r) // 2
    return (self._range_query(node * 2, l, mid, ql, qr) +
            self._range_query(node * 2 + 1, mid + 1, r, ql, qr))

class SegTreeLazy:
    def __init__(self, arr):
        self.N = len(arr)
        self.tree = [0] * (4 * self.N)
        self.lazy = [0] * (4 * self.N)
        self._build(1, 0, self.N - 1, arr)

    def _build(self, node, l, r, arr):
        if l == r:
            self.tree[node] = arr[l]
            return
        mid = (l + r) // 2
        self._build(node * 2, l, mid, arr)
        self._build(node * 2 + 1, mid + 1, r, arr)
        self.tree[node] = self.tree[node * 2] + self.tree[node * 2 + 1]

    def _push(self, node, l, r):
        if self.lazy[node] == 0:
            return
        self.tree[node] += self.lazy[node] * (r - l + 1)
        if l != r:
            self.lazy[node * 2] += self.lazy[node]
            self.lazy[node * 2 + 1] += self.lazy[node]
        self.lazy[node] = 0

    def add(self, ql, qr, val):
        self._range_add(1, 0, self.N - 1, ql, qr, val)

```

```

def _range_add(self, node, l, r, ql, qr, val):
    self._push(node, l, r)
    if ql > r or qr < l:
        return
    if ql <= l and r <= qr:
        self.lazy[node] += val
        self._push(node, l, r)
        return
    mid = (l + r) // 2
    self._range_add(node * 2, l, mid, ql, qr, val)
    self._range_add(node * 2 + 1, mid + 1, r, ql, qr, val)
    self.tree[node] = self.tree[node * 2] + self.tree[node * 2 + 1]

def sum(self, ql, qr):
    return self._range_sum(1, 0, self.N - 1, ql, qr)

def _range_sum(self, node, l, r, ql, qr):
    self._push(node, l, r)
    if ql > r or qr < l:
        return 0
    if ql <= l and r <= qr:
        return self.tree[node]
    mid = (l + r) // 2
    return (self._range_sum(node * 2, l, mid, ql, qr) +
            self._range_sum(node * 2 + 1, mid + 1, r, ql, qr))

# Example
arr = [1, 3, 5, 7, 9, 11]
seg = SegTree(arr)
print(seg.query(1, 4)) # 24
seg.update(2, 10)
print(seg.query(1, 4)) # 29

lazy_seg = SegTreeLazy(arr)
lazy_seg.add(2, 4, 5)
print(lazy_seg.sum(1, 5)) # 50

```

## 10. Code Explanation

### Basic Array Representation

- Tree stored in array: root at index 1. Left child =  $2i$ , right child =  $2i+1$ .
- Size needs to be  $4 * N$  to guarantee enough space ( $2^{\lceil \log_2(N)+1 \rceil} - 1 < 4N$ ).

### Build

- Recursively divide range until single element.
- Leaf stores `arr[l]`.
- Internal node combines children: `tree[node] = combine(tree[2*node], tree[2*node+1])`.
- The combine function depends on the operation: `+` for sum, `min()` for RMQ, etc.

## Query

- **Three cases:**
  1. No overlap ( $q_l > r$  ||  $q_r < l$ ): return identity (0 for sum, INF for min, -INF for max).
  2. Full overlap ( $q_l \leq l$  &&  $r \leq q_r$ ): return stored value.
  3. Partial overlap: recursively query both children, combine results.
- No push needed for point-update segment tree (lazy is separate).

## Point Update

- Go down to the leaf that matches `idx`.
- Update leaf value.
- On the way back up, `tree[node] = combine(children)`.

## Lazy Propagation

- **Range Add:** Instead of updating every leaf in the range, we mark the node as "dirty" (lazy).
- **Push:** Before accessing a node (in query or further updates), propagate its lazy value to children.
- **Apply to node:** When setting lazy, also update `tree[node]` immediately so that ancestors see the correct value.
- **Key:** `tree[node] += lazy[node] * (r-l+1)` for sum updates. For min/max, it's just `tree[node] += lazy[node]`.

## 11. Complexity Analysis

| Operation           | Time        | Space       |
|---------------------|-------------|-------------|
| Build               | $O(N)$      | $O(N)$      |
| Point Update        | $O(\log N)$ | $O(\log N)$ |
| Range Query         | $O(\log N)$ | $O(\log N)$ |
| Range Update (lazy) | $O(\log N)$ | $O(\log N)$ |

| Aspect          | Complexity  |
|-----------------|-------------|
| N = array size  |             |
| Tree node count | $\sim 4N$   |
| Build time      | $O(N)$      |
| Query/update    | $O(\log N)$ |
| Lazy array      | $O(N)$      |

## 12. Common Patterns

### Pattern 1: Range Sum with Point Updates

**Identify:** "Sum of subarray  $[l, r]$ " with updates to individual elements.

**Approach:** Standard segment tree with sum combiner.

**Problems:** LeetCode 307 (Range Sum Query — Mutable)

## Pattern 2: Range Minimum with Point Updates

**Identify:** "Find minimum in range" with updates.

**Approach:** Segment tree with `min` combiner. Identity = `INT_MAX`.

**Problems:** LeetCode (various RMQ problems)

## Pattern 3: Range Add and Range Sum (Lazy)

**Identify:** "Add value to range  $[l, r]$ ", "Get sum of range  $[l, r]$ "

**Approach:** Lazy segment tree with range add.

**Problems:** CSES Range Update Queries, LeetCode 370 (Range Addition — but diff array is simpler)

## Pattern 4: Range Assignment (Set to value)

**Identify:** "Set all elements in  $[l, r]$  to some value"

**Approach:** Lazy with assignment (need a different lazy flag). `lazy[node]` stores the assigned value or a sentinel (like `INF` for "no assignment").

**Problems:** CSES Range Assignment, LeetCode 699 (Falling Squares)

## Pattern 5: Segment Tree with Multiple Operations

**Identify:** "Get max and sum in range", or "count elements  $> X$  in range"

**Approach:** Store a struct in each node. For count  $> X$ , use merge sort tree (segment tree of sorted vectors).

**Problems:** Various Codeforces problems

## Pattern 6: Segment Tree on Tree (HLD)

**Identify:** "Query/update path in a tree"

**Approach:** Heavy-Light Decomposition + segment tree on chains.

**Problems:** CSES Path Queries

# 13. Common Mistakes

- **Array size too small** —  $4 * N$  is safe,  $2 * N$  is not always enough
- **Not using identity element correctly** — 0 for sum, `INT_MAX` for min, `INT_MIN` for max, 0 for xor, 1 for product
- **Forgetting to push in lazy segtree** — every query/update must push if lazy exists
- **Applying lazy to wrong field** — for min/max, `tree[node] += lazy[node]` is still correct for range add
- **Memory for long long** — sum queries can overflow 32-bit int; use `long long`
- **Indexing confusion** — 0-indexed vs 1-indexed arrays. Tree storage is 1-indexed. Array indices are 0-indexed.
- **Recursion depth in query/update** — tree depth is  $O(\log N)$ , fine for  $N$  up to  $10^6$
- **Not handling empty range** —  $ql > qr$  should return identity

# 14. Edge Cases

- **$N = 1$**  — tree has only root. Single element queries.

- **All zeros** — sum query returns 0.
- **Large values / overflow** — use `long long` for sums.
- **Query range = full array** — returns root value.
- **Query range = single element** — goes to leaf.
- **Update same element multiple times** — works fine.
- **Range update on single element** — lazy works same as point update.
- **Negative values** — sum works; min works; max works.

## 15. Variations

### 1. Fenwick Tree (Binary Indexed Tree)

- Simpler, faster, less memory for prefix queries
- Cannot handle range min/max (can only do prefix sum and related operations)
- See next section for details

### 2. Iterative Segment Tree (bottom-up)

- Uses array of size  $2N$  instead of  $4N$
- Leaves at indices  $N..2N-1$ , root at 1
- Faster, simpler for point updates and range queries
- Harder to extend with lazy propagation

### 3. Merge Sort Tree (Segment Tree of sorted arrays)

- Each node stores a sorted vector of its segment
- Queries: "How many elements in  $[l, r]$  are  $\leq X$ ?"
- $O(\log^2 N)$  per query,  $O(N \log N)$  memory

### 4. Sparse Table

- For static arrays:  $O(1)$  range queries (min, max, gcd, etc.)
- $O(N \log N)$  build, no updates
- Not a segment tree but often compared

### 5. 2D Segment Tree

- Nested segment trees: outer tree on one dimension, each node has inner tree on the other
- $O(\log^2 N)$  queries,  $O(N^2)$  memory — only for small  $N$  ( $\leq 1000$ )

## 16. Related Algorithms/Data Structures

| DS                  | Segment Tree vs ...                                                                                               |
|---------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Fenwick Tree</b> | Fenwick: simpler, faster, less memory, but prefix-only and no range min/max. Segment tree: general range queries. |
| <b>Prefix Sum</b>   | $O(1)$ range sum for static arrays. No updates.                                                                   |

| DS                       | Segment Tree vs ...                                                       |
|--------------------------|---------------------------------------------------------------------------|
| <b>Difference Array</b>  | $O(1)$ range add, $O(N)$ point query. No range queries.                   |
| <b>Sparse Table</b>      | $O(1)$ range min for static arrays. No updates.                           |
| <b>Sqrt Decomp</b>       | $O(\sqrt{N})$ queries and updates. Simpler, useful for $N \approx 10^5$ . |
| <b>Ordered Set (BST)</b> | For dynamic order statistics. Not range queries.                          |

## 17. Practice Problems

### Easy

1. **Range Sum Query — Mutable** — LeetCode 307  
*Basic segment tree with point updates.*
2. **Range Addition** — LeetCode 370  
*Range add (difference array is simpler, but good for lazy practice).*

### Medium

1. **My Calendar I** — LeetCode 729  
*Segment tree approach to find overlapping intervals.*
2. **Falling Squares** — LeetCode 699  
*Range update (max) + query. Lazy propagation.*
3. **Range Sum Query 2D — Mutable** — LeetCode 308  
*2D BIT or 2D segment tree.*

### Hard

1. **Count of Smaller Numbers After Self** — LeetCode 315  
*Segment tree over value range (coordinate compression + inversion count).*
2. **Range Queries with Updates on Tree** — CSES / Codeforces  
*HLD + segment tree for tree path queries.*

## 18. Interview Explanation

"A segment tree is a binary tree where each node represents a segment of the array and stores the combined result (sum, min, max, etc.) for that segment. It supports range queries and point updates in  $O(\log N)$  time.

The tree is stored in an array of size  $4N$ . Building takes  $O(N)$ . For queries, I check three cases: no overlap (return identity), full overlap (return stored), or partial (recursively query children and combine).

For range updates, I use lazy propagation: instead of updating every leaf, I mark a node as pending and only push the update when I need to visit its children. This keeps range updates  $O(\log N)$ .

The trade-off is memory:  $4N$  integers. But it's extremely versatile — I can query sum, min, max, gcd, XOR — anything associative. For simpler prefix queries, I'd use a Fenwick tree instead."

## 19. Revision Notes



- **Array representation:**  $root=1$ ,  $left=2node$ ,  $right=2node+1$ ,  $size=4N$
- **Build:**  $O(N)$ . Recursive:  $leaf = arr[l]$ ,  $internal = combine(children)$
- **Query:**  $O(\log N)$ . 3 cases: outside (identity), inside (stored), partial (recurse)
- **Point update:**  $O(\log N)$ . Go to leaf, update, recompute ancestors
- **Lazy:** `lazy[node]` for pending range updates. Push before accessing children
- **Combine:** sum (+), min (min), max (max), gcd, xor, etc.
- **Identity:** sum=0, min=INF, max=-INF, xor=0
- **Key code:** `tree[node] = tree[2*node] + tree[2*node+1]`

## 20. Final Cheat Sheet

SEGMENT TREE – Range queries + updates in  $O(\log N)$

USE: range sum/min/max/gcd/xor with point or range updates

SIZE: `tree[4*N]`, `lazy[4*N]`

BUILD:  $O(N)$  –  $leaf=arr[l]$ ,  $node=left+right$

QUERY: 3 cases: outside→identity, inside→stored, partial→recurse

POINT UPDATE: leaf update → recompute ancestors

LAZY: push before accessing children; `tree[node] += lazy * (r-l+1)`

TIME:  $O(\log N)$  per query/update      SPACE:  $O(N)$

TEMPLATE:

```
int combine(int a, int b) { return a + b; }
```

```
const int IDENTITY = 0;
```

TRAPS:  $4N$  size, long long for sums, push before query, identity values

## 10. FENWICK TREE (BINARY INDEXED TREE / BIT)

### 1. Overview

A **Fenwick Tree** (Binary Indexed Tree / BIT) is a data structure that maintains prefix sums and supports point updates in  $O(\log N)$  time. It uses a clever binary indexing trick to store cumulative sums in an array of size  $N$ , requiring only  $O(N)$  memory.

### 2. Intuition

**Simple explanation:** You want to support two operations: add a value to an element, and get the sum of the first  $k$  elements. A Fenwick tree does this in  $O(\log N)$  using bit manipulation.

**Analogy:** Think of a pyramid of buckets. Bucket 1 holds `arr[1]`. Bucket 2 holds `arr[1]+arr[2]`. Bucket 3 holds `arr[3]`. Bucket 4 holds `arr[1]+arr[2]+arr[3]+arr[4]`. Each bucket covers a range whose size is a power of two. When you add to an element, you update all buckets that cover it. When you query prefix sum, you add up the right buckets.

**Why it works:** The key insight is that any prefix can be expressed as a sum of  $O(\log N)$  disjoint ranges whose sizes are powers of two. The BIT stores the sum for each of these ranges. The index  $i$  stores the sum of the

range  $(i - \text{LSB}(i), i]$  where  $\text{LSB}$  is the least significant set bit.

### 3. When to Use It

- **Prefix sum queries** with point updates
- **Range sum queries** (can be derived from two prefix sums)
- **Inversion count** (classic application)
- **Frequency array queries** — "how many elements  $\leq X$ ?"
- **Order statistics** — "find k-th smallest" (with binary lifting on BIT)
- **Point updates, prefix queries** — faster and simpler than segment tree
- **2D BIT** — for 2D prefix sums (e.g., grid queries)

**Trigger phrases:** "prefix sum", "inversion count", "frequency", "cumulative sum", "Fenwick", "BIT", "binary indexed tree"

### 4. When Not to Use It

- **Range minimum/maximum queries** — BIT cannot handle non-invertible operations (min/max can't be subtracted)
- **Range updates and range queries** — possible with two BITs but complex; segment tree with lazy is simpler
- **Range assignment (set to value)** — segment tree is better
- **Static array** — prefix sum array is  $O(1)$  and simpler
- **Small N** — just use a regular array

**Common wrong assumption:** "BIT can do everything a segment tree can do." False — BIT only handles prefix queries with **invertible** operations (sum, xor, product). For min/max, use segment tree.

## 5. Core Concepts

### 5.1 Least Significant Bit (LSB)

- $i \& -i$  gives the lowest set bit
- Example:  $i=12$  (1100),  $-i = (-12) = \dots 1110100$ ,  $i \& -i = 100$  (4)
- This determines the range covered by  $\text{bit}[i]$

### 5.2 BIT Array

- $\text{bit}[i]$  stores sum of range  $(i - \text{LSB}(i), i]$
- Length of range =  $\text{LSB}(i)$
- Example:  $\text{bit}[12]$  (binary 1100,  $\text{LSB}=100=4$ ) covers indices 9-12
- Example:  $\text{bit}[8]$  (binary 1000,  $\text{LSB}=1000=8$ ) covers indices 1-8

### 5.3 Point Update

- Add value at index  $i$
- $i += (i \& -i)$  — propagate to ancestors
- Updates all BIT entries that cover index  $i$

### 5.4 Prefix Query

- Sum of indices  $[1, i]$
- $i -= (i \& -i)$  — remove LSB, accumulate
- Combines disjoint ranges

## 5.5 0-based vs 1-based

- BIT is naturally 1-based (index 1..N)
- For 0-based arrays, add 1 to all indices

## 6. Step-by-Step Algorithm

Point Update (add value at position  $i$ )

1. While  $i \leq N$ : a.  $bit[i] += delta$  b.  $i += i \& -i$  (go to parent)

Prefix Sum (sum from 1 to  $i$ )

1.  $sum = 0$
2. While  $i > 0$ : a.  $sum += bit[i]$  b.  $i -= i \& -i$  (go to sibling)
3. Return  $sum$

Range Sum (sum from  $l$  to  $r$ )

- $prefix(r) - prefix(l-1)$

Find  $k$ -th smallest (binary lifting on BIT)

1.  $pos = 0$ ,  $bitMask = \text{highest power of two} \leq N$
2. While  $bitMask > 0$ : a.  $next = pos + bitMask$  b. If  $next \leq N$  and  $bit[next] < k$ :
  - $k -= bit[next]$
  - $pos = next$  c.  $bitMask \gg= 1$
3. Return  $pos + 1$

## 7. Dry Run

BIT After Insertions:  $arr = [0, 3, 2, -1, 6, 5]$  (1-indexed)

Let's build BIT by adding each element:

**Initial:**  $bit = [0, 0, 0, 0, 0, 0]$

**Add 3 at  $i=1$ :**

- $i=1$ :  $bit[1] += 3$  (3).  $i += 1 \rightarrow 2$
- $i=2$ :  $bit[2] += 3$  (3).  $i += 2 \rightarrow 4$
- $i=4$ :  $bit[4] += 3$  (3).  $i += 4 \rightarrow 8 > 6$ , stop  $bit = [0, 3, 3, 0, 3, 0]$

**Add 2 at  $i=2$ :**

- $i=2$ :  $bit[2] += 2 \rightarrow 5$ .  $i += 2 \rightarrow 4$
- $i=4$ :  $bit[4] += 2 \rightarrow 5$ .  $i += 4 \rightarrow 8 > 6$ , stop  $bit = [0, 3, 5, 0, 5, 0]$

**Add -1 at i=3:**

- i=3: bit[3] += -1 → -1. i += LSB(3)=1 → 4
- i=4: bit[4] += -1 → 4. i += 4 → 8 > 6, stop bit = [0, 3, 5, -1, 4, 0]

**Add 6 at i=4:**

- i=4: bit[4] += 6 → 10. i += 4 → 8 > 6, stop bit = [0, 3, 5, -1, 10, 0]

**Add 5 at i=5:**

- i=5: bit[5] += 5 → 5. i += LSB(5)=1 → 6
- i=6: bit[6] += 5 → 5. i += LSB(6)=2 → 8 > 6, stop bit = [0, 3, 5, -1, 10, 5]

## Query prefix sum upto 5:

- i=5: sum += bit[5]=5. i -= 1 → 4
- i=4: sum += bit[4]=10. i -= 4 → 0
- sum = 15. Actual: 3+2+(-1)+6+5 = 15 ✓

## Query range sum [2, 4]:

- prefix(4) - prefix(1)
- prefix(4): i=4: sum += 10. i=0 → 10
- prefix(1): i=1: sum += 3. i=0 → 3
- Result = 10 - 3 = 7. Actual: 2+(-1)+6 = 7 ✓

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class Fenwick {
private:
    int N;
    vector<long long> bit;

public:
    Fenwick(int n) : N(n) {
        bit.assign(N + 1, 0); // 1-indexed
    }

    // Build from array (O(N log N) – or use O(N) construction)
    Fenwick(vector<int>& arr) : N(arr.size()) {
        bit.assign(N + 1, 0);
        for (int i = 0; i < N; ++i) {
            add(i + 1, arr[i]); // convert to 1-indexed
        }
    }

    // Add delta at position i (1-indexed)
    void add(int i, long long delta) {
```

```

        while (i <= N) {
            bit[i] += delta;
            i += i & -i;
        }
    }

    // Sum of [1, i]
    long long prefixSum(int i) {
        long long sum = 0;
        while (i > 0) {
            sum += bit[i];
            i -= i & -i;
        }
        return sum;
    }

    // Sum of [l, r]
    long long rangeSum(int l, int r) {
        return prefixSum(r) - prefixSum(l - 1);
    }

    // Find smallest index such that prefixSum(index) >= target
    // Requires all values non-negative
    int lowerBound(long long target) {
        int pos = 0;
        int bitMask = 1 << (31 - __builtin_clz(N)); // highest power of 2 ≤ N
        while (bitMask) {
            int next = pos + bitMask;
            if (next <= N && bit[next] < target) {
                target -= bit[next];
                pos = next;
            }
            bitMask >>= 1;
        }
        return pos + 1;
    }

    // Point query: get value at position i
    long long pointQuery(int i) {
        return rangeSum(i, i);
    }
};

// ---- Example Usage ----
int main() {
    vector<int> arr = {3, 2, -1, 6, 5};
    Fenwick bit(arr);

    cout << "Prefix sum [1,5]: " << bit.prefixSum(5) << endl; // 15
    cout << "Range sum [2,4]: " << bit.rangeSum(2, 4) << endl; // 7

    bit.add(3, 10); // add 10 to index 3
    cout << "Prefix sum [1,5] after add: " << bit.prefixSum(5) << endl; // 25
    cout << "Range sum [2,4]: " << bit.rangeSum(2, 4) << endl; // 17
}

```

```

// Inversion count example
vector<int> nums = {5, 3, 2, 4, 1};
Fenwick bit2(5);
long long inv = 0;
for (int i = 0; i < 5; ++i) {
    int val = nums[i];
    inv += i - bit2.prefixSum(val); // how many greater seen before
    bit2.add(val, 1);
}
cout << "Inversion count: " << inv << endl; // 7

return 0;
}

```

## 9. Python Implementation

```

class Fenwick:
    def __init__(self, n_or_arr):
        if isinstance(n_or_arr, int):
            self.N = n_or_arr
            self.bit = [0] * (self.N + 1)
        else:
            self.N = len(n_or_arr)
            self.bit = [0] * (self.N + 1)
            for i, val in enumerate(n_or_arr, 1):
                self.add(i, val)

    def add(self, i, delta):
        while i <= self.N:
            self.bit[i] += delta
            i += i & -i

    def prefix_sum(self, i):
        s = 0
        while i > 0:
            s += self.bit[i]
            i -= i & -i
        return s

    def range_sum(self, l, r):
        return self.prefix_sum(r) - self.prefix_sum(l - 1)

    def lower_bound(self, target):
        """Find smallest i with prefix_sum(i) >= target. Values must be non-negative."""
        pos = 0
        bit_mask = 1 << (self.N.bit_length() - 1)
        while bit_mask:
            nxt = pos + bit_mask
            if nxt <= self.N and self.bit[nxt] < target:

```

```

        target -= self.bit[nxt]
        pos = nxt
        bit_mask >>= 1
    return pos + 1

def point_query(self, i):
    return self.range_sum(i, i)

# Inversion count
def inversion_count(arr):
    # Coordinate compress
    vals = sorted(set(arr))
    comp = {v: i+1 for i, v in enumerate(vals)} # 1-indexed
    bit = Fenwick(len(vals))
    inv = 0
    for i, val in enumerate(arr):
        idx = comp[val]
        inv += i - bit.prefix_sum(idx)
        bit.add(idx, 1)
    return inv

# Example
arr = [3, 2, -1, 6, 5]
bit = Fenwick(arr)
print(bit.prefix_sum(5)) # 15
print(bit.range_sum(2, 4)) # 7
bit.add(3, 10)
print(bit.prefix_sum(5)) # 25

print(inversion_count([5, 3, 2, 4, 1])) # 7

```

## 10. Code Explanation

### add(i, delta)

- `while (i <= N)`: updates all nodes that cover index `i`.
- `i += i & -i`: moves to the next node that covers a larger range.
- Example: `i=3` (binary 11). LSB=1. `i=4` (100). LSB=4. `i=8 > N`.
- Nodes covering index 3 are: 3, 4, 8 (if  $N \geq 8$ ).

### prefixSum(i)

- `while (i > 0)`: accumulates disjoint ranges.
- `i -= i & -i`: removes the LSB, moving to the next non-overlapping range.
- Example: `i=13` (1101). LSB=1. Sum `bit[13]`. `i=12` (1100). LSB=4. Sum `bit[12]`. `i=8` (1000). LSB=8. Sum `bit[8]`. `i=0`. Stop.
- Ranges covered: (12,13], (8,12], (0,8]. Total: [1,13].

### rangeSum(l, r)

- `prefix(r) - prefix(l-1)`: standard prefix difference for invertible operations.

## lowerBound(target) (find kth)

- Requires all values non-negative.
- Uses binary lifting: starts with largest power of  $2 \leq N$ .
- At each step, if `bit[next] < target`, skip that range and reduce target.
- Returns `pos + 1`.

## Coordinate Compression for Inversion Count

- Values may be large or negative. Map them to  $1..M$  (sorted unique values).
- `inv += i - prefix_sum(compressed[val])`: count of elements seen so far that are  $> \text{val}$ .
- `add(compressed[val], 1)`: mark this value as seen.

## 11. Complexity Analysis

| Operation               | Time          | Space  |
|-------------------------|---------------|--------|
| Build (per element add) | $O(N \log N)$ | $O(N)$ |
| Build ( $O(N)$ )        | $O(N)$        | $O(N)$ |
| Point Update (add)      | $O(\log N)$   | $O(1)$ |
| Prefix Sum              | $O(\log N)$   | $O(1)$ |
| Range Sum               | $O(\log N)$   | $O(1)$ |
| Lower Bound             | $O(\log N)$   | $O(1)$ |

| Aspect         | Complexity  |
|----------------|-------------|
| N = array size |             |
| BIT array size | $N + 1$     |
| Time per op    | $O(\log N)$ |
| Memory         | $O(N)$      |

## 12. Common Patterns

### Pattern 1: Inversion Count

**Identify:** "Count pairs  $(i, j)$  with  $i < j$  and  $\text{arr}[i] > \text{arr}[j]$ "

**Approach:** Iterate array, for each element count how many seen so far are greater. BIT on value range (with coordinate compression).

**Problems:** LeetCode 315 (Count of Smaller Numbers After Self), SPOJ INVCNT

### Pattern 2: Range Sum with Point Updates

**Identify:** "Sum of subarray with updates to individual elements"

**Approach:** BIT for prefix sum; range sum via two prefix queries.

**Problems:** LeetCode 307 (Range Sum Query — Mutable) — can use BIT or segment tree



### Pattern 3: Order Statistics / K-th Smallest

**Identify:** "Find k-th smallest element among those inserted so far"

**Approach:** BIT as frequency array. Add 1 on insert. `LowerBound(k)` finds k-th.

**Problems:** LeetCode 703 (Kth Largest Element in a Stream) — but with dynamic updates

### Pattern 4: Count of Numbers $\leq X$ in a Range

**Identify:** "For each query (l, r, x), count how many elements  $\leq x$  in [l, r]"

**Approach:** Offline: sort queries by x, process elements in increasing order, add 1 at their position. BIT stores counts at indices.

**Problems:** LeetCode 327 (Count of Range Sum) — BIT on prefix sums

### Pattern 5: Difference Array with BIT (Range Add, Point Query)

**Identify:** "Add value to range, then query point values"

**Approach:** `bit.add(l, delta)`, `bit.add(r+1, -delta)`. Then `bit.prefixSum(i)` gives value at i.

**Problems:** LeetCode 370 (Range Addition), CSES Range Update Queries

### Pattern 6: 2D BIT

**Identify:** "2D grid with point updates and prefix sum queries"

**Approach:** Nested BIT: `bit[x][y]`. Update: `for i = x; i ≤ N; i += LSB(i)` then `for j = y; j ≤ M; j += LSB(j): bit[i][j] += delta`.

**Problems:** LeetCode 308 (Range Sum Query 2D — Mutable)

## 13. Common Mistakes

- **0-index vs 1-index confusion** — BIT is naturally 1-indexed. Always convert by adding 1.
- **Overflow in `i` & `-i`** — works for signed int; use `unsigned` if needed.
- **Forgetting N is max index** — update/walk beyond N goes out of bounds.
- **Range sum with `r < 1`** — return 0 or handle gracefully.
- **Negative values in `lowerBound`** — requires non-negative values.
- **Not coordinate-compressing for large values** — BIT needs index  $\leq N$ .
- **Using BIT for range min/max** — doesn't work (not invertible).
- **Building BIT in  $O(N \log N)$  vs  $O(N)$**  —  $O(N)$  build: `for i = 1..N: bit[i] = arr[i]; for j = i + LSB(i): bit[j] += bit[i]`.

## 14. Edge Cases

- **N = 0** — empty BIT, no operations.
- **N = 1** — single element, prefix works fine.
- **Large values (values up to  $10^9$ )** — coordinate compress.
- **Negative values** — BIT stores them fine for sum queries.
- **All values zero** — prefix sum = 0, all queries return 0.
- **Update on index > N** — out of bounds, handle with check.
- **Same value multiple times** — inversion count with compression handles this (need stable ordering: add 1 at position for each occurrence).

## 15. Variations

### 1. Range Update + Range Query (using two BITs)

- Maintain two BITs: **B1** and **B2**
- Range add  $[l, r] += x$ : **B1.add(l, x), B1.add(r+1, -x), B2.add(l, x\*(l-1)), B2.add(r+1, -x\*r)**
- Range sum  $[l, r] = \text{prefix}(r) - \text{prefix}(l-1)$
- Where  $\text{prefix}(i) = \text{B1.prefixSum}(i) * i - \text{B2.prefixSum}(i)$
- Used in CP for range add + range sum without lazy segment tree

### 2. 2D BIT

- Nested loops for update and query
- $O(\log^2 N)$  per operation
- Memory  $O(N^2)$  — only feasible for  $N \leq 10^4$

### 3. BIT for Non-invertible Operations? (No)

- BIT only works for invertible operations (addition, XOR, multiplication)
- For min/max, use segment tree

### 4. Persistent BIT

- Save version snapshots
- Used in some advanced CP problems

## 16. Related Algorithms/Data Structures

| DS/Algorithm            | Connection                                                                                        |
|-------------------------|---------------------------------------------------------------------------------------------------|
| <b>Segment Tree</b>     | More general (range queries on any associative op, lazy updates). BIT is simpler for prefix sums. |
| <b>Prefix Sum Array</b> | $O(1)$ queries but no updates. BIT is dynamic version.                                            |
| <b>Difference Array</b> | $O(1)$ range add, $O(N)$ point query. BIT makes it $O(\log N)$ for both.                          |
| <b>Merge Sort</b>       | Inversion count can also be done with merge sort ( $O(N \log N)$ ).                               |
| <b>Ordered Set</b>      | For order statistics. BIT is simpler and faster for integer keys.                                 |

## 17. Practice Problems

### Easy

1. **Range Sum Query — Mutable** — LeetCode 307  
*BIT for prefix sums with point updates.*
2. **Range Addition** — LeetCode 370  
*Difference array with BIT for range add + point query.*

### Medium

### 1. **Count of Smaller Numbers After Self** — LeetCode 315

*Inversion count with BIT + coordinate compression.*

### 2. **Reverse Pairs** — LeetCode 493

*Count pairs  $i < j$  with  $arr[i] > 2arr[j]$ . BIT with coordinate compression.\**

### 3. **Range Sum Query 2D — Mutable** — LeetCode 308

*2D BIT implementation.*

## Hard

### 1. **Count of Range Sum** — LeetCode 327

*BIT on prefix sums. Number of subarrays with sum in [lower, upper].*

### 2. **Maximum Sum of Subarray with Range Constraints** — Codeforces / AtCoder

*Advanced BIT with range updates.*

## 18. Interview Explanation

"A Fenwick tree, or BIT, maintains prefix sums and supports point updates in  $O(\log N)$  time. It uses an array where `bit[i]` stores the sum of a range ending at  $i$  with length equal to the lowest set bit of  $i$ . For update, I add the delta to `bit[i]`, then add the LSB to  $i$  to move to the next covering node. For prefix sum, I sum `bit[i]`, then subtract LSB to move to the next non-overlapping range. It's simpler and more memory-efficient than a segment tree — just  $N+1$  integers. But it only works for invertible operations like sum, XOR, or product. For min/max, I'd use a segment tree. Classic applications include inversion count (using BIT on compressed values), order statistics (using `lower_bound` with binary lifting), and range add + point query (difference array + BIT). The code is remarkably short: the update loop is `while (i <= N) { bit[i] += val; i += i & -i; }` and the query is `while (i > 0) { sum += bit[i]; i -= i & -i; }.`"

## 19. Revision Notes

- **BIT = Fenwick Tree = Binary Indexed Tree**
- **1-indexed.** Convert 0-index to 1-index by adding 1.
- **Update:** `i += i & -i`. Add delta to all covering ranges.
- **Query:** `i -= i & -i`. Sum non-overlapping ranges.
- **Range sum:** `prefix(r) - prefix(l-1)`
- **Inversion count:** compress values, BIT as freq array, `inv += i - prefix(freq[val])`
- **Lower bound:** binary lifting on BIT (all values  $\geq 0$ )
- **Range add + point query:** `add(1, delta), add(r+1, -delta)`, then `prefix(i)`
- **Complexity:**  $O(\log N)$  per op,  $O(N)$  space
- **Cannot do:** range min/max, range assign

## 20. Final Cheat Sheet

FENWICK TREE (BIT) — Prefix sums in  $O(\log N)$

USE: prefix sum, range sum, inversion count, frequency, order statistics

UPDATE: `while (i <= N) bit[i] += delta; i += i & -i`

QUERY: `while (i > 0) sum += bit[i]; i -= i & -i`

RANGE SUM: `prefix(r) - prefix(l-1)`

SPACE:  $O(N)$       TIME:  $O(\log N)$  per op  
 TRAPS: 1-indexed, no min/max, coordinate compress large values  
 KEY:  $LSB = i \& -i$

# 11. STRING MATCHING

## 1. Overview

**String matching** (pattern matching) algorithms find all occurrences of a pattern string  $P$  (length  $M$ ) in a text string  $T$  (length  $N$ ). Various algorithms trade off preprocessing time, matching time, and ease of implementation — from naive  $O(N \times M)$  to optimal  $O(N + M)$  using KMP, Z-algorithm, or Rabin-Karp.

## 2. Intuition

**Simple explanation:** Given a text "AABAACAADAABAABA" and a pattern "AABA", find every position where the pattern appears in the text.

**Analogy:** KMP is like having a cheat sheet that tells you how far to shift the pattern when a mismatch occurs, instead of starting from scratch. The trick is that the pattern may contain repeated prefixes — if we match "AAB" and then fail on the 4th character, we already know that "A" (the prefix of "AAB") matches the end of our current match, so we can skip ahead.

**Why they work:**

- **KMP:** Preprocesses the pattern to build a "failure function" (LPS array) that tells us the longest proper prefix of the pattern that is also a suffix of the matched part. On mismatch, instead of shifting by 1, we shift by `matched_length - lps[matched_length-1]`.
- **Z-algorithm:** Builds the Z-array (longest substring starting at  $i$  that matches prefix) for the concatenated string  $P + '$' + T$ . If  $Z[i] == M$ , we found a match.
- **Rabin-Karp:** Computes a rolling hash of the pattern and of each  $M$ -length window in the text. If hashes match, verify with direct comparison (to handle hash collisions).

## 3. When to Use It

- **Pattern search** — "find all occurrences of pattern in text"
- **String matching in editors** — Ctrl+F / search functionality
- **Plagiarism detection** — find common substrings
- **DNA sequence matching**
- **Detect repeated substrings** — Z-algorithm variant
- **Palindromic substring problems** — Manacher's algorithm (variation)
- **Multiple patterns** — Aho-Corasick (trie + failure links)

**Trigger phrases:** "pattern matching", "find substring", "search in string", "matching pattern", "KMP", "Rabin-Karp", "Z-algorithm", "string search"

## 4. When Not to Use It

- **Single occurrence, small N** — `T.find(P)` or `std::search` is good enough
- **Multiple patterns with one text** — Aho-Corasick is better than running KMP for each pattern
- **Need substring count, not position** — suffix automaton may be better
- **Vague/approximate matching** — regex, edit distance, or fuzzy matching

**Common wrong assumption:** "KMP is always the fastest." For random text with short patterns, the naive algorithm (with break on mismatch) is often faster in practice due to CPU prefetching and branch prediction.

## 5. Core Concepts

### 5.1 LPS Array (Longest Proper Prefix that is also Suffix)

- `lps[i]` = length of the longest proper prefix of `P[0..i]` that is also a suffix
- Proper prefix means prefix shorter than the whole string
- Used in KMP to determine how far to shift on mismatch

### 5.2 Z-Array

- `Z[i]` = length of the longest substring starting at position `i` that matches the prefix `P[0..]`
- Computed for the combined string `P + '$' + T`
- Built using the Z-box technique in  $O(N+M)$

### 5.3 Rolling Hash (Rabin-Karp)

- Hash of a sliding window computed incrementally:  $\text{hash} = (\text{hash} * \text{base} - \text{old\_char} * \text{base}^M + \text{new\_char}) \% \text{mod}$
- Or polynomial hash:  $\text{hash}(s) = (s[0]*B^{(M-1)} + s[1]*B^{(M-2)} + \dots + s[M-1]*B^0) \% \text{mod}$
- Double hashing reduces collision probability

### 5.4 Prefix Function (Pi)

- `pi[i]` = length of the longest proper prefix of `S[0..i]` that is also a suffix
- Generalization of lps for any string, not just pattern
- Foundation for KMP and prefix-based string algorithms

## 6. Step-by-Step Algorithm

### KMP Algorithm

#### Compute LPS array:

1. `lps[0] = 0, len = 0, i = 1`
2. While `i < M`:
  - If `P[i] == P[len]`: `len++, lps[i] = len, i++`
  - Else if `len > 0`: `len = lps[len-1]`
  - Else: `lps[i] = 0, i++`

#### Matching:

1. `i = 0` (index in text), `j = 0` (index in pattern)
2. While `i < N`:

- If  $T[i] == P[j]$ :  $i++$ ,  $j++$ . If  $j == M$ : match found at  $i - j$ ,  $j = lps[j-1]$
- Else if  $j > 0$ :  $j = lps[j-1]$
- Else:  $i++$

## Z-Algorithm

**Compute Z-array (for  $S = P + '$' + T$ ):**

1.  $l = 0$ ,  $r = 0$  (Z-box boundaries)
2. For  $i = 1$  to  $L-1$ :
  - If  $i > r$ : compute  $Z[i]$  by direct comparison
  - Else:  $k = i - l$ ,  $Z[i] = \min(Z[k], r - i + 1)$ . Extend if possible.
  - If  $i + Z[i] - 1 > r$ : update  $l = i$ ,  $r = i + Z[i] - 1$
3. If  $Z[i] == M$ : match at position  $i - M - 1$  in  $T$

## Rabin-Karp

1. Compute hash of pattern:  $hashP$
2. Compute hash of first window of text:  $hashT$
3. For  $i = 0$  to  $N - M$ :
  - If  $hashT == hashP$  and text matches pattern at  $i$ : record match
  - Update hash for next window: remove  $T[i]$ , add  $T[i+M]$
4. Use double hash (two mod values) to reduce collisions

## 7. Dry Run

KMP:  $P = "ABABAC"$ ,  $T = "ABABABCABABAC"$

**LPS for "ABABAC":**

| i | P[i] | len (before) | Match?     | len (after)    | lps[i] |
|---|------|--------------|------------|----------------|--------|
| 0 | A    | 0            | -          | 0              | 0      |
| 1 | B    | 0            | $A \neq B$ | 0              | 0      |
| 2 | A    | 0            | $A=A$ ✓    | 1              | 1      |
| 3 | B    | 1            | $B=B$ ✓    | 2              | 2      |
| 4 | A    | 2            | $A=A$ ✓    | 3              | 3      |
| 5 | C    | 3            | $C \neq B$ | $len=lps[2]=1$ |        |
| 5 | C    | 1            | $C \neq B$ | $len=lps[0]=0$ |        |
| 5 | C    | 0            | $C \neq A$ | 0              | 0      |

**lps = [0, 0, 1, 2, 3, 0]**

**Matching "ABABAC" in "ABABABCABABAC":**

| i  | j | T[i] | P[j] | Action                            |
|----|---|------|------|-----------------------------------|
| 0  | 0 | A    | A    | match, i=1, j=1                   |
| 1  | 1 | B    | B    | match, i=2, j=2                   |
| 2  | 2 | A    | A    | match, i=3, j=3                   |
| 3  | 3 | B    | B    | match, i=4, j=4                   |
| 4  | 4 | A    | A    | match, i=5, j=5                   |
| 5  | 5 | B    | C    | mismatch, j = lps[4]=3            |
| 5  | 3 | B    | B    | match, i=6, j=4                   |
| 6  | 4 | C    | A    | mismatch, j = lps[3]=2            |
| 6  | 2 | C    | A    | mismatch, j = lps[1]=0            |
| 6  | 0 | C    | A    | mismatch, i=7, j=0                |
| 7  | 0 | A    | A    | match, i=8, j=1                   |
| 8  | 1 | B    | B    | match, i=9, j=2                   |
| 9  | 2 | A    | A    | match, i=10, j=3                  |
| 10 | 3 | B    | B    | match, i=11, j=4                  |
| 11 | 4 | A    | A    | match, i=12, j=5                  |
| 12 | 5 | C    | C    | match, j=6 == M → found at 12-6=7 |

Match at position 7 (0-indexed). "ABABAC" starts at index 7 in T.

Text: A B A B A B C A B A B A C Pat: A B A B A C at index 7 ✓

## 8. C++ Implementation

### KMP Algorithm

```
#include <bits/stdc++.h>
using namespace std;

vector<int> computeLPS(const string& pattern) {
    int M = pattern.size();
    vector<int> lps(M, 0);
    int len = 0; // length of previous longest prefix suffix
    int i = 1;

    while (i < M) {
        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
            i++;
        }
    }
}
```

```

        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
}
return lps;
}

vector<int> KMPSearch(const string& text, const string& pattern) {
    vector<int> matches;
    int N = text.size(), M = pattern.size();
    if (M == 0) return matches;

    vector<int> lps = computeLPS(pattern);

    int i = 0; // index for text
    int j = 0; // index for pattern

    while (i < N) {
        if (text[i] == pattern[j]) {
            i++;
            j++;
        }

        if (j == M) {
            matches.push_back(i - j);
            j = lps[j - 1];
        } else if (i < N && text[i] != pattern[j]) {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }
    return matches;
}

```

## Z-Algorithm

```

vector<int> computeZ(const string& s) {
    int L = s.size();
    vector<int> Z(L, 0);
    int l = 0, r = 0;

    for (int i = 1; i < L; ++i) {
        if (i > r) {

```



```

        l = r = i;
        while (r < L && s[r] == s[r - 1]) r++;
        Z[i] = r - 1;
        r--;
    } else {
        int k = i - 1;
        if (Z[k] < r - i + 1) {
            Z[i] = Z[k];
        } else {
            l = i;
            while (r < L && s[r] == s[r - 1]) r++;
            Z[i] = r - 1;
            r--;
        }
    }
}
return Z;
}

vector<int> ZSearch(const string& text, const string& pattern) {
    string combined = pattern + '$' + text;
    vector<int> Z = computeZ(combined);
    vector<int> matches;
    int M = pattern.size();

    for (int i = M + 1; i < (int)combined.size(); ++i) {
        if (Z[i] == M) {
            matches.push_back(i - M - 1);
        }
    }
    return matches;
}

```

## Rabin-Karp

```

vector<int> rabinKarp(const string& text, const string& pattern) {
    const int BASE = 31;
    const int MOD = 1e9 + 7;
    int N = text.size(), M = pattern.size();
    if (M > N) return {};

    // Precompute powers of base
    vector<long long> powB(N + 1, 1);
    for (int i = 1; i <= N; ++i) powB[i] = (powB[i-1] * BASE) % MOD;

    // Hash of pattern
    long long hashP = 0;
    for (int i = 0; i < M; ++i)
        hashP = (hashP * BASE + (pattern[i] - 'a' + 1)) % MOD;

    // Hash of all prefixes of text

```

```

vector<long long> hashT(N + 1, 0);
for (int i = 0; i < N; ++i)
    hashT[i + 1] = (hashT[i] * BASE + (text[i] - 'a' + 1)) % MOD;

vector<int> matches;
for (int i = 0; i + M <= N; ++i) {
    // Hash of substring text[i..i+M-1]
    long long subHash = (hashT[i + M] - hashT[i] * powB[M] % MOD + MOD) % MOD;
    if (subHash == hashP) {
        // Verify (optional – good enough with large MOD)
        if (text.substr(i, M) == pattern) {
            matches.push_back(i);
        }
    }
}
return matches;
}

```

## Example Usage

```

int main() {
    string text = "ABABABCABABAC";
    string pattern = "ABABAC";

    auto kmp = KMPSearch(text, pattern);
    cout << "KMP matches: ";
    for (int pos : kmp) cout << pos << " ";
    cout << endl;

    auto z = ZSearch(text, pattern);
    cout << "Z matches: ";
    for (int pos : z) cout << pos << " ";
    cout << endl;

    auto rk = rabinKarp(text, pattern);
    cout << "Rabin-Karp matches: ";
    for (int pos : rk) cout << pos << " ";
    cout << endl;

    return 0;
}

```

## 9. Python Implementation

```

def compute_lps(pattern):
    M = len(pattern)
    lps = [0] * M
    length = 0
    i = 1

```

```

while i < M:
    if pattern[i] == pattern[length]:
        length += 1
        lps[i] = length
        i += 1
    else:
        if length != 0:
            length = lps[length - 1]
        else:
            lps[i] = 0
            i += 1
return lps

def kmp_search(text, pattern):
    if not pattern:
        return []
    N, M = len(text), len(pattern)
    lps = compute_lps(pattern)
    matches = []
    i = j = 0
    while i < N:
        if text[i] == pattern[j]:
            i += 1
            j += 1
        if j == M:
            matches.append(i - j)
            j = lps[j - 1]
        elif i < N and text[i] != pattern[j]:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return matches

def compute_z(s):
    L = len(s)
    Z = [0] * L
    l = r = 0
    for i in range(1, L):
        if i > r:
            l = r = i
            while r < L and s[r] == s[r - 1]:
                r += 1
            Z[i] = r - l
            r -= 1
        else:
            k = i - l
            if Z[k] < r - i + 1:
                Z[i] = Z[k]
            else:
                l = i
                while r < L and s[r] == s[r - 1]:
                    r += 1
                Z[i] = r - l

```

```

        r -= 1

    return Z

def z_search(text, pattern):
    combined = pattern + '$' + text
    Z = compute_z(combined)
    M = len(pattern)
    return [i - M - 1 for i in range(M + 1, len(combined)) if Z[i] == M]

def rabin_karp(text, pattern):
    BASE, MOD = 31, 10**9 + 7
    N, M = len(text), len(pattern)
    if M > N:
        return []

    powB = [1] * (N + 1)
    for i in range(1, N + 1):
        powB[i] = (powB[i-1] * BASE) % MOD

    hashP = 0
    for ch in pattern:
        hashP = (hashP * BASE + (ord(ch) - ord('a') + 1)) % MOD

    hashT = [0] * (N + 1)
    for i, ch in enumerate(text):
        hashT[i + 1] = (hashT[i] * BASE + (ord(ch) - ord('a') + 1)) % MOD

    matches = []
    for i in range(N - M + 1):
        sub_hash = (hashT[i + M] - hashT[i] * powB[M]) % MOD
        if sub_hash == hashP:
            if text[i:i+M] == pattern:
                matches.append(i)
    return matches

# Example
text = "ABABABCABABAC"
pattern = "ABABAC"
print("KMP:", kmp_search(text, pattern))
print("Z:", z_search(text, pattern))
print("RK:", rabin_karp(text, pattern))

```

## 10. Code Explanation

### KMP — LPS Construction

**Key insight:** The LPS array tells us the longest proper prefix of  $P[0..i]$  that is also a suffix. This is computed iteratively.

- If  $P[i] == P[len]$ : extend the current match, increment len.
- If mismatch  $P[i] != P[len]$  and  $len > 0$ : fall back to  $lps[len-1]$ . This is the crucial backtrack — we've matched a prefix of length len, but failed to extend. The next best candidate is  $lps[len-1]$ .

- Example: For "ABABAC", at  $i=4$  we have matched "ABA" ( $\text{len}=3$ ).  $P[4]='A'$  and  $P[3]='A'$  both... wait, let me re-read. At  $i=4$ ,  $\text{len}=3$  (from  $i=3$ ,  $P[3]='B'$ ,  $\text{len}$  became 3 with  $\text{lps}[3]=3$ ... hmm, actually let's re-check.)

Actually above was giving  $\text{lps} = [0,0,1,2,3,0]$ . Let me re-verify:

- $i=0$ :  $\text{lps}[0]=0$
- $i=1$ :  $P[1]='B'$ ,  $P[0]='A'$ , mismatch,  $\text{len}=0$ ,  $\text{lps}[1]=0$
- $i=2$ :  $P[2]='A'$ ,  $P[0]='A'$ , match,  $\text{len}=1$ ,  $\text{lps}[2]=1$
- $i=3$ :  $P[3]='B'$ ,  $P[1]='B'$ , match,  $\text{len}=2$ ,  $\text{lps}[3]=2$
- $i=4$ :  $P[4]='A'$ ,  $P[2]='A'$ , match,  $\text{len}=3$ ,  $\text{lps}[4]=3$
- $i=5$ :  $P[5]='C'$ ,  $P[3]='B'$ , mismatch.  $\text{len}=\text{lps}[2]=1$ .  $P[5]='C'$ ,  $P[1]='B'$  Mismatch.  $\text{len}=\text{lps}[0]=0$ .  $P[5]='C'$ ,  $P[0]='A'$  Mismatch.  $\text{lps}[5]=0$ .

Yes,  $\text{lps} = [0,0,1,2,3,0]$  ✓

## KMP — Matching

When a mismatch occurs at position  $j$  in pattern, we set  $j = \text{lps}[j-1]$ . This is because we've already matched  $P[0..j-1]$ , and the longest prefix of  $P$  that is also a suffix of  $P[0..j-1]$  is  $\text{lps}[j-1]$  characters long.

When a full match occurs ( $j == M$ ), we record it and set  $j = \text{lps}[j-1]$  to continue searching for overlapping matches.

## Z-Algorithm

The Z-box  $[l, r]$  maintains the rightmost substring that matches the prefix. For each position  $i$ :

- If  $i$  is outside the box, compute  $Z[i]$  from scratch.
- If  $i$  is inside the box, we can use precomputed  $Z[k]$  (where  $k = i - l$ ) to skip ahead.
- $Z[i]$  is bounded by the remaining distance to  $r$ :  $Z[i] = \min(Z[k], r - i + 1)$ .

## Rabin-Karp

**Rolling hash:** Using base  $B$  and large prime  $\text{MOD}$ , we compute the hash of each window incrementally.

The prefix hash approach: precompute hash of all prefixes, then get substring hash as  $\text{hash}[1..r] = (\text{hash}[r+1] - \text{hash}[1] * B^{(r-1+1)}) \% \text{MOD}$ .

**Collision:** With a single  $\text{MOD}$ , collisions are possible (but rare for  $\text{MOD} \approx 10^9+7$ ). Double hashing (two  $\text{MOD}$  values) or character-by-character verification adds safety.

# 11. Complexity Analysis

| Algorithm  | Preprocess | Matching        | Overall         | Space    |
|------------|------------|-----------------|-----------------|----------|
| Naive      | $O(1)$     | $O(N \times M)$ | $O(N \times M)$ | $O(1)$   |
| KMP        | $O(M)$     | $O(N)$          | $O(N+M)$        | $O(M)$   |
| Z-algo     | $O(N+M)$   | $O(N+M)$        | $O(N+M)$        | $O(N+M)$ |
| Rabin-Karp | $O(M)$     | $O(N+M)$ avg    | $O(N+M)$ avg    | $O(N)$   |

| Aspect                 | Complexity      |
|------------------------|-----------------|
| N = text length        |                 |
| M = pattern length     |                 |
| KMP (best/worst)       | $O(N+M)$        |
| RB (average)           | $O(N+M)$        |
| RB (worst, collisions) | $O(N \times M)$ |

## 12. Common Patterns

### Pattern 1: Find All Occurrences

**Identify:** "Find all positions where pattern appears in text"

**Approach:** KMP, Z-algorithm, or Rabin-Karp.

**Problems:** LeetCode 28 (Find the Index of the First Occurrence in a String)

### Pattern 2: Count Distinct Substrings

**Identify:** "How many distinct substrings does a string have?"

**Approach:** Build all suffixes, add each to prefix tree. Or use Z-algorithm / suffix array.

**Problems:** CSES Distinct Substrings

### Pattern 3: Longest Palindromic Prefix/Suffix

**Identify:** "Longest prefix that is also a palindrome" or "Shortest palindrome by adding chars to front"

**Approach:** Combine string + '#' + reverse, compute LPS/Z.

**Problems:** LeetCode 214 (Shortest Palindrome)

### Pattern 4: Repeated Substring Pattern

**Identify:** "Is the string formed by repeating a substring?"

**Approach:**  $S = T + T$ , remove first and last char, check if T is in S. Or use LPS: check if  $lps[N-1] > 0 \ \&\& \ N \% (N - lps[N-1]) == 0$ .

**Problems:** LeetCode 459 (Repeated Substring Pattern)

### Pattern 5: Anagram Substring Search

**Identify:** "Find all positions where pattern (or its anagram) occurs"

**Approach:** Sliding window with frequency count. Not KMP, but often confused.

**Problems:** LeetCode 438 (Find All Anagrams in a String)

### Pattern 6: Multiple Pattern Search (Aho-Corasick)

**Identify:** "Find all occurrences of many patterns in one text"

**Approach:** Build trie of patterns + failure links (like LPS for trie).

**Problems:** LeetCode 1032 (Stream of Characters), Codeforces problems

## 13. Common Mistakes

- **LPS array indexing** —  $j = \text{lps}[j-1]$  uses  $j-1$ , not  $j$ . Must check  $j > 0$  first.
- **Off-by-one in Z-algorithm** — combined string is  $P + \text{sep} + T$ ; matches are at  $Z[i] == M$  and  $\text{position} = i - M - 1$ .
- **Not handling empty pattern** — return empty or all positions depending on problem.
- **Hash overflow** — use `long long` and `% MOD` carefully in C++.
- **Hash collision** — verify match with direct comparison after hash match.
- **Negative hash values** —  $(\text{hashT}[i+M] - \text{hashT}[i] * \text{powB}[M] \% \text{MOD} + \text{MOD}) \% \text{MOD}$  — add MOD before %.
- **Rabin-Karp base/power** — base should be  $\geq$  alphabet size; power of base must be precomputed.

## 14. Edge Cases

- **Empty pattern** → match at every position or none (depends on spec)
- **Empty text** → no matches (unless pattern also empty)
- **Pattern longer than text** → no matches
- **Pattern = text** → match at position 0
- **All same characters** → "aaaa" in "aaaaaaaa" → multiple overlapping matches
- **No match** → empty result
- **Pattern at the very end of text** → should be found
- **Overlapping matches** → KMP handles these (lps allows overlap)
- **Case sensitivity** — typically case-sensitive; adjust if needed

## 15. Variations

### 1. Z-Algorithm (for single pattern search)

- Concatenate  $P + '$' + T$ , compute Z-array.
- Positions where  $Z[i] == M$  are matches.
- Simpler than KMP, used in many CP problems.

### 2. Rabin-Karp (for multiple pattern search)

- Compute hashes of all patterns.
- Slide window through text, check if hash matches any pattern.
- Good for "dictionary" of patterns.

### 3. Prefix Function (generalization of LPS)

- Works for any string, returns array  $\text{pi}$ .
- Used for string periodicity, border computation.
- KMP's LPS is the prefix function of the pattern.

### 4. Suffix Array + LCP

- For more advanced string problems (longest common substring, repeated substrings).
- $O(N \log N)$  build,  $O(1)$  LCP queries.

### 5. Manacher's Algorithm

- Find all palindromic substrings in  $O(N)$ .
- Related: palindrome problems often use string matching ideas.

## 6. Aho-Corasick

- Multiple pattern matching: trie + failure links (like KMP on a trie).
- $O(N + \text{total pattern length})$  for matching.

## 16. Related Algorithms/Data Structures

| Algorithm/DS            | Connection                                                                 |
|-------------------------|----------------------------------------------------------------------------|
| <b>Trie</b>             | For prefix search. Aho-Corasick builds on trie + failure links (like KMP). |
| <b>Suffix Array</b>     | For substring existence, counting, LCP queries.                            |
| <b>Suffix Automaton</b> | Minimal DFA for all substrings. $O(N)$ build. Powerful.                    |
| <b>Rolling Hash</b>     | Foundation of Rabin-Karp. Also used in many other problems.                |
| <b>Sliding Window</b>   | Not a string matching algo, but works for anagram search.                  |

## 17. Practice Problems

### Easy

1. **Find the Index of the First Occurrence in a String** — LeetCode 28  
*KMP or two-pointer/naive for simplicity.*
2. **Repeated Substring Pattern** — LeetCode 459  
*LPS-based: check if  $\text{lps}[N-1] > 0 \ \&\& \ N \% (N - \text{lps}[N-1]) == 0$ .*

### Medium

1. **Shortest Palindrome** — LeetCode 214  
*KMP on  $s + \text{'\#'} + \text{reverse}(s)$  to find longest palindromic prefix.*
2. **Find All Anagrams in a String** — LeetCode 438  
*Sliding window + frequency, not KMP. But commonly grouped.*
3. **String Matching in a Stream** — LeetCode 1032  
*Trie + failure links (Aho-Corasick).*

### Hard

1. **Longest Happy Prefix** — LeetCode 1392  
*LPS/Z-value approach. A happy prefix is a non-empty prefix that's also a suffix.*
2. **Distinct Substrings** — Codeforces / SPOJ / CSES  
*Suffix array + LCP or suffix automaton.*

## 18. Interview Explanation

"For exact string matching, I use KMP, Z-algorithm, or Rabin-Karp depending on the constraints. KMP precomputes an LPS array for the pattern that tells us, on mismatch, how far to shift — avoiding



redundant comparisons. Both preprocessing and matching are  $O(N+M)$ . The core idea is: if we've matched a prefix of the pattern and then fail, the longest prefix that is also a suffix of the matched part tells us where to resume.

Z-algorithm is my preferred alternative: I concatenate pattern + '\$' + text, compute the Z-array (longest substring starting at  $i$  that matches the prefix), and check where  $Z[i]$  equals pattern length. It's simpler to implement than KMP.

Rabin-Karp uses rolling hash — it's  $O(N+M)$  on average but  $O(N \times M)$  worst-case due to collisions. I'd use it when I need to match multiple patterns simultaneously.

For multiple patterns, I'd use Aho-Corasick, which extends KMP's failure function to a trie of patterns."

## 19. Revision Notes

- **KMP:** LPS array  $\rightarrow$  on mismatch,  $j = \text{lps}[j-1]$ . Preprocess  $O(M)$ , match  $O(N)$ .
- **Z-algo:** Z-array for  $P + '$' + T$ . Match when  $Z[i] == M$ .  $O(N+M)$ .
- **Rabin-Karp:** Rolling hash. Hash collision  $\rightarrow$  verify.  $O(N+M)$  avg.
- **LPS recurrence:** `if P[i]==P[len]: len++, lps[i]=len, i++. Else if len>0: len=lps[len-1]. Else: lps[i]=0, i++`
- **Key trap:**  $j = \text{lps}[j-1]$  with  $j-1$  bound check; empty pattern; hash overflow

## 20. Final Cheat Sheet

STRING MATCHING – Find pattern in text,  $O(N+M)$

KMP: LPS array  $\rightarrow$  on mismatch,  $j = \text{lps}[j-1]$  (not  $j = j-1!$ )

Z:  $P + '$' + T \rightarrow Z[i] == M \rightarrow$  match at  $i-M-1$  in  $T$

RK: Rolling hash:  $\text{subHash} = (\text{hash}[l..r+1] - \text{hash}[l] * B^{(len)}) \% \text{MOD}$   
Verify on hash match (collisions!)

TIME:  $O(N+M)$  (KMP, Z),  $O(N+M)$  avg (RK)      SPACE:  $O(M)$  or  $O(N+M)$

TRAPS: lps indexing ( $j-1$ ), hash overflow, empty pattern, overlapping matches

## 12. DIGIT DP

### 1. Overview

**Digit DP** is a dynamic programming technique used to count numbers satisfying specific constraints within a range  $[L, R]$ , where the numbers are processed digit by digit. Typically, the state includes the current position in the digit string, a tight flag, and any problem-specific constraints.

### 2. Intuition

**Simple explanation:** Instead of iterating through every number in a range (impossible for large ranges like  $10^{18}$ ), you count numbers digit by digit using DP. The key is that numbers with the same "prefix" and same remaining length behave identically.

**Analogy:** Think of building a number from left to right, digit by digit. At each position, you decide which digit to place (0-9), subject to constraints like "no consecutive same digits" or "sum of digits  $\leq K$ ". Two partially-

built numbers are in the same DP state if they have the same remaining positions and the same constraints on the already-built prefix.

**Why it works:** Numbers up to  $10^N$  have  $N$  digits. By processing positions from most significant to least, we only need to track: (1) current position, (2) tight flag (is our prefix equal to the upper bound's prefix so far?), (3) started flag (have we placed a non-zero digit yet?), plus problem-specific info.

### 3. When to Use It

- **Count numbers in  $[L, R]$  with specific digit properties**
- **Sum of digits of all numbers in a range** (digit sum)
- **Numbers with no consecutive same digits**
- **Numbers divisible by  $K$**  (modulo in state)
- **Numbers with at most  $K$  distinct digits**
- **Numbers where digit sum / product satisfies a condition**
- **Palindromic numbers** (sometimes)
- **Count of numbers where specific digit doesn't appear**

**Trigger phrases:** "count numbers in range  $[L, R]$  such that...", "digit DP", "digit constraints", "digit sum", "number with property", "between A and B"

### 4. When Not to Use It

- **Range is small ( $\leq 10^6$ )** — simple iteration is sufficient
- **Problem involves arithmetic operations on the whole number** — normal DP or math may be better
- **The property depends on the full number, not its digits** — e.g., prime numbers in a range (use sieve)
- **No digit-level constraint** — just  $R - L + 1$
- **Need to generate the actual numbers** — DP is for counting

**Common wrong assumption:** "Digit DP works for all range counting problems." It works only when the condition can be checked digit by digit with limited state.

## 5. Core Concepts

### 5.1 Digit Representation

- The number is represented as a string of digits (left to right)
- Leading zeros are handled with a **started** flag

### 5.2 Tight Flag

- **tight = true** means the prefix so far equals the upper bound's prefix
- When tight, the current digit is limited to  $0..bound[pos]$
- When not tight, the current digit can be  $0..9$

### 5.3 Started Flag

- **started = false** means we haven't placed any non-zero digit yet
- This handles leading zeros correctly (e.g., "0015" should be counted as "15")
- Once we place a non-zero digit, **started = true**

## 5.4 State

- Typical: `dp[pos][tight][started][additional...]`
- Additional: `sum`, `mod`, `lastDigit`, `countDistinct`, etc.
- State dimensions must be small ( $\leq 10^6$ ) for memoization

## 5.5 Recursive DP with Memoization (most natural for digit DP)

```
long long DP(int pos, bool tight, bool started, ...) {
    if (pos == N) return ... ? 1 : 0; // base
    if (dp[pos][tight][started][...] != -1) return ...;
    int limit = tight ? digits[pos] : 9;
    long long ans = 0;
    for (int d = 0; d <= limit; ++d) {
        bool ntight = tight && (d == limit);
        bool nstarted = started || (d != 0);
        ans += DP(pos+1, ntight, nstarted, ...);
    }
    return dp[pos][tight][started][...] = ans;
}
```

## 6. Step-by-Step Algorithm

### General Digit DP

1. Convert R (upper bound) to a digit string `s`
2. Define DP function `f(pos, tight, started, ...)`:
  - Base: `pos == len(s) → return 1` if valid (e.g., `started` is true to count only positive numbers)
  - Memoize on state
  - `limit = tight ? s[pos] - '0' : 9`
  - For each digit `d = 0..limit`:
    - `ntight = tight && (d == limit)`
    - `nstarted = started || (d != 0)`
    - Recurse with updated state
3. `countUpTo(R) = f(0, true, false)`
4. `countInRange(L, R) = countUpTo(R) - countUpTo(L-1)`

Example: Count numbers  $\leq R$  with digit sum  $\leq 10$

State: `dp[pos][tight][started][sum]`

- `sum`: digit sum so far, up to 10 (or larger)
- If `sum > 10`, prune (return 0)
- Base: return 1 if `started` is true (we count the number)

Example: Count numbers with no consecutive same digits

State: `dp[pos][tight][started][lastDigit]`

- lastDigit: the last non-zero digit placed (or -1 if not started)
- At each step, skip digit d if `d == lastDigit`
- Special handling for leading zeros: if not started, lastDigit doesn't matter

## 7. Dry Run

Problem: Count numbers from 1 to 25 where digit sum  $\leq 5$

`s = "25"`, `N = 2`

Let's trace DP(0, tight=true, started=false, sum=0):

**pos=0, tight=true, limit=2**

- `d=0`: `ntight=(0==2)→false`, `nstarted=false`
  - `pos=1, tight=false, started=false, sum=0`
    - `limit=9, started=false`
    - `d=0`: `nstarted=false, pos=2 → started=false → count?` Usually we count only if started. Let's say we want counting from 1, so return 0 for unstated.
    - `d=1`: `nstarted=true, sum=1, pos=2 → count=1` (number 1)
    - `d=2`: `nstarted=true, sum=2 → count=1` (number 2)
    - ... `d=5`: `nstarted=true, sum=5 → count=1` (number 5)
    - `d=6-9`: `sum > 5`, prune
    - Total from `d=0`: 5 numbers (1,2,3,4,5)
- `d=1`: `ntight=(1==2)→false`, `nstarted=true, sum=1`
  - `pos=1, tight=false, started=true, sum=1`
    - `limit=9, started=true`
    - `d=0`: `sum=1+0=1, pos=2 → count=1` (10)
    - `d=1`: `sum=2 → count=1` (11)
    - `d=2`: `sum=3 → count=1` (12)
    - `d=3`: `sum=4 → count=1` (13)
    - `d=4`: `sum=5 → count=1` (14)
    - `d=5-9`: `sum > 5`, prune
    - Total: 5 numbers (10,11,12,13,14)
- `d=2`: `ntight=(2==2)→true`, `nstarted=true, sum=2`
  - `pos=1, tight=true, started=true, sum=2`
    - `limit=5` (tight, bound digit = 5)
    - `d=0`: `sum=2, pos=2 → count=1` (20)
    - `d=1`: `sum=3 → count=1` (21)
    - `d=2`: `sum=4 → count=1` (22)
    - `d=3`: `sum=5 → count=1` (23)
    - `d=4`: `sum=6 > 5 → prune`
    - `d=5`: `sum=7 > 5 → prune`
    - Total: 4 numbers (20,21,22,23)

**Total:**  $5 + 5 + 4 = 14$  numbers from 1 to 25 with digit sum  $\leq 5$ .

(Verification: 1,2,3,4,5,10,11,12,13,14,20,21,22,23 — 14 numbers ✓)

## 8. C++ Implementation

### Generic Digit DP Template

```
#include <bits/stdc++.h>
using namespace std;

// Count numbers in [L, R] with digit sum  $\leq K$ 
class DigitDP {
private:
    string s;
    int N, K;
    long long dp[20][2][2][100]; // pos, tight, started, sum

    long long solve(int pos, bool tight, bool started, int sum) {
        if (sum > K) return 0; // prune
        if (pos == N) return started ? 1 : 0; // count only if started

        if (dp[pos][tight][started][sum] != -1)
            return dp[pos][tight][started][sum];

        int limit = tight ? (s[pos] - '0') : 9;
        long long ans = 0;

        for (int d = 0; d <= limit; ++d) {
            bool ntight = tight && (d == limit);
            bool nstarted = started || (d != 0);
            int nsum = sum + d;
            ans += solve(pos + 1, ntight, nstarted, nsum);
        }

        return dp[pos][tight][started][sum] = ans;
    }

public:
    long long countUpTo(long long R) {
        s = to_string(R);
        N = s.size();
        memset(dp, -1, sizeof(dp));
        return solve(0, true, false, 0);
    }

    long long countRange(long long L, long long R) {
        return countUpTo(R) - countUpTo(L - 1);
    }
};
```

## Count numbers with no consecutive same digits

```

class NoConsecutive {
private:
    string s;
    int N;
    long long dp[20][2][2][10]; // pos, tight, started, lastDigit (0-9, last=10
means none)

    long long solve(int pos, bool tight, bool started, int last) {
        if (pos == N) return started ? 1 : 0;
        if (dp[pos][tight][started][last] != -1)
            return dp[pos][tight][started][last];

        int limit = tight ? (s[pos] - '0') : 9;
        long long ans = 0;

        for (int d = 0; d <= limit; ++d) {
            if (started && d == last) continue; // skip consecutive same
            bool ntight = tight && (d == limit);
            bool nstarted = started || (d != 0);
            int nlast = nstarted ? d : 10; // 10 = "no previous digit"
            ans += solve(pos + 1, ntight, nstarted, nlast);
        }

        return dp[pos][tight][started][last] = ans;
    }

public:
    long long countUpTo(long long R) {
        s = to_string(R);
        N = s.size();
        memset(dp, -1, sizeof(dp));
        return solve(0, true, false, 10);
    }

    long long countRange(long long L, long

```